On Best-Possible One-Time Programs

Aparna Gupte
MIT

Jiahui Liu
Fujitsu Research

Luowen Qian
NTT Research

Justin Raizes
NTT Research

Bhaskar Roberts
University of California, Berkeley

Mark Zhandry
Stanford University

Abstract

One-time programs (OTPs) aim to let a user evaluate a program on a single input while revealing nothing else. Classical OTPs require hardware assumptions, and even with quantum information, deterministic functionalities remain impossible due to gentle-measurement attacks (Broadbent, Gutoski and Stebila, 2013). While recent works achieve positive results for randomized functionalities with high-entropy outputs, the fundamental limits and the strongest achievable security notions remain poorly understood.

Inspired by analogous successes in the classical obfuscation setting, we ask for a "best-possible" analogue of obfuscation for OTPs: a generic transformation that, for any functionality, achieves the strongest one-time security achievable by any construction. Our first result is negative. We show that a generic best-possible one-time compiler cannot exist even for classical randomized functionalities. We prove this under the assumption that lossy encryption schemes exist (e.g. from either the Learning with Errors or weakly pseudorandom group actions). Our proof identifies computationally indistinguishable families for which any best-possible transformation would be forced to behave incompatibly.

Given this impossibility, we introduce a natural subclass of one-time compilers called "testable one-time program" compilers, which output quantum states augmented with reflection oracles for themselves. We show that best-possible security for this subclass, i.e. best-possible *testable* one-time compilers, are most likely achievable. For this, we give two results. (1) We formulate a simplified, generalized *Single-Effective-Query (SEQ)* simulation security notion for quantum channels and show that SEQ security implies best-possible testable one-time security. (2) We construct SEQ-secure OTPs for *all* quantum functionalities in the classical oracle model, yielding the first positive results for arbitrary quantum channels beyond classical randomized functionalities. Thus, SEQ security could serve as a testable one-time analogue of virtual black-box (VBB) security in the many-time obfuscation setting.

Finally, we propose stateful quantum indistinguishability obfuscation (stateful quantum iO) — quantum state obfuscation for stateful quantum programs. We show that (1) stateful quantum iO implies best-possible testable OTPs and (2) stateful quantum iO is also achievable in the classical oracle model. These results identify stateful quantum iO as a promising approach towards best-possible testable OTPs.

Contents

1 Introduction

The notion of one-time programs (OTPs) was introduced by Goldwasser, Kalai, and Rothblum [GKR08]. An OTP allows a user to evaluate the program on a single input of their choice, while preventing them from learning anything else about the program. OTPs can be thought of as a strengthening of obfuscation where the only information revealed is the output on a single input. If realized, OTPs would have many applications throughout cryptography such as software protection.

It is not hard to see that OTPs cannot be achieved in the plain model, as a user can always copy the program and evaluate it on multiple inputs. Thus the original work of Goldwasser et al. [GKR08] considered constructing OTPs in the hardware token model. Subsequently, Broadbent, Gutoski, and Stebila [BGS13a] ruled out constructing OTPs with the aid of quantum information for any deterministic classical functionality. Although quantum mechanics forbids cloning, [BGS13a] showed that multiple evaluations can still be achieved generically via gentle measurements. Therefore, hardware assumptions remain necessary for secure OTPs even with quantum information.

By contrast, a work by Ben-David and Sattath [BDS23] constructed "quantum one-time signature tokens," which can be viewed as a quantum one-time program for the signing functionality. This result circumvents the impossibility of [BGS13a] by implicitly leveraging the fact that the signing functionality is a *randomized* (classical) functionality with high entropy outputs. More recently, two works [GLR+25, GM24] revisited the definitions of quantum OTPs. Specifically, they proposed and proved the security for a quantum construction of OTPs for general randomized (classical) functionalities with entropic outputs, the first general positive result without hardware assumptions[1].

Despite these exciting new results, if we look more closely at the intuitive ideal security goal of only "revealing information about a single input", we see that even randomized functionalities do not completely circumvent the impossibility of [BGS13a]. In particular, as pointed out in [GLR+25], if it were possible to deterministically extract some piece of information from the probabilistic output (say, all possible outputs for a given input had the same parity), then this information can be extracted from multiple inputs by a generalization of [BGS13a]. This seemingly violates the intuition for what a one-time program should be. The prior works [GLR+25, GM24] do give some one-time security guarantee but it is unclear what this actually means for the one-time security of general programs, and whether their notions are the strongest security one could hope for. In particular, [GLR+25] propose a strong simulation security notion of one-time security, but leave open the question whether it is the strongest notion of one-time security one could hope for. For example, it would be ideal to have a security guarantee that says that the program protected is one-time except for certain classes of attacks such as the one above. We further discuss the security definition from prior works in Section 1.5.

Given the discussion, the following fundamental question naturally arises.

For a given family of functionalities,
what is the strongest achievable one-time security notion for quantum OTPs?

For inspiration, let us momentarily turn our attention to an analogous problem but for classical obfuscation, the goal of hiding the implementation of a program while maintaining its input-output behaviors. Here, a similar issue arises. Virtual black-box (VBB) obfuscation is the natural ideal notion for obfuscation: the behavior of any adversary receiving the obfuscated program cannot be distinguished from that of a simulator with black-box access to an oracle computing the functionality. But it is provably impossible [BGI+12] for certain functionalities. Due to this general impossibility, two approaches have been taken. First, [BGI+12] propose a weaker notion of indistinguishability obfuscation (iO), which roughly states that the obfuscations of two equivalent programs are indistinguishable. This notion avoids the impossibility, and is even potentially achievable (e.g. [GGH+16, JLS20] in the pre-quantum setting, or [BGMZ18, HJL25]). On the other hand, it is a priori unclear what

[1] From this point onwards, we will exclusively focus on OTPs without hardware assumptions.

sorts of guarantees iO provides, as we usually care about the security of a single program, and it is not clear we gain an insights into the security of a particular program by looking at equivalent programs.

The other direction is to stick with VBB, but show that it *is* achievable for some very specific functionalities [CD08, LPS04, WZ17, GKW17]. Unfortunately, there is a wide gulf between what is known to be VBB obfuscatable and what is known to be un-obfuscatable.

These two directions both have major limitations. Fortunately, the work of Goldwasser and Rothblum [GR07] provides a satisfying way to unify both approaches through the lens of "best possible" obfuscation. Instead of trying to determine whether a given functionality can be obfuscated, they instead just try to give an obfuscator which is "best possible", in the intuitive sense that if a functionality can be obfuscated security by *any* obfuscator, then the given obfuscator also obfuscates that functionality securely. Surprisingly, they show that such best-possible obfuscation is actually *equivalent* to iO. With the subsequent emergence of iO constructions, we now have best-possible obfuscation for all programs. Now, in order to VBB obfuscate some functionality for which VBB obfuscation is possible, all that is necessary is to show that *some* obfuscator exists, which then implies that any iO scheme is in particular a VBB obfuscator for that functionality.

Inspired by the success of best-possible obfuscation and iO, it is natural to ask if there is also an analog of a best-possible obfuscator for quantum OTPs, i.e. whether there exists a generic quantum OTP transformation that always achieves the strongest possible one-time security for any possible functionality. More succinctly,

Is there a best-possible one-time compiler?

1.1 Our Results on Best-Possible One-Time Programs

The results in this work on best-possible one-time programs are three-fold. First, our work gives convincing evidence that perhaps surprisingly, there is rather *no* generic way to achieve best-possible one-time programs (with or without oracles). Second, we also show that best-possible security is achievable via a simulation security for a very natural class of quantum one-time compilers called "testable one-time programs" in the oracle model. Finally, we point to a plausible approach for constructing best-possible testable quantum one-time programs in the plain model.

Impossibility of Best-Possible One-Time Programs. First, we introduce the definition of best-possible one-time programs, à la "best-possible obfuscation" of [GR07]. For this result, we state impossibility for randomized classical functionalities. This already rules out generic best-possible compilers for *all* quantum functionalities, since randomized classical functionalities are a special case.

Definition 1 (Informal; Definition 12)**.** *A one-time compiler* OTP^* *is* best-possible *for a family* $\mathcal{F}$ *of sampling functionalities if there exists a simulator* Sim *such that for any sampling functionality* $f \in \mathcal{F}$*, and for any quantum program* P *that* one-time *implements* f*,* $\mathsf{OTP}^*(f, 1^{|P|})$ *is computationally indistinguishable from* $\mathsf{Sim}(P, 1^{|f|})$*.*

Intuitively, this definition captures the idea that among all the programs P that implement the sampling functionality f, $\mathsf{OTP}^*(f)$ leaks the minimum amount. The length of the program is also unavoidably leaked similar to indistinguishability obfuscation and best-possible obfuscation [GR07].

Our main theorem rules out best-possible one-time program compilers that work for any program, even if restricted to randomized classical functionalities.

Theorem 1 (Informal; Theorem 8)**.** *Best-possible one-time compilers do not exist unless lossy encryptions do not exist. This holds relative to all oracles.*

Corollary 1 (Informal; Theorem 7)**.** *Assuming either Learning with Errors (LWE) is hard or weak pseudorandom effective group actions*[2]*, best-possible one-time compilers do not exist.*

Even though we state our theorem as the impossibility of the best-possible one-time compiler as we defined above, our proof does not actually rely on the subtleties in defining best-possible one-time programs. In particular, we prove this by identifying two classes of randomized programs that are computationally indistinguishable yet the best-possible one-time compilers must work very differently, leading to a contradiction. We elaborate on this in the technical overview and even further in Section 1.4.

Best-Possible Testable One-Time Programs. We now turn to a positive direction. Instead of comparing against all one-time implementations, we compare within a natural restricted class and ask for best-possible security among what we call "testable one-time compilers".

Definition 2 (Informal; Definition 13)**.** *A one-time compiler* $\mathsf{OTP}(P)$ *is* testable *if it (possibly randomly) outputs two programs* $(|\tilde{P}\rangle, R)$ *such that* $|\tilde{P}\rangle$ *is the one-time program that one-time implements* P *and* R *implements the reflection unitary* $I - 2|\tilde{P}\rangle\langle\tilde{P}|$*.*

Intuitively, this is the class of one-time compilers whose outputs are well-defined pure states: the pure state $|\tilde{P}\rangle$ that one-time implements the functionality is "defined" by the reflection program R. We emphasize that the obfuscator's output can still be a mixed state, in which case the mixed state should be entirely supported on testable pure state programs. Crucially, this definition disallows a mixed state program that *on average* one-time implements P. Looking ahead, this restriction is exactly what breaks the counterexample construction in the impossibility proof: that construction uses mixed implementations that are only correct on average, which testability excludes. We further expand on this subtlety later in Section 1.4.

The main motivation for this definition is that there are concrete and recurring ways to modify most existing one-time compiler constructions to become testable[3]. For starters, a trivial case is the classical-state case: if $|\tilde{P}\rangle$ is always a computational-basis string, then one can first read out that string (equivalently, measure in the computational basis) and then efficiently implement the exact reflection $I-2|\tilde{P}\rangle\langle\tilde{P}|$ coherently via an equality check and phase kickback. The most common pattern for designing a one-time compiler is to issue an uncloneable token that collapses upon use, along with an obfuscated classical program that only works when the two-basis measurement outcome on the token is correct; implementing the reflection in this case is also straightforward by simply making use of the obfuscated classical program to check if the token is undisturbed. More generally, the reflection oracle can usually be implemented by simply gently measuring if the one-time program is still functional. In fact, we conjecture that the program implicitly having a reflection oracle might be an unavoidable property for many natural function classes, since an approximate version of this test is always possible if the output of the functionality is verifiable, such as a signature token or a one-time NIZK.

Our next contribution shows that best-possible testable one-time programs, i.e. one-time programs that are best-possible among testable ones, are achievable in the oracular setting via a simulation-based security, which is a revised and generalized version of Single Effective Query (SEQ) security from [GLR+25]. We further elaborate on this in Section 1.2.

[2]The impossibility from LWE is slightly weaker: in that case, we only rule out best-possible one-time programs among those that one-time implements the same functionality up to a negligible statistical distance; the reason is basically that the lossiness of the LWE-based construction is only statistically close. The impossibility from group actions rules out the weaker notion of best-possible one-time programs among those that one-time implements the exact same functionality (with no error).

[3]The only counterexample that we are aware of is given in this work in the impossibility result, which is arguably contrived.

Theorem 2 (Informal; Corollary 2)**.** *There exists a classical oracle relative to which there exists an SEQ-secure one-time compiler for all quantum non-oracular functionalities.*[4]

Theorem 3 (Informal; Theorem 9)**.** *Relative to all oracles, any one-time compiler that achieves SEQ security is best-possible among testable one-time compilers.*

Theorem 4 (Informal; Theorem 15)**.** *For every non-trivial randomized classical functionality $f : \mathcal{X} \times \mathcal{R} \to \mathcal{Y}$ with $|\mathcal{X}| > 1$, there is a quantum channel Φ_f such that the classical query interface O_f^{CSEQ} and the generalized query interface $O_{\Phi_f}^{\mathsf{SEQ}}$ are efficiently inter-simulatable. Consequently, the classical* CSEQ *security is a special case of the generalized* SEQ *security.*

Best-Possible Testable OTP in the Plain Model? Towards achieving best possible testable one-time programs in the plain model, we give a new notion of obfuscation which is sufficient, and seemingly necessary, for constructing these.

Our new notion of *stateful* quantum obfuscation allows obfuscation of quantum programs which maintain an internal state that may evolve as the program is queried. Since one-time programs inherently change their behavior over subsequent queries, allowing for evolution seems necessary. However, accounting for it requires some care. If two programs may behave similarly on the first evaluation, but degrade in a way that subsequent evaluations are distinguishable, then it should be hard to obfuscate them into programs that are indistinguishable. In fact, our impossibility for best-possible one-time programs formalizes this intuition: it is in general impossible to indistinguishably obfuscate two programs that are equivalent only on a single query. Instead, we ask to indistinguishably obfuscate two circuits only if they produce the same output distributions over *many* sequential evaluations. We call this notion *stateful* quantum iO.

Theorem 5 (Informal; Theorems 10 and 11)**.** *Assuming stateful quantum iO, there exists a best possible testable one-time compiler for all quantum functionalities. Furthermore, stateful quantum iO exists in the classical oracle model.*

1.2 One-Time Security for Quantum Functionalities

Along the lines of investigating best-possible one-time programs, we also extend the study of one-time security from randomized classical functionalities to all quantum (channel) functionalities. As far as we are aware, our Theorems 2 and 5 are also the first positive results for constructing one-time programs for arbitrary quantum channels without hardware assumptions.

As mentioned earlier, we revise and generalize the Single Effective Query (SEQ) security defined by Gupte et al. [GLR+25] from randomized classical functionalities to all quantum functionalities. In this work, we call our new generalized notion simply SEQ and refer to the older version from [GLR+25] as classical SEQ (CSEQ). As the prior work [GLR+25] has already shown, SEQ as a simulation-based security is unachievable for certain randomized circuits in the plain model by generalizing the impossibility of VBB [BGI+01b].

Despite this VBB-style impossibility, we believe that SEQ on its own is still a very useful notion for the topic of one-time programs. Of course, the VBB-style impossibility does not rule out SEQ for *all* interesting functionalities; but more importantly, similar to the role VBB plays today, SEQ provides a relatively simple yet *realistic ceiling* on the one-time security that one could hope for in the plain model for a given functionality using *testable* OTP obfuscators. Indeed, it will turn out that SEQ is also capable of capturing the fact (from the gentle-measurement impossibility [BGS13a]) that

[4] The oracle here is simply used to obfuscate and evaluate programs. Here we disallow the functionalities being obfuscated to access the oracle to avoid the VBB-style impossibility [BGI+01a, GLR+25]. In comparison with our impossibility for best-possible one-time programs (instead of testable ones), the functionalities considered by Corollary 1 are nonoracular and still hold in our oracle model assuming LWE/group action assumptions.

any one-time correct implementation of a deterministic/unitary program will inevitably allow an unbounded number of evaluations. Additionally, if one aims to construct a one-time compiler that achieves security unachievable by even SEQ, the OTP construction *must actively* prevent its output program from being testable.

In order for SEQ security to provide this useful role, we point out that our revised version of SEQ is in fact much simpler than the original formulation, despite being a generalization. We hope that our simplification would help downstream applications of one-time programs in future work. For completeness, we give a self-contained presentation of SEQ below.

To motivate the definition, let us briefly recall the intuition for SEQ security from [GLR+25]. SEQ security states that the information extractable from a SEQ-secure OTP of a functionality Ψ can be reduced to that extractable from an ideal query interface of Ψ. This interface allows for a one-time evaluation of Ψ but refuses to cooperate with any other query as much as possible. This is in contrast to older simulation-based one-time security notions such as in [GKR08, BGS13a] where the interface allows literally one physical query, regardless what has been learned in the query.

Without hardware assumptions, an adversary could always compute and uncompute the unitary implementation of the interface. This motivates the following definition of a "self-adjoint implementation" of a quantum channel.

Definition 3 (Informal; Definition 14)**.** *Consider a quantum channel Ψ using input/output register $\mathcal{X}$ which is implemented by some unitary U_Ψ which might use a private auxiliary register $\mathcal{A}$ that is initialized to 0. Let $\mathcal{C}$ be a qubit register acting as a counter. Let the* self-adjoint implementation $S(U_\Psi)$ *of Ψ be the unitary that swaps*

$$|x\rangle_{\mathcal{X}} |0\rangle_{\mathcal{A}} \otimes |0\rangle_{\mathcal{C}} \leftrightarrow (U_\Psi |x\rangle_{\mathcal{X}} |0\rangle_{\mathcal{A}}) \otimes |1\rangle_{\mathcal{C}}$$

and acts as identity everywhere else.

Fact 1. *$S(U_\Psi)$ is a well-defined self-adjoint unitary and can be efficiently implemented given controlled query access to U_Ψ.*

Definition 4 (Informal; Definition 15)**.** *A one-time compiler is SEQ-secure if $\mathsf{OTP}(P)$ can be efficiently (computationally indistinguishably) simulated by a simulator that only gets access to P via querying the self-adjoint implementation for P without having access to its $\mathcal{A}, \mathcal{C}$ registers.*

It is so called "Single Effective Query" because (1) to compute Ψ a second time, you must first return to the state $|x\rangle_{\mathcal{X}} |0\rangle_{\mathcal{A}} \otimes |0\rangle_{\mathcal{C}}$ by uncomputing it; but more importantly, (2) if the output is "meaningfully" disturbed, then further evaluations become effectively impossible because the evaluator cannot return to the initial state $|x\rangle_{\mathcal{X}} |0\rangle_{\mathcal{A}} \otimes |0\rangle_{\mathcal{C}}$.

A helpful example to illustrate SEQ would be to consider a channel that ignores the input and outputs some random coins. In this example, the simulator could invoke the interface and obtain the random coins and possibly uncompute it subsequently; however, if it decides to completely measure the output, this breaks up the entanglement between $\mathcal{A}$ and $\mathcal{X}$ registers, and the SEQ interface would subsequently reject (acting very close to identity) if queried again.

1.3 Technical Overview

Best-possible impossibility. The main idea of proving the impossibility of best-possible one-time compilers is identifying two families of functionalities D and E such that:

1. The one-time compilers for D and E must behave very differently to achieve the strongest one-time security;

2. Yet it is impossible to efficiently distinguish D from E.

Security notion	Impossibilities	Constructions
Single physical query, simulation-based, OTP for quantum functionalities [BGS13b]	Strong impossibility for most functionalities relative to all oracles	For single physical-query learnable (trivial) functions only [BGS13b]; For constant-distribution functionalities (implicit in this work)
Classical SEQ, simulation-based, OTP for classical functionalities [GLR+25]	VBB-style impossibility for contrived functionalities in the plain model [GLR+25]	For all classical functions, relative to a classical oracle [GLR+25]
Generalized SEQ, simulation-based, OTP for quantum functionalities (this work)		For all quantum functionalities relative to a classical oracle (this work)
Best-possible OTP compiler (this work)	Impossibility for generic compilers *relative to all oracles* that admit secure lossy encryptions (this work)	
Best-possible OTP compiler among only testable programs (this work)		From either SEQ security or quantum stateful iO (this work)

Figure 1: One-time program security notions with impossibilities and constructions.

As a first step, we consider D to be a randomized functionality that can be *information-theoretically* one-time protected. Such a functionality can be constructed by simply defining $D(x)$ to be sampling from a fixed distribution C independent of x. Then, note that the ideal one-time security could actually be achieved as follows:

- The one-time compiler simply samples C by evaluating on a fixed input such as $D(0)$ to get a sample y; the compiler then outputs y as the description of the one-time program.
- The evaluator simply ignores the input x and outputs y.

It is clear that this one-time program can be perfectly simulated with just one query to D, thus it achieves the strongest possible one-time security of being simulatable via one "physical" query.

This example is interesting since on one hand, it can be perfectly one-time protected; but on the other hand, the one-time compiler above in general is correct *only* if the functionality in question samples the same distribution for every input. However, intuitively whether a circuit's output depends on the input should not be a property that can be efficiently learned (by the obfuscator). Therefore, the idea is that we should start by considering E to be a sampling circuit that samples a different distribution for every input, yet all of those distributions look indistinguishable to C, the distribution that D samples.

It turns out that we can find such D and E by leverging lossy encryptions. A lossy encryption scheme has two modes, an injective mode and a lossy mode. The injective mode corresponds to a regular encryption scheme; the lossy mode corresponds to a scheme where the encryptions of any two different messages are statistically indistinguishable. More importantly, the public encryption keys sampled in these two modes are computationally indistinguishable. We can construct a post-quantum lossy encryption scheme from LWE or effective group actions using folklore techniques. We remark that our first construction in fact only needs lossy trapdoor functions rather than the full power of LWE.

Now, we can consider the previously proposed function E to be an injective encryption algorithm $\mathsf{Enc}_{\mathsf{inj}}$ for input x, whereas D will be a lossy encryption algorithm $\mathsf{Enc}_{\mathsf{lossy}}$ whose ciphertext

produced is statistically independent of x (this ciphertext distribution would be the distribution C mentioned before).

More concretely, we consider the following families of classical circuits. $\mathsf{Enc}(\mathsf{pk}, x; r)$ is the encryption algorithm of a lossy encryption with inputs public key pk, message x and randomness r.

1. $D_{\mathsf{pk}_{\mathsf{lossy}}}(x; r) := \mathsf{Enc}(\mathsf{pk}_{\mathsf{lossy}}, x; r)$
2. $E_{\mathsf{pk}_{\mathsf{inj}}}(x; r) := \mathsf{Enc}(\mathsf{pk}_{\mathsf{inj}}, x; r)$

To finish the proof, we must show how we can use a best-possible one-time protector OTP^* to efficiently distinguish the lossy key from the injective key. A priori, the best-possible $\mathsf{OTP}^*(D)$ need not output a fixed sample from C even though C achieves the best one-time security for D. Nevertheless, we argue that $\mathsf{OTP}^*(D_{\mathsf{pk}_{\mathsf{lossy}}})$ must still necessarily be effectively a constant function whereas $\mathsf{OTP}^*(E_{\mathsf{pk}_{\mathsf{inj}}})$ cannot possibly be (or correctness would be violated). Then, to distinguish, a QPT algorithm can simply run the program on a uniform superposition over input x and measure the output. Then if $\mathsf{OTP}^*(D_{\mathsf{pk}_{\mathsf{lossy}}})$ is run, the superposition would not collapse since the output is constant and thus deterministic; on the other hand, if $\mathsf{OTP}^*(E_{\mathsf{pk}_{\mathsf{inj}}})$ is run, then the superposition must collapse by injectivity. In the end, we can distinguish $D_{\mathsf{pk}_{\mathsf{lossy}}}$ from $E_{\mathsf{pk}_{\mathsf{inj}}}$ by simply measuring whether the pre-image superposition collapses or not.

Finally, to show that $\mathsf{OTP}^*(D_{\mathsf{pk}_{\mathsf{lossy}}})$ must act like constant function, we use the following observation: $D_{\mathsf{pk}_{\mathsf{lossy}}}$ is in fact one-time equivalent to an ensemble of constant functions. This is because we can just map each ciphertext (image) $\mathsf{Enc}(\mathsf{pk}_{\mathsf{lossy}}, x; r^*)$ in the output of $D_{\mathsf{pk}_{\mathsf{lossy}}}$ to a constant function $C_{\mathsf{pk}_{\mathsf{lossy}}, r^*}(\cdot) := \mathsf{Enc}(\mathsf{pk}_{\mathsf{lossy}}, 0; r^*)$ outputting the fixed $\mathsf{Enc}(\mathsf{pk}_{\mathsf{lossy}}, 0; r^*)$ on any input x, where randomness r^* is pre-sampled uniformly at random and then fixed for the circuit. By the property of the lossy encryption key, $C_{\mathsf{pk}_{\mathsf{lossy}}, r^*}(\cdot)$'s output distribution is statistically indistinguishable from sampling from the output distribution of $D_{\mathsf{pk}_{\mathsf{lossy}}}$ on input x, i.e. all ciphertexts $\mathsf{Enc}(\mathsf{pk}_{\mathsf{lossy}}, x; r)$ for r is sampled uniformly at random upon evaluation.

By the correctness of OTP^*, $\mathsf{OTP}^*(C_{\mathsf{pk}_{\mathsf{lossy}}, r^*})$ must always output $\mathsf{Enc}(\mathsf{pk}_{\mathsf{lossy}}, 0; r^*)$ regardless of the input. However, by best-possible security, $\mathsf{OTP}^*(D_{\mathsf{pk}_{\mathsf{lossy}}})$ must be computationally indistinguishable from applying OTP^* to the distribution over $C_{\mathsf{pk}_{\mathsf{lossy}}, r^*}$ where the fixed output $\mathsf{Enc}(\mathsf{pk}_{\mathsf{lossy}}, 0; r^*)$ is sampled by $r^* \leftarrow \{0,1\}^{|r|}$. Therefore, we can conclude a contradiction here.

We conclude by noting that this impossibility relativizes even in the presence of unitary oracles.

Remark 1. *We note that our impossibility can be dequantized to state that best-possible one-time programs are also impossible with classical obfuscators assuming the existence of the same (but classically secure) lossy encryptions. The only step that needs to be changed is that rather using a quantum distinguisher that tests if the input superposition collapses, we can simply rewind the classical program to test if it is a constant function.*

Best possible testable one time programs. Having ruled out best-possible OTPs, we next turn to testable OTPs. Here, we show that a one-time program achieving SEQ security is as secure as *any* testable one-time program.

To show that SEQ security is best-possible for testable one-time programs, we must show how to simulate $\mathsf{SEQ}(f)$ using an alternative quantum program g which is equivalent to f for one query and has a reflection oracle R. The idea of the simulator is straightforward: we simply implement $\mathsf{SEQ}(f)$ by implementing $\mathsf{SEQ}(g)$ instead. The intuition is that the SEQ interface would hide the implementation details of whether f or g is being queried since they are one-time equivalent.

Towards that goal, it is helpful to first see how SEQ interface can be efficiently implemented. In particular, let U_f be a purified unitary of f, $\mathsf{SEQ}(f)$ is implemented by the following quantum circuit $S(U_f)$:

1. Controlled on $|1\rangle_{\mathcal{C}}$, compute $U_f^\dagger$.

2. Controlled on the auxiliary register $\mathcal{A}$ being all zeroes, flip $\mathcal{C}$ register.

3. Controlled on $|1\rangle_{\mathcal{C}}$, compute U_f.

To formalize our intuition above, we establish the following lemma showing that self-adjoint implementations hide the implementation details of the channel, which could be of independent interest.

Lemma 1 (Informal; Lemma 11)**.** *For any two unitary implementations U_Ψ, U'_Ψ of the same channel Ψ, $S(U_\Psi)$ and $S(U'_\Psi)$ are perfectly indistinguishable given unbounded query access to only the input/output register $\mathcal{X}$.*

This already suffices to prove the statement if f and g are classically described programs. However, there is one subtlety we need to address, which is that g may be a quantum state describing a program $|g\rangle$. After evaluating it once, the program state may be disturbed. Unlike classically, we cannot say that the program state is readonly due to no cloning.

This is where the reflection oracle R plays a crucial role: the reflection oracle R well defines the quantum program $|g\rangle$. In particular, the way we fix this is by modifying the step 2 of $\mathsf{SEQ}(g)$ to in addition also control on the quantum program register being $|g\rangle$, which can be efficiently implemented using the reflection oracle. The intuitive reason that this works is that this way, we are effectively implementing the following quantum program that does *not* involve a quantum auxiliary input $|g\rangle$ and yet is perfectly equivalent to g:

1. Perform a swap $|0\rangle \leftrightarrow |g\rangle$ in the program register.

2. Carry out the original computation.

Observe that the SEQ for this program would reflect around all zeroes on both the auxiliary register and the program register, which is equivalent to reflecting around all zeroes on the auxiliary register and $|g\rangle$ on the program register if we do not do the first step.

Achieving SEQ Security with Oracles. Note that our Lemma 1 gives a straightforward way for instantiating SEQ-secure one-time programs with *stateful unitary* ideal obfuscation. In such an obfuscation scheme, the obfuscator takes as input a unitary U that acts on a public register $\mathcal{X}$ and an internal auxiliary register $\mathcal{A}$ that is initialized to some state that is given; and it outputs $\tilde{U}$ which can only be queried in either the forward or backward direction where the internal register is inaccessible. Then to one-time protect f, we can simply use the ideal obfuscation to obfuscate and give out $\mathsf{SEQ}(f)$.

Intuitively, such a stateful unitary ideal obfuscation scheme can be implemented in a (stateless) unitary oracle model, or by ideal unitary obfuscation. The idea is that we can use a quantum authentication scheme to protect the internal state so that we can delegate the internal state management to the adversary without compromising security. Furthermore, such a scheme can be ported to the classical oracle model using the compiler of [HT25]. While this compiler introduces quantum states into the program, this does not matter for our application since our obfuscated programs are allowed to have quantum states.

Towards the Plain Model via Stateful Quantum Obfuscation. Even though SEQ security is achievable using ideal obfuscation or classical oracles, we already know that there are randomized classical functionalities that cannot be SEQ-securely one-time protected in the plain model [GLR+25]. A natural question then is whether we can nevertheless achieve best-possible testable one-time programs in the plain model. Perhaps this is achievable through an alternative notion of obfuscation which plausibly exists in the plain model, similar to how for many-time security, iO and best-possible obfuscation are equivalent.

Previous works propose natural definitions for indistinguishability obfuscation of quantum circuits that compute either pseudodeterministic functions [BBV24, CG24] or unitaries [HT25] — for every two quantum programs that compute the same pseudodeterministic function (or unitary), their obfuscations should be indistinguishable. A natural attempt at generalizing this to arbitrary quantum *channels* would be to ask that if two quantum programs output similar mixed states, their obfuscations should look indistinguishable. However, if two programs produce similar outputs for the first evaluation, but then degrade so that subsequent evaluations are distinguishable, it should be hard to obfuscate them into programs that are indistinguishable. In fact, our previously discussed impossibility result formalizes this intuition.

This is a uniquely quantum issue for two reasons. First, quantum programs with auxiliary quantum states inherently maintain state over evaluations that involve non-gentle measurements. Second, quantum circuits have the ability to self-produce randomness, and there is no general way to "de-randomize" them, like in the classical case. Classical descriptions of circuits do not degrade with evaluations, and it is always possible to de-randomize a probabilistic circuit $C(x;r)$ and obfuscate $C(x;F_k(x))$, so that the obfuscations of two equivalent classical probabilistic circuits C_1, C_2 will always produce the same output across evaluations. To avoid the impossibility, we explicitly consider obfuscating programs whose state may change over time.

Definition 5 (Stateful iO, Informal; Definition 20)**.** *Stateful indistinguishability obfuscation allows two stateful quantum programs to be indistinguishably obfuscated if and only if they produce the same output distributions over* any *sequence of evaluations.*

To show that stateful iO evades our impossibility result, we show that it is implied by ideal unitary obfuscation. Intuitively, a unitary program can authenticate a private register to itself in order to maintain state across multiple queries. Using ideal obfuscation, the authentication key is protected, so the register remains private from the evaluator. Similarly as before, we can further port this to the classical oracle model using the compiler of [HT25].

Furthermore, unlike ideal obfuscation, there does not seem to be an inherent barrier to constructing stateful iO in the plain model. Because the output distributions are required to be close even for query sequences that can depend on the programs being obfuscated, stateful iO avoids the self-referential techniques used to rule out ideal obfuscation [BGI+01a].

Finally, we show that stateful iO implies best-possible testable one-time programs. Thus, if one could construct stateful iO in the plain model, they would also construct best-possible testable one-time programs in the plain model. This again uses Lemma 1, which when combined with the security guarantee of stateful iO, shows that whatever could be learned from the stateful obfuscation of $\mathsf{SEQ}(f)$ could also be learned from g by statefully obfuscating $\mathsf{SEQ}(g)$.

1.4 Discussions

Should Best-Possible OTPs Consider Mixed Programs? One possible concern regarding our definition of best-possible one-time programs (and the resulting impossibility) is that it differs from best-possible obfuscation in one technical respect. Namely, we require the simulator Sim to work for any *mixed-state* implementation that one-time implements the same functionality on average, rather than only for fixed pure-state implementations.

This choice is intentional. Interpreting "best-possible" literally, i.e. as secure as *any* one-time implementation, naturally leads to a comparison class that includes all efficient implementations, including mixed ones.

For comparison, let us also briefly look at why best-possible obfuscation is traditionally defined with respect to (effectively) pure-state implementations only [GR07]. The reason is simply that for obfuscation of deterministic functionalities, this distinction collapses: if a mixed state implements a deterministic functionality on average, then every pure state in its support implements the same

functionality, so quantifying over mixed states only makes the definition conceptually more involved. Turning back to one-time programs of randomized functionalities, the distinction need not collapse: a mixed implementation can be correct only in aggregate while its pure components are not individually correct. This gap is exactly what drives our impossibility for unrestricted best-possible OTPs, and also why restricting to testable programs avoids that counterexample.

A pure-only best-possible notion is arguably still meaningful, but it should be interpreted as a weaker security guarantee: best among pure-state implementations only. Our definition of testable OTP can exactly be seen as a formal approach for capturing pure-state implementations. However, if the goal is the *absolute strongest* benchmark in the usual English sense of best-possible, then quantifying over mixed-state implementations is the more appropriate definition.

Scope of the Impossibility. Our impossibility result considers one-time protection of a classical sampling channel: the input is classical (or measured in the standard basis if not) and the output is one classical sample. This is the natural interface for one-time protection of randomized functionalities. For example, for one-time signatures, one naturally asks that the adversary can obtain at most one classical signature on one classical message.

With that said, one could also consider one-time protection of different quantum channels that wrap a classical functionality. Whether our impossibility extends to every such restricted coherent extension is unclear. We leave this setting to future work for the reasons below.

- Our goal is to rule out generic best-possible compilers for the broadest functionality classes (all quantum channels); ruling out the classical sampling channel already suffices for this purpose. By contrast, restricting the protected class can evade impossibility, one example of which being restricting the class to be the constant-distribution functionalities.
- It is unclear what is the right/meaningful coherent extension channel of a randomized functionality. Should the randomness be sampled once and used for every input, or should the randomness be sampled fresh for every input (such as by querying a random oracle/pairwise independent hash on the input)?
- Relatedly, it is unclear which coherent extension channel is useful for downstream applications, such as one-time signatures.

Prevalence of Our Best-Possible OTP Impossibility. In our best-possible impossibility, we only identify two classes of functionalities for which best-possible OTPs cannot exist. One possible loophole of our impossibility is that perhaps once you exclude one class from consideration, then best-possible OTPs may indeed be possible. Inspecting our impossibility further, the lossy class is probably unlikely to be of interest for one-time programs, where the randomized program samples the same distribution for every input.

While this is true, we argue that the impossibility could creep up even in unsuspecting functionalities. For starters, consider the randomized functionality $f(x;r) \to (y_1, y_2)$ where y_1 is sampled from a fixed distribution independent of x. Then, this is a functionality that is not lossy yet (a suitably adapted version of) our impossibility still applies. One could even further consider more involved variants of this where the lossy structure is even less apparent, such as applying a pseudorandom permutation to the output. Given these examples, we suspect that it is unlikely to identify a meaningful and large class of functionalities where best-possible OTPs are possible, since any class that is closed under augmentation of such structures is also susceptible to our impossibility.

Implications for Defining Obfuscation of Quantum Programs. Previous work propose natural definitions for indistinguishability obfuscation for quantum circuits that compute pseudodeterministic classical functions [BBV24, CG24] and unitaries [HT25] — for every two quantum programs

that compute the same pseudodeterministic function (or unitary), their obfuscations should be indistinguishable. A natural attempt at generalizing this to quantum *channels* would be to ask that if two quantum programs output (approximately) the same mixed state, their obfuscations should look indistinguishable. Note that this natural attempt only asks the two programs to produce samples from same mixed state for the *first* evaluation. This is because for (approximately) classical functions or more generally unitaries, one-time equivalence is equivalent to many-time equivalence by gentle measurement. However, when the functionality is not unitary and the program is described by a program state, the program state itself may evolve after the first query. Crucially, we can rule out this one-time security notion by invoking our best-possible OTP impossibility, since intuitively if such an object exists, then it would give a best-possible OTP similar to how iO is equivalent to best-possible obfuscation. We formalize this in Section C.

Our definition of stateful iO gets around this problem by considering *many-time* equivalence of two programs rather than *one-time* equivalence. Thus, when defining obfuscation for quantum sampling programs, a notion which takes into account behavior on sequential queries, like stateful iO, seems necessary.

Future Directions. Are there any interesting examples of one-time programs that are not testable? Less interesting examples appear in our impossibility where lossy-mode encryptors or constant-distribution samplers are considered. It would be interesting to identify other examples that can be one-time protected yet is different from the one considered in our impossibility.

To further motivate one-time program security beyond SEQ, we sketch a generic efficient attack against any testable one-time program f. The attack aims to estimate multiple (efficient) observables $O_1, ..., O_t$ on different input states $x_1, ..., x_n$; in other words, we aim to estimate $\mathsf{Tr}(O_i f(x_j))$ for all i, j up to a small inverse polynomial error. (For example, we can estimate how often each bit of the output is 0 for $x_1, ..., x_n$.) While this attack is not always useful (say for forging multiple signatures against a one-shot signature), it is a clear separation between SEQ and a single physical query (or even any polynomial number of physical queries) for almost all functionalities.

For a single observable and a single input, this can be estimated using Marriott–Watrous style rewinding [MW05]: one simply alternates the measurement of the observable and the projection back to the original state using the reflection oracle. To extend this to multiple observables on multiple inputs, it suffices to make each estimation measurement gentle: this can be done via the Laplace noise measurement [AR19a, Corollary 6]. We formalize this generic attack in Section D.

On the (many-time) obfuscation front, does unitary indistinguishability obfuscation imply stateful iO? The main difficulty in lifting our proof from ideal obfuscation is to adapt the argument that the obfuscator protects the authentication key. Furthermore, does classical indistinguishability obfuscation imply unitary iO?

1.5 One-Time Security in Previous Works

For completeness, we survey one-time security notions studied in prior works in this section.

- The work by Gupte et al. [GLR+25] propose a notion called "single effective query (SEQ) simulation-based one-time security" that circumvents the impossibility results (without hardware assumptions) of the simulation-based one-time security defined in preceding works [GKR08, BJ15]. In this work, we revise and generalize their SEQ definition to all functionalities and, to distinguish, we refer to their original (classical) version as CSEQ.

  More concretely, the CSEQ one-time security notion requires that the adversary's view after maliciously utilizing the one-time program of f can be simulated by only querying a restricted stateful query interface to f. Such an interface attempts to record prior queries made to f and only answers the query if no prior queries are recorded.

They show that this notion is achievable for *all* functionalities in the classical oracle model, which is both good and also unsatisfactory. It is good because it is a single notion that captures all functionalities. It is however unsatisfactory because it delegates the problem of identifying what security their construction achieves to the problem of what can be learned through the CSEQ interface. For example, the CSEQ interface would allow an unlimited number of evaluations for a deterministic functionality thus keeping consistency with the impossibility result [BGS13a]. However, for a general randomized functionality, it is not clear what can be learned through the CSEQ interface since the interface is somewhat complicated. Our SEQ notion somewhat mitigates this issue since our SEQ security is much simpler.

- Apart from CSEQ security, [GLR+25] has presented several security notions in the plain model, but are relatively restricted to a specific setting or targeting a specific application. These notions can be viewed as extensions of the security requirement for one-time signature tokens in [BDS23] to more functionalities. [BDS23] states that it is impossible to produce two valid outputs signatures for two distinct inputs (messages) with respect to the signature verification algorithm. This security can be generalized to any unforgeable functionalities such as one-time NIZKs [GLR+25]. Similarly, the security of one-time PRFs (where part of the PRF input is sampled randomly) [GLR+25] only states that it is impossible to simultaneously distinguish the outputs on two different inputs from random.

- The work by Gunn and Movassagh [GM24] showed that for their construction, a somewhat natural class of attackers (including the attack in [BGS13a]) cannot even produce a second output as long as the functionality samples a high min entropy output on every input. However, it is possible that a more malicious attacker can still produce multiple outputs: an example of this can be seen from the plain-model impossibility in [GLR+25, Theorem 7.7].

We conclude by pointing out that all the constructions in these works can be straightforwardly modified so that they are testable (using ideas discussed before). Therefore, these security notions are all achievable by using a best-possible testable one-time compiler.

2 Preliminaries

2.1 Quantum Computation

We fix canonical description formats for all objects. For a classical randomized function f, we write $|f|$ for the bit-length of its canonical description (excluding any oracle access it might carry). For a quantum sampling program $P = (\rho, C)$, we write $|P|$ for the length of a canonical description consisting of a description of the state ρ together with the description of the (possibly oracle-aided) circuit C; oracle access is not counted toward length. For a stateful quantum program $(U, |\psi\rangle)$, we similarly write $|(U, |\psi\rangle)|$ for the length of a canonical description of U together with a description of $|\psi\rangle$.

When comparing programs by length (e.g., in indistinguishability obfuscation), we allow *padding* to equalize lengths without changing behavior: padding may add ancilla qubits initialized to $|0\rangle$ and insert no-op gates that leave the program's input–output behavior unchanged.

When we say a result relativizes to any unitary oracle, we mean that it holds in the model where all parties receive query access to a family of unitaries including inverse and control as is standard [Zha25].

We will need a few facts about the trace distance of two pure states whose amplitudes are defined by classical probability distributions.

Definition 6 (Hellinger Distance). *For two probability density functions f, g over a finite domain $\mathcal{X}$, the squared Hellinger distance between f and g is defined as*

$$H^2(f,g) = \frac{1}{2}\sum_{x\in\mathcal{X}}\left(\sqrt{f(x)} - \sqrt{g(x)}\right)^2 = 1 - \sum_{x\in\mathcal{X}}\sqrt{f(x)g(x)}.$$

Lemma 2. *Let D_1, D_2 be two probability density functions over a finite domain $\mathcal{X}$, then*

$$\frac{1}{2}H^2(D_1,D_2) \leq \mathrm{TV}(D_1,D_2) \leq H(D_1,D_2).$$

Lemma 3. *Let $\mathcal{X}$ be a finite set and let D_1, D_2 be probability densities on $\mathcal{X}$. Let*

$$|\psi_1\rangle = \sum_{x\in\mathcal{X}}\sqrt{D_1(x)}\,|x\rangle \quad \text{and} \quad |\psi_2\rangle = \sum_{x\in\mathcal{X}}\sqrt{D_2(x)}\,|x\rangle \ .$$

Then, the trace distance between the pure states $|\psi_1\rangle, |\psi_2\rangle$ is

$$T(|\psi_1\rangle\langle\psi_1|, |\psi_2\rangle\langle\psi_2|) = \sqrt{1-(1-H^2(D_1,D_2))^2}.$$

2.2 Quantum Authentication Schemes

Definition 7 (Quantum Authentication Scheme). *A **quantum authentication scheme** is a tuple of QPT algorithms* $(\mathsf{KeyGen}, \mathsf{Enc}, \mathsf{Dec}, \mathsf{Ver})$ *with the following behavior.*

- $k \leftarrow \mathsf{KeyGen}(1^\lambda, n)$ *takes as input the security parameter λ and a length $n \in \mathbb{N}$, then outputs a key k. Here, n is the length of the state to be authenticated.*
- $|\psi'\rangle \leftarrow \mathsf{Enc}_k(|\psi\rangle)$ *takes as input a key k and an n-qubit state $|\psi\rangle$, then outputs another state $|\psi'\rangle$.*
- $|\psi''\rangle / \bot \leftarrow \mathsf{Dec}_k(|\psi'\rangle)$ *takes as input a key k and a state $|\psi'\rangle$, then outputs another state $|\psi''\rangle$ or $\bot$.*
- $\mathsf{Acc}/\mathsf{Rej} \leftarrow \mathsf{Ver}_k(|\psi'\rangle)$ *takes as input a key k and a state $|\psi'\rangle$, then outputs* Acc *(accept) or* Rej *(reject).* Ver_k*'s behavior is identical to* Dec_k*'s, except* Ver_k *outputs* Acc *whenever* Dec_k *would output $|\psi\rangle$ and outputs* Rej *whenever* Dec_k *would output $\bot$.*

A quantum authentication scheme must satisfy correctness and security properties.

- ***Correctness.*** *For every k in the support of* KeyGen *and every state $|\psi\rangle$,*

$$\mathsf{Dec}_k(\mathsf{Enc}_k(|\psi\rangle)) = |\psi\rangle$$

- ***Security.*** *For every QPT adversary $\mathcal{A}$, there exists an $\varepsilon(\lambda) \in [0,1]$ such that for every $|\psi\rangle$,*

$$\left\{\mathsf{Dec}_k(\rho) \;\middle|\; \begin{array}{c} k \leftarrow \mathsf{KeyGen}(1^\lambda) \\ \rho \leftarrow \mathcal{A}(\mathsf{Enc}_k(|\psi\rangle)) \end{array}\right\} \approx (1-\varepsilon)\{|\psi\rangle\} + \varepsilon\{\bot\}$$

  If this holds given oracle access to Ver_k *(or if there is a public key* vk *that allows implementing* Ver_k *and it holds given access to* vk*, we say that the scheme is publicly verifiable.*

[BBV24] shows how to construct publicly verifiable quantum authentication using coset states.

2.3 Ideal Obfuscation of Unitaries

In this section, we will deal with quantum programs $(|\psi\rangle, Q)$ that approximately implement a unitary transform $\rho \mapsto U\rho U^\dagger$.

Definition 8. *A quantum program $(|\psi\rangle, Q)$ is an ε-approximation of a unitary transformation U if*

$$\|Q(\cdot \otimes |\psi\rangle) - U(\cdot)\|_\diamond \leq \varepsilon.$$

Definition 9 (Quantum State Obfuscation for Unitaries)**.** *A* quantum state ideal obfuscation *for the class of approximately-unitary quantum programs in the classical oracle model is a pair of QPT algorithms $(\mathsf{QObf}, \mathsf{QEval})$ with the following syntax:*

- $\mathsf{QObf}(1^\lambda, (|\psi\rangle, Q)) \to (|\widetilde{\psi}\rangle, \mathsf{F})$*: The obfuscator takes as input the security parameter 1^λ and a quantum program $(|\psi\rangle, Q)$ in the plain model*[5]*, and outputs an obfuscated program specified by a state $|\widetilde{\psi}\rangle$ and a classical function F.*
- $\mathsf{QEval}^{\mathsf{F}}(\rho_{\mathsf{in}}, |\widetilde{\psi}\rangle) \to \rho_{\mathsf{out}}$*: The evaluation algorithm executes the obfuscated program on quantum input ρ_{in} by making use of the state $|\widetilde{\psi}\rangle$ and making superposition queries to the classical oracle F, and produces the quantum output ρ_{out}.*

These algorithms have to satisfy the following properties.

- ***Functionality-Preserving:*** *For every quantum program $(|\psi\rangle, Q)$ which is an $\mathsf{negl}(\lambda)$-approximation of some unitary U, the quantum program*

  $$\mathbb{E}_{(|\widetilde{\psi}\rangle, \mathsf{F}) \leftarrow \mathsf{QObf}(1^\lambda, (|\psi\rangle, Q))} \left(|\widetilde{\psi}\rangle, \mathsf{QEval}^{\mathsf{F}} \right)$$

  is also a $\mathsf{negl}(\lambda)$-approximation of $\mathcal{U}$.
- ***Ideal Obfuscation:*** *There exists a QPT simulator Sim such that for every QPT adversary $\mathcal{A}$ and quantum program $(|\psi\rangle, Q)$ that $\mathsf{negl}(\lambda)$-approximates some unitary U, and every QPT distinguisher D,*

  $$\left| \Pr\left[1 \leftarrow D(\mathcal{A}(\mathsf{QObf}(1^\lambda, (|\psi\rangle, Q)))\right] - \Pr\left[1 \leftarrow D(\mathsf{Sim}^{U, U^\dagger}(1^\lambda, \mathcal{A}, n(\lambda), m(\lambda))\right] \right| = \mathsf{negl}(\lambda)$$

  Here, n is the input length of the quantum program, m is the size of the quantum program.

Theorem 6 (Theorem 7.2 of [HT25])**.** *There exists a quantum state ideal obfuscation for the class of approximately unitary programs with quantum inputs and outputs in the classical oracle model, assuming post-quantum one-way functions.*[6]

3 Impossibility of Best-Possible One-Time Compilers

3.1 Definition of a Best-Possible One-Time Program Compiler

A one-time program compiler OTP takes a randomized function f and outputs a sampling program P that one-time implements f. This means that for any input $x \in \mathcal{X}$, the distribution of $P(x)$ is statistically close to $f(x)$. Finally OTP is best-possible if $\mathsf{OTP}(f)$ can be simulated by any program implementing f for which $|f| = |P|$.

[5] To circumvent impossibility results of [BGI+01a], we say that Q does not make use oracles.

[6] Alternatively, the one-way functions can be replaced with a random oracle, in which case the classical oracle becomes inefficient.

Randomized Functions. A randomized function $f : \mathcal{X} \times \mathcal{R} \to \mathcal{Y}$ takes an input $x \in \mathcal{X}$, samples randomness $r \xleftarrow{\$} \mathcal{R}$ and outputs $y = f(x; r) \in \mathcal{Y}$. Let $f(x)$ be the distribution over y-values that results from this procedure. By considering f as a circuit description of itself, f is computable in time $\mathsf{poly}(|f|)$, where $|f|$ is its description length.

Let $\mathcal{F}$ be a set of randomized functions, and for each $\lambda \in \mathbb{N}$, let $\mathcal{F}_\lambda$ be the set of all functions in $\mathcal{F}$ with description length λ.

Sampling Programs. Given a randomized function $f : \mathcal{X} \times \mathcal{R} \to \mathcal{Y}$, a sampling program P that implements f is a quantum program $P = (\rho, \mathsf{Eval})$ comprising a (possibly mixed) quantum state ρ and a (possibly oracle-aided) quantum unitary circuit Eval. We write $|P|$ for its description length.

P can be evaluated on any $x \in \mathcal{X}$ by computing

$$\mathsf{Eval}(|x\rangle\langle x|_{\mathcal{X}} \otimes |0\rangle\langle 0|_{\mathcal{Y}} \otimes \rho_{\mathcal{P}})$$

and measuring the $\mathcal{Y}$ register to obtain output y. Let $P(x)$ refer to the distribution over y-values that results from this evaluation procedure.

Definition 10 (Correctness of Sampling Programs)**.** *Let $\mu(\lambda)$ be a negligible function. Given a randomized function $f \in \mathcal{F}_\lambda$ and a sampling program P, we say that P **implements** f **with error** μ if for every $x \in \mathcal{X}$, the statistical distance between $P(x)$ and $f(x)$ is $\leq \mu(\lambda)$.*

Our notion of correctness is really a notion of one-time correctness. It says that the first time the program is evaluated, it will sample from approximately the desired distribution. However, measuring the output of the program may destroy the program state ρ, so there is no guarantee that the second evaluation of the program will be correct.

One-Time Program Compilers. A one-time program compiler OTP takes a randomized function f and outputs a sampling program P that implements f. The compiler is best-possible if $\mathsf{OTP}(f)$ can be simulated by any program implementing f for which $|f| = |P|$.

Definition 11 (One-Time Program Compiler)**.** *Let $\mathcal{F}$ be a family of randomized functions. A **one-time program compiler for** $\mathcal{F}$ is a QPT machine OTP that takes as input the description of a function $f \in \mathcal{F}$ and outputs a sampling program $P = \mathsf{OTP}(f)$. For correctness, we require that there is a negligible function $\mu(\lambda)$ such that for any $\lambda \in \mathbb{N}$ and any $f \in \mathcal{F}_\lambda$, $\mathsf{OTP}(f)$ implements f with error $\mu(\lambda)$.*

The following definition says that a one-time program compiler is best-possible if $\mathsf{OTP}(f)$ can be simulated given any program P that implements f with low error. We consider two notions of equivalence: perfect equivalence requires P to implement f with 0 error, and statistical equivalence allows P to implement f with a non-zero, but still negligible, error.

Definition 12 (Best-Possible One-Time Program Compiler)**.** *Let $\mathcal{F}$ be a family of randomized functions, and let OTP^* be a one-time program compiler for $\mathcal{F}$.*

*OTP^* is a **best-possible one-time program compiler for** $\mathcal{F}$ with statistical/perfect equivalence if for every QPT adversary $\mathcal{A}$ and any function $\mu(\lambda)$ that is (respectively) negligible/identically zero, there is a QPT simulator Sim and a negligible function $\nu(\lambda)$ such that for any $\lambda \in \mathbb{N}$, any $f \in \mathcal{F}_\lambda$, any sampling program P that implements f with error $\mu(\lambda)$ and satisfies $|P| = |f|$, and any QPT distinguisher D,*

$$\left|\Pr\left[1 \leftarrow D(1^\lambda, \mathcal{A}(1^\lambda, \mathsf{OTP}^*(f)))\right] - \Pr\left[1 \leftarrow D(1^\lambda, \mathsf{Sim}(1^\lambda, P))\right]\right| \leq \nu(\lambda).$$

Note that the distinguisher D can implicitly depend on f because D is chosen after f.

In the study of best-possible obfuscation, [GR07] requires perfect equivalence. In other words, the simulator only needs to work correctly when P implements f with 0 error. Statistical equivalence allows μ to be a non-zero, but negligible, function. Requiring statistical equivalence yields a stronger notion of security. Now the simulator must work correctly even if P does not perfectly implement f.

We can rule out both notions of best-possible OTP compilers. Ruling out the notion that requires statistical equivalence (definition 12) is impossible assumes the hardness of LWE; ruling out the notion that requires perfect equivalence is impossible assumes the pseudorandomness of group actions.

3.2 Impossibility Result

We show that there exists a family of functions for which there is no best-possible one-time program compiler.

Theorem 7. *Assuming the post-quantum hardness of LWE for the parameter choices given in Theorem 12, there exists a function family $\mathcal{F}$ for which there is no best-possible one-time program compiler with statistical equivalence (Definition 12).*

Assuming the weak pseudorandomness of group actions (Assumption 1), there exists a function family $\mathcal{F}$ for which there is no best-possible one-time program compiler with perfect equivalence (Definition 12).

Proof. We can construct statistically lossy PKE from LWE (Theorem 13). Then Theorem 8 below says that statistically lossy PKE rules out a best-possible one-time program compiler with statistical equivalence.

Next, we can construct perfectly lossy PKE from group actions (Theorem 14). Theorem 8 below says that perfectly lossy PKE rules out a best-possible one-time program compiler with perfect equivalence. □

Theorem 8. *Assuming the existence of statistically/perfectly lossy PKE scheme (Definition 22), there exists a function family $\mathcal{F}$ for which there is no best-possible one-time program compiler with statistical/perfect equivalence (Definition 12). This relativizes to all unitary oracles.*

The rest of Section 3.2 is devoted to proving Theorem 8.

The proof has the following roadmap. The function family $\mathcal{F}$ is the union of three other families $\mathcal{C}, \mathcal{D}, \mathcal{E}$. $\mathcal{C}$ contains constant functions; $\mathcal{E}$ mostly contains injective functions; $\mathcal{D}$ contains functions that are, in certain settings, indistinguishable from both $\mathcal{C}$ and $\mathcal{E}$.

Then we show that there exists a QPT adversary $\mathcal{A}$ that can easily distinguish OTPs for functions in $\mathcal{E}$ from OTPs for functions in $\mathcal{C}$. Essentially, $\mathcal{A}$ checks whether evaluating the program entangles the input and output registers. Injective functions do create entanglement, whereas constant functions do not.

For functions in $\mathcal{D}$, $\mathcal{A}$ has contradictory behavior. Functions sampled from $\mathcal{D}$ should be computationally indistinguishable from those in $\mathcal{E}$. However, we also show that a mixture over functions in $\mathcal{C}$ actually implements a function in $\mathcal{D}$. This will imply our contradiction.

The function family $\mathcal{F}$

Assuming the post-quantum hardness of LWE, Theorem 13 says that there exists a statistically lossy PKE scheme (Definition 22) comprising the functions $(\mathsf{Gen}, \mathsf{GenPK}, \mathsf{Enc}, \mathsf{Dec})$ with message space $\mathcal{M} = \{0,1\}^\lambda$. Let $\mathcal{X} = \mathcal{M}$ and $\mathcal{R} = \mathcal{R}_{\mathsf{Enc}}$.

For a given $\lambda \in \mathbb{N}$, let us define three function families $\mathcal{C}_\lambda, \mathcal{D}_\lambda, \mathcal{E}_\lambda$ as follows. Any differences among the families are highlighted in red or blue.

- $\mathcal{C}_\lambda$: Each function $c \in \mathcal{C}_\lambda$ is a constant function described by a pk in the support of $\mathsf{GenPK}(1^\lambda, \mathsf{lossy})$ and a string $r_{\mathsf{Enc}} \in \mathcal{R}_{\mathsf{Enc}}$. The function ignores its inputs (x, r) and outputs $y = \mathsf{Enc}(\mathsf{pk}, 0^\lambda; r_{\mathsf{Enc}})$.
- $\mathcal{D}_\lambda$: Each function $d \in \mathcal{D}_\lambda$ is described by a pk in the support of $\mathsf{GenPK}(1^\lambda, \mathsf{lossy})$. The function takes inputs (x, r) and computes $y = \mathsf{Enc}(\mathsf{pk}, x; r)$.
- $\mathcal{E}_\lambda$: Each function $e \in \mathcal{E}_\lambda$ is described by a pk in the support of $\mathsf{GenPK}(1^\lambda, \mathsf{inj})$. The function takes inputs (x, r) and computes $y = \mathsf{Enc}(\mathsf{pk}, x; r)$.

Let $\mathcal{F}_\lambda = \mathcal{C}_\lambda \cup \mathcal{D}_\lambda \cup \mathcal{E}_\lambda$, and let $\mathcal{F} = \bigcup_{\lambda \in \mathbb{N}} \mathcal{F}_\lambda$. Let $\mathcal{C}, \mathcal{D}, \mathcal{E}$ be defined analogously to $\mathcal{F}$.

Next, let us require that all functions in $\mathcal{F}_\lambda$ have the same description length as each other. We can pad the descriptions with 0s to ensure this is the case. Finally, assume toward contradiction that there exists a one-time program compiler OTP^* that is best-possible for $\mathcal{F}$ with statistical/perfect equivalence.

The adversary $\mathcal{A}$

Next, let us define a quantum algorithm $\mathcal{A}$ that tests whether querying a given sampling program $P = (\rho, \mathsf{Eval})$ will entangle the input and output registers. $\mathcal{A}$ acts on the query register $\mathcal{Q} = \mathcal{X} \times \mathcal{Y}$, the program register $\mathcal{P}$, and two ancilla registers $\mathcal{X}'$ and $\mathcal{Y}'$, which will store inputs and outputs respectively.

1. Initialization: $\mathcal{A}$ prepares an EPR pair of input values on the $\mathcal{X}$ and $\mathcal{X}'$ registers. Let us call this state $|{+}{+}\rangle$.
$$|{+}{+}\rangle := \frac{1}{\sqrt{2^\lambda}} \sum_{x \in \{0,1\}^\lambda} |x\rangle_{\mathcal{X}} \otimes |x\rangle_{\mathcal{X}'}$$
The $\mathcal{P}$ register contains the program state ρ, and $\mathcal{Y} \times \mathcal{Y}'$ contains $|0\rangle \otimes |0\rangle$.
2. $\mathcal{A}$ evaluates the program by applying Eval to the $\mathcal{X} \times \mathcal{Y} \times \mathcal{P}$ registers.
3. $\mathcal{A}$ CNOTs the value on the $\mathcal{Y}$ register onto the $\mathcal{Y}'$ register.
4. $\mathcal{A}$ uncomputes step 2 by applying $\mathsf{Eval}^\dagger$ to the $\mathcal{X} \times \mathcal{Y} \times \mathcal{P}$ registers.
5. $\mathcal{A}$ checks whether the $\mathcal{X} \times \mathcal{X}'$ registers are still in the original state $|{+}{+}\rangle$. If so, $\mathcal{A}$ outputs 0. If not, $\mathcal{A}$ outputs 1.

Intuitively, if P outputs the same value for every input, then the $\mathcal{X} \times \mathcal{X}'$ registers are returned to their original state by the end of $\mathcal{A}$'s execution, so $\mathcal{A}$ outputs 0. If P is injective, then $\mathcal{X}'$ is entangled with $\mathcal{Y}'$, so the $\mathcal{X}' \times \mathcal{X}$ registers will be far from the state $|{+}{+}\rangle$. Then $\mathcal{A}$ will output 1 with high probability.

$\mathcal{A}$ usually outputs 0 for family $\mathcal{C}$.

With overwhelming probability, $\mathcal{A}$ outputs 0 for function sequences in $\mathcal{C}$ (Lemma 4). This is because they describe constant function, which do not entangle the input with the output.

Lemma 4. *There is a negligible function* $\mathsf{negl}(\lambda)$ *such that for any* $\lambda \in \mathbb{N}$ *and any* $c \in \mathcal{C}_\lambda$,

$$\Pr[\mathcal{A}(1^\lambda, \mathsf{OTP}^*(c)) \to 1] \leq \mathsf{negl}(\lambda)$$

Proof. c is described by $(\mathsf{pk}, r_{\mathsf{Enc}})$, and for any input $x \in \{0,1\}^\lambda$, c outputs $y_c := \mathsf{Enc}(\mathsf{pk}, 0^\lambda; r_{\mathsf{Enc}})$.

Next, since OTP^* is a one-time program compiler for $\mathcal{F}$ (definition 11), $\mathsf{OTP}^*(c)$ implements c with negligible error. Formally, we say there is a negligible function $\mu(\lambda)$ such that for any $\lambda \in \mathbb{N}$, any $c \in \mathcal{C}_\lambda$, and any $x \in \{0,1\}^\lambda$, when $\mathsf{OTP}^*(c)$ is evaluated on x, it outputs y_c with probability $\geq 1 - \mu(\lambda)$.

After step 2 of $\mathcal{A}$ (which evaluates the program), let us condition on the event that $\mathcal{Y}$ contains y_c. This event occurs with overwhelming probability, so conditioning on this event changes the state of the system and the probability that $\mathcal{A}$ outputs 1 by negligible amounts.

Then step 3 copies the value y_c over to the $\mathcal{Y}'$ register. Now we can trace out (forget about) the $\mathcal{Y}'$ register. Step 3 does not change the state on the remaining registers $\mathcal{X}' \times \mathcal{X} \times \mathcal{Y} \times \mathcal{P}$ because the value copied to $\mathcal{Y}'$ is deterministic.

Step 4 uncomputes step 2 and returns the state of $\mathcal{X}' \times \mathcal{X} \times \mathcal{Y} \times \mathcal{P}$ to be negligibly close to their initial state.

Step 5 checks whether the state on $\mathcal{X}' \times \mathcal{X}$ is the same as the initial state $|{+}{+}\rangle$. This check will accept with overwhelming probability because the state on $\mathcal{X}' \times \mathcal{X}$ is negligibly close to the initial state. Then $\mathcal{A}(1^\lambda, \mathsf{OTP}^*(c))$ outputs 0 with overwhelming probability and 1 with negligible probability. □

$\mathcal{A}$ usually outputs 1 for family $\mathcal{E}$.

With overwhelming probability, $\mathcal{A}$ outputs 1 for functions sampled from $\mathcal{E}$ (Lemma 5). This is because with overwhelming probability, the function we sample is injective, so it will entangle the input with the output.

Lemma 5. *For any given $\lambda \in \mathbb{N}$, let $e \in \mathcal{E}_\lambda$ be chosen by sampling a key $\mathsf{pk} \leftarrow \mathsf{GenPK}(1^\lambda, \mathsf{inj})$ and setting $e = \mathsf{Enc}(\mathsf{pk}, \cdot; \cdot)$. Next, there is a negligible function $\mathsf{negl}(\lambda)$ such that over the randomness of sampling e and the randomness of $\mathcal{A}$ and OTP^*,*

$$\Pr[\mathcal{A}(1^\lambda, \mathsf{OTP}^*(e)) \to 1] \geq 1 - \mathsf{negl}(\lambda)$$

Proof. For any $x \in \{0,1\}^\lambda$, let $\mathcal{Y}_x$ comprise all y-values in the support of $e(x)$. With overwhelming probability over the sampling of $\mathsf{pk} \leftarrow \mathsf{GenPK}(1^\lambda, \mathsf{inj})$, e is injective, meaning the sets $\{\mathcal{Y}_x\}_{x \in \{0,1\}^\lambda}$ are disjoint. This follows from the correctness property of lossy PKE (Definition 22).

Next, since OTP^* is a one-time program compiler for $\mathcal{F}$ (definition 11), $\mathsf{OTP}^*(e)$ implements e with negligible error. That means there exists a negligible function $\mu(\lambda)$ such that for any $\lambda \in \mathbb{N}$, any injective $e \in \mathcal{E}_\lambda$, and any $x \in \{0,1\}^\lambda$, when $\mathsf{OTP}^*(e)$ is evaluated on x, it outputs a value in $\mathcal{Y}_x$ with probability $\geq 1 - \mu(\lambda)$.

Let us step through the execution of $\mathcal{A}(1^\lambda, \mathsf{OTP}^*(e))$ and condition on the event that e is injective.

After step 2 of $\mathcal{A}$ (which evaluates the program), let us condition on the event that the $\mathcal{X}' \times \mathcal{Y}$ registers contain values (x, y) such that $y \in \mathcal{Y}_x$. This event occurs with overwhelming probability because $\mathcal{X}'$ contains the value x that was inputted to $\mathsf{OTP}^*(e)$, and $\mathcal{Y}$ contains the output of $\mathsf{OTP}^*(e)$. When we condition on this event, we change the state of the system and the probability that $\mathcal{A}$ outputs 1 by negligible amounts.

Step 3 CNOTs the value on the $\mathcal{Y}$ register onto the $\mathcal{Y}'$ register. At the end of this step, the $\mathcal{X}' \times \mathcal{Y}'$ registers contain values (x, y) such that $y \in \mathcal{Y}_x$.

Step 4 acts only on the $\mathcal{X} \times \mathcal{Y} \times \mathcal{P}$ registers and does not touch the $\mathcal{X}' \times \mathcal{Y}'$ registers. After this step, it is still true that $\mathcal{X}' \times \mathcal{Y}'$ contain values (x, y) such that $y \in \mathcal{Y}_x$.

Step 5 checks whether the $\mathcal{X} \times \mathcal{X}'$ registers are in the state $|{+}{+}\rangle$. Let Π_{++} be the corresponding

projector acting on $\mathcal{X} \times \mathcal{X}' \times \mathcal{Y}'$:

$$\begin{aligned}\Pi_{++} &:= |++\rangle\langle++|_{\mathcal{X}\times\mathcal{X}'} \otimes \mathbb{I}_{\mathcal{Y}'} \\ &= \frac{1}{2^\lambda}\sum_{x,x'} |x\rangle\langle x'|_{\mathcal{X}} \otimes |x\rangle\langle x'|_{\mathcal{X}'} \otimes \mathbb{I}_{\mathcal{Y}'}\end{aligned}$$

Next, let Π_{inj} be a projector that acts on $\mathcal{X} \times \mathcal{X}' \times \mathcal{Y}'$ and projects onto all states in which $\mathcal{X}' \times \mathcal{Y}'$ contain values (x, y) such that $y \in \mathcal{Y}_x$:

$$\Pi_{\mathsf{inj}} = \sum_x \sum_{y\in\mathcal{Y}_x} \mathbb{I}_{\mathcal{X}} \otimes |x\rangle\langle x|_{\mathcal{X}'} \otimes |y\rangle\langle y|_{\mathcal{Y}'}$$

Let ρ be the state of the system at the start of step 5 on the $\mathcal{X} \times \mathcal{X}' \times \mathcal{Y}'$ registers. We know that

$$\rho = \Pi_{\mathsf{inj}}\rho\Pi_{\mathsf{inj}}$$

because the $\mathcal{X}' \times \mathcal{Y}'$ registers of ρ only contain values (x, y) such that $y \in \mathcal{Y}_x$.

Next,

$$\begin{aligned}\Pr[\mathcal{A} \to 0] &= \mathsf{Tr}\left[\Pi_{++}\rho\right] \\ &= \mathsf{Tr}\left[\Pi_{++}\left(\Pi_{\mathsf{inj}}\rho\Pi_{\mathsf{inj}}\right)\right] \\ &= \mathsf{Tr}\left[\Pi_{\mathsf{inj}} \cdot \Pi_{++} \cdot \Pi_{\mathsf{inj}} \cdot \rho\right] \\ &\leq \|\Pi_{\mathsf{inj}} \cdot \Pi_{++} \cdot \Pi_{\mathsf{inj}}\|\end{aligned}$$

Next,

$$\begin{aligned}\Pi_{\mathsf{inj}} \cdot \Pi_{++} \cdot \Pi_{\mathsf{inj}} &= \left(\sum_x \sum_{y\in\mathcal{Y}_x} \mathbb{I} \otimes |x\rangle\langle x| \otimes |y\rangle\langle y|\right) \cdot \left(\frac{1}{2^\lambda}\sum_{x'',x'''} |x''\rangle\langle x'''| \otimes |x''\rangle\langle x'''| \otimes \mathbb{I}\right) \\ &\quad \cdot \left(\sum_{x'} \sum_{y'\in\mathcal{Y}_{x'}} \mathbb{I} \otimes |x'\rangle\langle x'| \otimes |y'\rangle\langle y'|\right) \\ &= \frac{1}{2^\lambda} \cdot \sum_{x,x',x'',x'''} \sum_{y\in\mathcal{Y}_x} \sum_{y'\in\mathcal{Y}_{x'}} \left(\mathbb{I} \cdot |x''\rangle\langle x'''| \cdot \mathbb{I}\right)_{\mathcal{X}} \\ &\quad \otimes \left(|x\rangle\langle x|\,|x''\rangle\langle x'''|\,|x'\rangle\langle x'|\right)_{\mathcal{X}'} \otimes \left(|y\rangle\langle y| \cdot \mathbb{I} \cdot |y'\rangle\langle y'|\right)_{\mathcal{Y}'} \\ &= \frac{1}{2^\lambda} \cdot \sum_{x,x'} \sum_{y\in\mathcal{Y}_x\cap\mathcal{Y}_{x'}} |x\rangle\langle x'| \otimes |x\rangle\langle x'| \otimes |y\rangle\langle y| \\ &= \frac{1}{2^\lambda} \cdot \sum_x \sum_{y\in\mathcal{Y}_x} |x\rangle\langle x| \otimes |x\rangle\langle x| \otimes |y\rangle\langle y|\end{aligned}$$

We used the fact that $\mathcal{Y}_x \cap \mathcal{Y}_{x'} = \emptyset$ if $x \neq x'$. Continuing on,

$$\begin{aligned}\Pi_{\mathsf{inj}} \cdot \Pi_{++} \cdot \Pi_{\mathsf{inj}} &= \frac{1}{2^\lambda} \cdot \sum_x \sum_{y\in\mathcal{Y}_x} |x, x, y\rangle\langle x, x, y| \\ \|\Pi_{\mathsf{inj}} \cdot \Pi_{++} \cdot \Pi_{\mathsf{inj}}\| &= \frac{1}{2^\lambda} \\ &= \mathsf{negl}(\lambda)\end{aligned}$$

$$\begin{aligned}\Pr[\mathcal{A} \to 0] &\leq \|\Pi_{\mathsf{inj}} \cdot \Pi_{++} \cdot \Pi_{\mathsf{inj}}\| \\ &= \mathsf{negl}(\lambda) \\ \Pr[\mathcal{A} \to 1] &\geq 1 - \mathsf{negl}(\lambda)\end{aligned}$$

□

$\mathcal{A}$ has contradictory behavior for family $\mathcal{D}$.

How likely is $\mathcal{A}$ to output 1 for functions sampled from $\mathcal{D}$? On one hand, we can show that $\mathcal{A}$ will output 1 with overwhelming probability (Lemma 6). This is because functions sampled from $\mathcal{D}$ are indistinguishable from functions sampled from $\mathcal{E}$, so $\mathcal{A}$'s behavior should be similar for these two families.

On the other hand, we can show that $\mathcal{A}$ outputs 0 with overwhelming probability (Lemma 7). This is because d can be implemented by a mixture over functions in $\mathcal{C}$. The simulator, given this mixture, will output 0 with overwhelming probability. That implies that $\mathcal{A}$, given $\mathsf{OTP}^*(d)$, will also output 0 with overwhelming probability. We've reached a contradiction, so in fact, the family $\mathcal{F}$ does not have a best-possible one-time program compiler.

For any given $\lambda \in \mathbb{N}$, let $d \in \mathcal{D}_\lambda$ be chosen by sampling a key $\mathsf{pk} \leftarrow \mathsf{GenPK}(1^\lambda, \mathsf{lossy})$ and setting $d = \mathsf{Enc}(\mathsf{pk}, \cdot; \cdot)$.

Lemma 6. *There is a negligible function* $\mathsf{negl}(\lambda)$ *such that over the randomness of sampling* d *and the randomness of* $\mathcal{A}$ *and* OTP^**,*

$$\Pr[\mathcal{A}(1^\lambda, \mathsf{OTP}^*(d)) \to 1] \geq 1 - \mathsf{negl}(\lambda)$$

Proof. Otherwise, we could use $\mathcal{A}$ and OTP^* to break the indistinguishability of modes property of lossy PKE (Definition 22). This property says that the output distributions of $\mathsf{GenPK}(1^\lambda, \mathsf{inj})$ and $\mathsf{GenPK}(1^\lambda, \mathsf{lossy})$ are indistinguishable to any QPT adversary.

Let us construct a QPT adversary that tries to distinguish these two distributions. Given a pk sampled from either $\mathsf{GenPK}(1^\lambda, \mathsf{inj})$ or $\mathsf{GenPK}(1^\lambda, \mathsf{lossy})$, let us define the function $f_{\mathsf{pk}}(x; r) = \mathsf{Enc}(\mathsf{pk}, x; r)$. If pk is lossy, then $f_{\mathsf{pk}} \in \mathcal{D}_\lambda$, and if pk is injective, then $f_{\mathsf{pk}} \in \mathcal{E}_\lambda$. Next, let our distinguisher compute $\mathcal{A}(1^\lambda, \mathsf{OTP}^*(f_{\mathsf{pk}}))$ and output the result. Since $\mathcal{A}$ and OTP^* are QPT, our distinguisher is QPT as well. For each $\mathsf{mode} \in \{\mathsf{inj}, \mathsf{lossy}\}$, the probability that the distinguisher outputs 1 is:

$$\Pr_{\mathsf{pk} \leftarrow \mathsf{GenPK}(1^\lambda, \mathsf{mode})}[\mathcal{A}(1^\lambda, \mathsf{OTP}^*(f_{\mathsf{pk}})) \to 1]$$

By the indistinguishability of modes,

$$\left| \Pr_{\mathsf{pk} \leftarrow \mathsf{GenPK}(1^\lambda, \mathsf{inj})}[\mathcal{A}(1^\lambda, \mathsf{OTP}^*(f_{\mathsf{pk}})) \to 1] - \Pr_{\mathsf{pk} \leftarrow \mathsf{GenPK}(1^\lambda, \mathsf{lossy})}[\mathcal{A}(1^\lambda, \mathsf{OTP}^*(f_{\mathsf{pk}})) \to 1] \right| \leq \mathsf{negl}(\lambda)$$

Lemma 5 implies that in injective mode, the distinguisher will output 1 with probability $\geq 1 - \mathsf{negl}(\lambda)'$:

$$\Pr_{\mathsf{pk} \leftarrow \mathsf{GenPK}(1^\lambda, \mathsf{inj})}[\mathcal{A}(1^\lambda, \mathsf{OTP}^*(f_{\mathsf{pk}})) \to 1] \geq 1 - \mathsf{negl}(\lambda)'$$

Therefore,

$$\Pr_{\mathsf{pk} \leftarrow \mathsf{GenPK}(1^\lambda, \mathsf{lossy})}[\mathcal{A}(1^\lambda, \mathsf{OTP}^*(f_{\mathsf{pk}})) \to 1] \geq 1 - \mathsf{negl}(\lambda)''$$

□

Lemma 7. *There is a negligible function* $\mathsf{negl}(\lambda)$ *such that over the randomness of sampling* d *and the randomness of* $\mathcal{A}$ *and* OTP^**,*

$$\Pr[\mathcal{A}(1^\lambda, \mathsf{OTP}^*(d)) \to 1] \leq \mathsf{negl}(\lambda)$$

Proof. d is described by a pk in the support of $\mathsf{GenPK}(1^\lambda, \mathsf{lossy})$. Note that $d(x) = \mathsf{Enc}(\mathsf{pk}, x; r)$ for a random r. The statistical/perfect lossiness property of the encryption scheme (Definition 22) implies that there is a negligible/identically zero function $\mu(\lambda)$ such that with overwhelming probability over the sampling of pk, the following is true for every $x \in \{0,1\}^\lambda$: the distributions of $\mathsf{Enc}(\mathsf{pk}, x)$ and $\mathsf{Enc}(\mathsf{pk}, 0^\lambda)$ (over the randomness of Enc) are $\mu(\lambda)$-close in statistical distance. Let us assume from now on that this is the case.

Furthermore, for any $r \in \mathcal{R}_{\mathsf{Enc}}$, (pk, r) describe a function in $\mathcal{C}_\lambda$. Let us define some sampling programs that compute functions in $\mathcal{C}_\lambda$ and $\mathcal{D}_\lambda$.

$$\text{Let } \rho_r = |r\rangle\langle r|$$
$$\rho = \frac{1}{|\mathcal{R}_{\mathsf{Enc}}|} \sum_{r \in \mathcal{R}_{\mathsf{Enc}}} \rho_r$$

Next, let $\mathsf{Eval}_{\mathsf{pk}}$ be a circuit that maps

$$|x, 0, r\rangle \xrightarrow{\mathsf{Eval}_{\mathsf{pk}}} |x, \mathsf{Enc}(\mathsf{pk}, 0^\lambda; r), r\rangle$$

Finally, let us define the sampling programs:

$$\text{Let } P_r = (\rho_r, \mathsf{Eval}_{\mathsf{pk}})$$
$$P = (\rho, \mathsf{Eval}_{\mathsf{pk}})$$

Let the descriptions of P_r and P be padded so that $|P_r|$ and $|P|$ equal the description length $|f|$ of any function $f \in \mathcal{F}_\lambda$.

Claim 1. *For any $r \in \mathcal{R}_{\mathsf{Enc}}$, P_r implements a function in $\mathcal{C}_\lambda$ with 0 error.*

Proof. If we evaluate P_r on any input x, the output will be $\mathsf{Enc}(\mathsf{pk}, 0^\lambda; r)$. This is exactly the same output distribution as a function $c \in \mathcal{C}_\lambda$. □

Claim 2. *P implements d with error μ.*

Proof. The program state ρ is a uniform mixture over the values $r \in \mathcal{R}_{\mathsf{Enc}}$. For any input x, $P(x)$ is distributed as $\mathsf{Enc}(\mathsf{pk}, 0^\lambda; r)$ for a random $r \xleftarrow{\$} \mathcal{R}_{\mathsf{Enc}}$. Additionally, $d(x)$ is distributed as $\mathsf{Enc}(\mathsf{pk}, x; r)$ for a random $r \xleftarrow{\$} \mathcal{R}_{\mathsf{Enc}}$. By the lossiness of the encryption scheme, the distributions of $P(x)$ and $d(x)$ are $\mu(\lambda)$-close in statistical distance. Therefore, P implements d with error μ. □

Since OTP^* is best-possible for $\mathcal{F}$ with statistical/perfect equivalence (definition 12), there is a QPT simulator Sim and there is a negligible function $\nu(\lambda)$ such that for any $\lambda \in \mathbb{N}$, any function $f \in \mathcal{F}_\lambda$, and any sampling program P that implements f with error $\mu(\lambda)$ and satisfies $|P| = |f|$, the outputs of $\mathcal{A}(1^\lambda, \mathsf{OTP}^*(f))$ and $\mathsf{Sim}(1^\lambda, P)$ are computationally indistinguishable, and in particular:

$$\left|\Pr[\mathcal{A}(1^\lambda, \mathsf{OTP}^*(f)) \to 1] - \Pr[\mathsf{Sim}(1^\lambda, P) \to 1]\right| \leq \nu(\lambda)$$

Lemma 4 says that for any $c \in \mathcal{C}_\lambda$,

$$\Pr[\mathcal{A}(1^\lambda, \mathsf{OTP}^*(c)) \to 1] = \mathsf{negl}(\lambda)$$

Then since P_r implements c with 0 error,

$$\Pr[\mathsf{Sim}(1^\lambda, P_r) \to 1] \leq \mathsf{negl}'(\lambda)$$

Next, the quantum part of P is ρ, and it is a mixture over states $\{\rho_r\}_{r\in\mathcal{R}_{\mathsf{Enc}}}$. Then

$$\begin{aligned}\Pr[\mathsf{Sim}(1^\lambda, P)\to 1] &= \frac{1}{|\mathcal{R}_{\mathsf{Enc}}|}\sum_{r\in\mathcal{R}_{\mathsf{Enc}}}\Pr[\mathsf{Sim}(1^\lambda,P_r)\to 1]\\ &\leq \mathsf{negl}'(\lambda)\end{aligned}$$

Finally, since P implements d with error μ,

$$\begin{aligned}\Pr[\mathcal{A}(1^\lambda,\mathsf{OTP}^*(d))\to 1] &\leq \Pr[\mathsf{Sim}(1^\lambda,P)\to 1]+\mathsf{negl}''(\lambda)\\ &\leq \mathsf{negl}'''(\lambda)\end{aligned}$$

□

We have now reached a contradiction. Lemma 7 says that

$$\Pr[\mathcal{A}(1^\lambda,\mathsf{OTP}^*(d))\to 1]\leq \mathsf{negl}(\lambda)$$

but Lemma 6 says that

$$\Pr[\mathcal{A}(1^\lambda,\mathsf{OTP}^*(d))\to 1]\geq 1-\mathsf{negl}(\lambda)$$

Therefore, the initial assumption must be false, and in fact, there does not exist a one-time program compiler that is best-possible for $\mathcal{F}$ with statistical/perfect equivalence.

Relativization The relativization follows from the fact that if we replace the steps in the above proof with steps with access to an (arbitrary unitary) oracle machine where the lossy encryption remains post-quantum secure, the argument still goes through. All the steps are information-theoretic except invoking the computational security of lossy encryption. As shown in the preliminaries, lossy encryption can also be derived from LWE/weak pseudorandomness of group actions in a black-box way where each step can be replaced with an oracle-assisted step as long as the oracle does not help solve LWE/weak pseudorandomness of group actions.

4 SEQ Implies Best-Possible Testable One-Time Security

Definition 13 (Testable quantum program)**.** *A quantum program $P=(|\psi\rangle,\mathsf{Eval})$ specified by a pure state $|\psi\rangle$ and a unitary* Eval *can be augmented with a reflection unitary*[7]

$$R=\mathsf{X}\otimes|\psi\rangle\langle\psi|+I\otimes(I-|\psi\rangle\langle\psi|)$$

We say that the resulting program $P'=(|\psi\rangle,\mathsf{Eval},R)$ is testable.

A testable one-time program compiler is a one-time program compiler that outputs testable quantum programs.

Note that it makes sense only to define a reflection oracle (the test oracle) about a pure state $|\psi\rangle$. For simplicity, we assume that the reflection unitary R is provided in the form of an oracle, or the classical description of a unitary; rather than a quantum program itself with a quantum auxiliary input, so that it does not degrade over uses.

Next we revise and generalize the SEQ definition of [GLR+25] to all quantum functionalities and adopt the name SEQ for this generalized notion. To distinguish from the prior work, we refer to their original, classical version as CSEQ. Intuitively, the SEQ oracle embeds a single effective application of the purified channel into a globally unitary, repeatable interface.

[7] Typically, the reflection oracle is defined as $R'=I-2|\psi\rangle\langle\psi|$, but we adopt a different (but equivalent) definition for convenience. To see why R and R' are equivalent, note that R' can be implemented by first applying R to $|0\rangle|\phi\rangle$, applying a Z gate to the first register, and then applying R again to uncompute the first register. $R|b\rangle|\phi\rangle$ can be implemented as follows: apply H to the first register, then controlled on the first register containing 1, apply R' to the state $|\phi\rangle$, and finally apply H to the first register again.

Stinespring form and notation. Let $\Phi : \mathsf{L}(\mathcal{H}_Q) \to \mathsf{L}(\mathcal{H}_Q)$ be a quantum channel. Without loss of generality (by padding with dummy qubits if needed), assume the Stinespring dilation has matching input/output dimensions, so we fix a Stinespring unitary $U_\Phi : \mathcal{H}_Q \otimes \mathcal{H}_P \to \mathcal{H}_Q \otimes \mathcal{H}_P$ with the private register P initialized to $|0^m\rangle_P$ as expected by the purification. Tracing out the private register yields Φ on the query register.

Definition 14 (The Single Effective Query Oracle (SEQ))**.** *For a channel Φ with fixed purification U_Φ as above, the single effective query oracle O_Φ^{SEQ} acts on a query register $\mathcal{Q}$ and maintains a private register $\mathcal{P}$ and a one-qubit computed flag $\mathcal{C}$. The registers are initialized to $|0^m\rangle_\mathcal{P} \otimes |0\rangle_\mathcal{C}$. On each query, the oracle applies the following unitary on $(\mathcal{Q}, \mathcal{P}, \mathcal{C})$:*

1. *Controlled on $\mathcal{C}$=1, apply $U_\Phi^\dagger$ to $(\mathcal{Q}, \mathcal{P})$.*
2. *Apply the swap on the subspace spanned by $|0\rangle_\mathcal{C} \otimes |0^m\rangle_\mathcal{P}$ and $|1\rangle_\mathcal{C} \otimes |0^m\rangle_\mathcal{P}$ (and act as the identity on the orthogonal subspace). Equivalently, flip $\mathcal{C}$ controlled on $\mathcal{P}$= $|0^m\rangle$.*
3. *Controlled on $\mathcal{C}$=1, apply U_Φ to $(\mathcal{Q}, \mathcal{P})$.*

O_Φ^{SEQ} may also be called $O_{U_\Phi}^{\mathsf{SEQ}}$ to make the particular Stinespring dilation U_Φ explicit.

It is often unnecessary to write $O_{U_\Phi}^{\mathsf{SEQ}}$, which makes the particular dilation U_Φ explicit, and instead O_Φ^{SEQ} suffices. Lemma 11 says that any two Stinespring dilations U, U' of Φ will produce indistinguishable oracles O_U^{SEQ} and $O_{U'}^{\mathsf{SEQ}}$.

The above makes the SEQ interface unitary-by-construction. Intuitively, the first time we query, Step 1 is inactive, Step 2 flips the computed flag, and Step 3 applies U_Φ. On subsequent queries, Steps 1 and 3 cancel on $(\mathcal{Q}, \mathcal{P})$, and Step 2 flips the flag only in the $\mathcal{P}$= $|0^m\rangle$ subspace; the overall interface remains unitary and well-defined.

In fact, we can verify that this unitary exactly maps the well-initialized input to its output and vice versa, while acting as identity on everything else.

Definition 15 (SEQ-based simulation security for one-time programs)**.** *A one-time program compiler* OTP *satisfies* SEQ*-based simulation security for a class $\mathcal{C}$ of quantum channels if there exists a q.p.t. simulator* Sim *such that for every $\Phi \in \mathcal{C}$ and for every q.p.t. distinguisher D, there exists a negligible function* $\mathsf{negl}(\lambda)$ *with*

$$\left|\Pr\left[1 \leftarrow D(1^\lambda, \Phi, \mathsf{OTP}(1^\lambda, \Phi))\right] - \Pr\left[1 \leftarrow D(1^\lambda, \Phi, \mathsf{Sim}^{O_\Phi^{\mathsf{SEQ}}}(1^\lambda))\right]\right| \leq \mathsf{negl}(\lambda).$$

As before, giving Φ to D means that D may depend on Φ (e.g., via a classical description or black-box evaluation access); this redundancy matches the order of quantifiers.

We now define the notion of a best-possible one-time program compiler, which produces one-time programs that are "best-possible" among all programs that implement the same sampling task and are testable.

Definition 16 (Best-possible OTP among testable programs)**.** *Let $\mathcal{C}$ be a family of quantum channels. A testable one-time program compiler OTP^* is* ***best-possible among testable programs*** *for $\mathcal{C}$ if there exists a q.p.t. simulator* Sim *such that for every $\Phi \in \mathcal{C}$, for every testable quantum program P that implements Φ, and for every q.p.t. distinguisher D,*

$$\left|\Pr\left[1 \leftarrow D(1^\lambda, \Phi, \mathsf{OTP}^*(1^\lambda, \Phi))\right] - \Pr\left[1 \leftarrow D(1^\lambda, \Phi, \mathsf{Sim}(1^\lambda, P))\right]\right| \leq \mathsf{negl}(\lambda).$$

Here, "P implements Φ" means that tracing out P's private register after applying Eval *to the program state $|\psi\rangle$ realizes the channel Φ on the external interface.*

Theorem 9. *Any one-time program compiler* OTP^* *satisfying* SEQ *security for a class of quantum channels (Definition 15) is a best-possible one-time program compiler among testable programs for that class (Definition 16). Moreover, this implication relativizes to all unitary oracles.*

Proof. Since OTP^* satisfies SEQ security, there exists a q.p.t. simulator Sim such that for every channel Φ and every q.p.t. distinguisher D,

$$\left|\Pr[1 \leftarrow D(1^\lambda, \Phi, \mathsf{OTP}^*(1^\lambda, \Phi))] - \Pr[1 \leftarrow D(1^\lambda, \Phi, \mathsf{Sim}^{O_\Phi^{\mathsf{SEQ}}}(1^\lambda))]\right| \leq \mathsf{negl}(\lambda).$$

It therefore suffices to show that given any testable implementation P of Φ, we can simulate oracle access to O_Φ^{SEQ} using P. Composing Sim with this wrapper yields the simulator required by Definition 16.

Let $P = (|\psi\rangle, \mathsf{Eval}, R)$ be a testable quantum program that implements Φ, where $R = \mathsf{X} \otimes |\psi\rangle\langle\psi| + I \otimes (I - |\psi\rangle\langle\psi|)$ acts on a one-qubit flag and the program register. Without loss of generality, we can think about $|\psi\rangle$ padded with zero qubits which are used for the auxiliary wires, and the reflection unitary is extended to also reflect around zeroes for those qubits. The following simulator Sim' takes any such testable program P that implements Φ and simulates O_Φ^{SEQ}.

Definition 17 (Simulator $\mathsf{Sim}'(P)$ for O_Φ^{SEQ} from a testable program.)**.** *The oracle* $\mathsf{Sim}'(P)$ *maintains internally the program register* $\mathcal{P}$ *initialized to* $|\psi\rangle$ *and a one-qubit computed flag* $\mathcal{C}$ *initialized to* $|0\rangle$*. On each query on register* $\mathcal{Q}$*,* $\mathsf{Sim}'(P)$ *applies the following unitary on* $(\mathcal{Q}, \mathcal{P}, \mathcal{C})$*:*

1. *Controlled on* $\mathcal{C}$*=*1*, apply* $\mathsf{Eval}^\dagger$ *to* $(\mathcal{Q}, \mathcal{P})$*, mapping back to the input space of* Eval*.*
2. *Apply the reflection-controlled flip* R *to* $(\mathcal{C}, \mathcal{P})$*, i.e., flip* $\mathcal{C}$ *iff the program register equals* $|\psi\rangle$*.*
3. *Controlled on* $\mathcal{C}$*=*1*, apply* Eval *to* $(\mathcal{Q}, \mathcal{P})$*.*

We now argue that $\mathsf{Sim}'(P)$ is perfectly indistinguishable from O_Φ^{SEQ}. We write $\mathcal{A}$ for the adversary's private workspace register, which the adversary may initialize and act upon arbitrarily; the oracle's hidden registers $(\mathcal{C}, \mathcal{P})$ remain inaccessible.

Lemma 8. *Let* P *be any (potentially oracle-aided)*[8] *testable quantum program that implements* Φ*. Then oracles for* $\mathsf{Sim}'(P)$ *(definition 17) and* O_Φ^{SEQ} *are perfectly indistinguishable after any number of quantum queries.*

Proof. Let U_Φ be the canonical Stinespring dilation unitary for Φ. O_Φ^{SEQ} involves applying U_Φ. We will refer to O_Φ^{SEQ} as $O_{U_\Phi}^{\mathsf{SEQ}}$ to distinguish it from a similar oracle that applies a different unitary.

We will show that $\mathsf{Sim}'(P)$ is equivalent to $O_{U'_\Phi}^{\mathsf{SEQ}}$, for a particular Stinespring dilation U'_Φ of the channel Φ. Note that the Stinespring dilation needs $\mathcal{P}$ to be initialized to $|0^m\rangle$, whereas $\mathsf{Sim}'(P)$ initializes it to $|\psi\rangle$. We handle this discrepancy by conjugating each step of $\mathsf{Sim}'(P)$ with a swap operation that swaps the states $|0^m\rangle$ and $|\psi\rangle$.

Let U'_Φ be the following unitary:

1. Swap $|0^m\rangle_\mathcal{P}$ and $|\psi\rangle_\mathcal{P}$.
2. Apply Eval to $(\mathcal{Q}, \mathcal{P})$.
3. Swap $|0^m\rangle_\mathcal{P}$ and $|\psi\rangle_\mathcal{P}$.

Lemma 9. U'_Φ *is a valid Stinespring dilation of* Φ*.*

[8] P may query an oracle that maintains a quantum state as long as the reflection oracle correctly reflects around the initial state of the oracle.

Proof. Given a state ρ on the query register $\mathcal{Q}$, consider the following three procedures for handling the query:

1. Apply channel Φ to $\rho_{\mathcal{Q}}$.
2. Apply Eval to $\rho_{\mathcal{Q}} \otimes |\psi\rangle_{\mathcal{P}}$ and then trace out $\mathcal{P}$.
3. Apply U'_Φ to $\rho_{\mathcal{Q}} \otimes |0^m\rangle_{\mathcal{P}}$ and then trace out $\mathcal{P}$.

Procedure 1 is equivalent to procedure 2 because P implements Φ. Next, procedure 2 is equivalent to procedure 3 because U'_Φ maps $|0^m\rangle_{\mathcal{P}} \to |\psi\rangle_{\mathcal{P}}$ before applying Eval. This shows that U'_Φ is a Stinespring dilation of Φ. □

Let $O^{\mathsf{SEQ}}_{U'_\Phi}$ be the SEQ oracle (definition 14) that uses unitary U'_Φ instead of U_Φ.

Lemma 10. *$O^{\mathsf{SEQ}}_{U'_\Phi}$ and $\mathsf{Sim}'(P)$ are perfectly indistinguishable after any number of quantum queries.*

Proof. Let's consider the following hybrids, which transform $O^{\mathsf{SEQ}}_{U'_\Phi}$ to $\mathsf{Sim}'(P)$:

Hybrid 1 – $O^{\mathsf{SEQ}}_{U'_\Phi}$ – Start with state $|0\rangle_{\mathcal{C}} |0^m\rangle_{\mathcal{P}}$ and handle each query as follows:

1. Controlled on $\mathcal{C}$=1, apply ${U'_\Phi}^\dagger$ as follows:
   (a) Swap $|0^m\rangle_{\mathcal{P}}$ and $|\psi\rangle_{\mathcal{P}}$.
   (b) Apply $\mathsf{Eval}^\dagger$ to $(\mathcal{Q}, \mathcal{P})$.
   (c) Swap $|0^m\rangle_{\mathcal{P}}$ and $|\psi\rangle_{\mathcal{P}}$.
2. Controlled on $\mathcal{P}$= $|0^m\rangle$, flip $\mathcal{C}$.
3. Controlled on $\mathcal{C}$=1, apply U'_Φ as follows:
   (a) Swap $|0^m\rangle_{\mathcal{P}}$ and $|\psi\rangle_{\mathcal{P}}$.
   (b) Apply Eval to $(\mathcal{Q}, \mathcal{P})$.
   (c) Swap $|0^m\rangle_{\mathcal{P}}$ and $|\psi\rangle_{\mathcal{P}}$.

Hybrid 2: Start with state $|0\rangle_{\mathcal{C}} |0^m\rangle_{\mathcal{P}}$ and handle each query as follows:

1. Controlled on $\mathcal{C}$=1:
   (a) Swap $|0^m\rangle_{\mathcal{P}}$ and $|\psi\rangle_{\mathcal{P}}$.
   (b) Apply $\mathsf{Eval}^\dagger$ to $(\mathcal{Q}, \mathcal{P})$.
   (c) Swap $|0^m\rangle_{\mathcal{P}}$ and $|\psi\rangle_{\mathcal{P}}$.
2. (a) Swap $|0^m\rangle_{\mathcal{P}}$ and $|\psi\rangle_{\mathcal{P}}$.
   (b) Controlled on $\mathcal{P}$=$|\psi\rangle$, flip $\mathcal{C}$.
   (c) Swap $|0^m\rangle_{\mathcal{P}}$ and $|\psi\rangle_{\mathcal{P}}$.
3. Controlled on $\mathcal{C}$=1:
   (a) Swap $|0^m\rangle_{\mathcal{P}}$ and $|\psi\rangle_{\mathcal{P}}$.
   (b) Apply Eval to $(\mathcal{Q}, \mathcal{P})$.
   (c) Swap $|0^m\rangle_{\mathcal{P}}$ and $|\psi\rangle_{\mathcal{P}}$.

Hybrids 1 and 2 are equivalent. The only difference is step 2. In hybrid 1, we control on $\mathcal{P} = |0^m\rangle$. In hybrid 2, we swap $|0^m\rangle$ with $|\psi\rangle$ and control on $\mathcal{P} = |\psi\rangle$. These procedures implement the same operation.

Hybrid 3 – $\mathsf{Sim}'(P)$ – Start with state $|0\rangle_{\mathcal{C}} |\psi\rangle_{\mathcal{P}}$ and handle each query as follows:

1. Controlled on $\mathcal{C}$=1, apply $\mathsf{Eval}^\dagger$ to $(\mathcal{Q}, \mathcal{P})$.
2. Controlled on $\mathcal{P}$=$|\psi\rangle$, flip $\mathcal{C}$.
3. Controlled on $\mathcal{C}$=1, apply Eval to $(\mathcal{Q}, \mathcal{P})$.

The difference between hybrids 2 and 3 is that in hybrid 3, we have omitted all the swap operations (the steps that swap $|0^m\rangle$ and $|\psi\rangle$), and the initial state of $\mathcal{P}$ is $|\psi\rangle$, not $|0^m\rangle$.

We will argue that hybrids 2 and 3 are perfectly indistinguishable. Given any user that submits a sequence of queries to the oracle in hybrids 2 or 3, we can view their sequence of queries as a sequence of invocations of steps 1, 2, and 3. We will prove the following invariant: at the start or end of any step in the sequence, the current state of the system in hybrid 2 can be mapped to the current state of the system in hybrid 3 by applying the swap operation to $\mathcal{P}$, which swaps $|0^m\rangle$ and $|\psi\rangle$.

First, before the first step of the first query, the state of the oracle's registers are $|0\rangle_{\mathcal{C}} |0^m\rangle_{\mathcal{P}}$ in hybrid 2 and $|0\rangle_{\mathcal{C}} |\psi\rangle_{\mathcal{P}}$ in hybrid 3, so the invariant is satisfied at this point.

Second, let us assume the invariant is satisfied at the start of some invocation of step 1, and let's step through the execution of step 1 to show that the invariant is satisfied at the end. If $\mathcal{C} = 0$, then step 1 acts as the identity in both hybrids, so the invarinat will be satisfied at the end. Next, let's consider the case where $\mathcal{C} = 1$. In hybrid 2, step 1 applies the swap operation, which transforms the state to match the initial state of hybrid 3. Then it applies $\mathsf{Eval}^\dagger$, as is done in hybrid 3. At this point, the state is the same in the two hybrids. Finally, in hybrid 2, we apply the swap operation again so the final state in hybrid 2 could be transformed into the final state in hybrid 3 by another swap operation. Therefore the invariant is satisfied at the end of step 1.

Third, let us assume the invariant is satisfied at the start of some invocation of step 2 or 3. We can show that the invariant is still satisfied at the end of this step using similar reasoning to our argument for step 1.

In conclusion, before or after any step of any query, the current state of the system in hybrid 2 can be mapped to the current state of the system in hybrid 3 by applying a swap operation to $\mathcal{P}$. The swap operation is applied to an internal register of the oracle, which is not part of the user's view. If a computationally unbounded user makes an unbounded number of queries to either the hybrid 2 or hybrid 3 oracle and outputs a final state on their registers, this state will be the same once we trace out $\mathcal{P}$. Therefore, hybrids 2 and 3 are perfectly indistinguishable.

In total, we have shown that $O^{\mathsf{SEQ}}_{U'_\Phi}$ and $\mathsf{Sim}'(P)$ are perfectly indistinguishable after any number of queries. □

It just remains to show that $O^{\mathsf{SEQ}}_{U_\Phi}$ and $O^{\mathsf{SEQ}}_{U'_\Phi}$ are perfectly indistinguishable. This is implied by the following lemma.

Lemma 11 (Self-adjoint implementations hide dilation differences)**.** *Let U, U' be two Stinespring dilations of Φ with the same ancilla register $\mathcal{P}$ initialized to $|0^m\rangle_{\mathcal{P}}$. Then the oracles O^{SEQ}_U and $O^{\mathsf{SEQ}}_{U'}$ (definition 14) are perfectly indistinguishable after any number of queries.*

Proof. Let $P_0 := I_{\mathcal{Q}} \otimes |0^m\rangle \langle 0^m|_{\mathcal{P}}$ and $P_\perp := I - P_0$. Writing O^{SEQ}_U in 2×2 block form on $\mathcal{C}$ using the identities

$$\mathrm{C}_{\mathcal{C}}U = \begin{pmatrix} I & 0 \\ 0 & U \end{pmatrix}, \quad \mathrm{C}_{\mathcal{C}}U^\dagger = \begin{pmatrix} I & 0 \\ 0 & U^\dagger \end{pmatrix}, \quad \mathrm{C}_{\mathcal{P}=|0^m\rangle}\mathrm{NOT}_{\mathcal{C}} = \begin{pmatrix} P_\perp & P_0 \\ P_0 & P_\perp \end{pmatrix},$$

where we use the shorthand $\mathrm{C}_{\mathcal{P}=|0^m\rangle}\mathrm{NOT}_{\mathcal{C}}$ to denote the controlled-NOT on target $\mathcal{C}$ with control projector P_0 on $\mathcal{P}$, namely $\mathrm{C}_{\mathcal{P}=|0^m\rangle}\mathrm{NOT}_{\mathcal{C}} := \mathsf{X}_{\mathcal{C}} \otimes P_0 + I_{\mathcal{C}} \otimes P_\perp$. Writing $\mathsf{X}_{\mathcal{C}} = |0\rangle\langle 0|\,1_{\mathcal{C}} + |1\rangle\langle 1|\,0_{\mathcal{C}}$ and $I_{\mathcal{C}} = |0\rangle\langle 0|\,0_{\mathcal{C}} + |1\rangle\langle 1|\,1_{\mathcal{C}}$, the resulting 2×2 block form on $\mathcal{C}$ is precisely $\left(\begin{smallmatrix} P_\perp & P_0 \\ P_0 & P_\perp \end{smallmatrix}\right)$. We obtain

$$O_U^{\mathsf{SEQ}} = \begin{pmatrix} P_\perp & P_0 U^\dagger \\ U P_0 & U P_\perp U^\dagger \end{pmatrix}_{\mathcal{C};(\mathcal{P},\mathcal{Q})}. \tag{1}$$

By Stinespring uniqueness (for equal-dimension ancillae), there exists a unitary $V_{\mathcal{P}}$ on $\mathcal{P}$ such that $U' = (I_{\mathcal{Q}} \otimes V_{\mathcal{P}}) \cdot U$. Let $W := I_{\mathcal{Q}} \otimes V_{\mathcal{P}}$ and define a unitary on $(\mathcal{C}, \mathcal{P}, \mathcal{Q})$ that is controlled by $\mathcal{C}$:

$$S := |0\rangle\langle 0|_{\mathcal{C}} \otimes I_{\mathcal{P}\mathcal{Q}} + |1\rangle\langle 1|_{\mathcal{C}} \otimes W\,.$$

Then, using the block form from (1),

$$\begin{aligned} S\, O_U^{\mathsf{SEQ}}\, S^\dagger &= \begin{pmatrix} I & 0 \\ 0 & W \end{pmatrix} \begin{pmatrix} P_\perp & P_0 U^\dagger \\ U P_0 & U P_\perp U^\dagger \end{pmatrix} \begin{pmatrix} I & 0 \\ 0 & W^\dagger \end{pmatrix} \\ &= \begin{pmatrix} P_\perp & P_0 U^\dagger W^\dagger \\ W U P_0 & W U P_\perp U^\dagger W^\dagger \end{pmatrix} \\ &= \begin{pmatrix} P_\perp & P_0 (U')^\dagger \\ U' P_0 & U' P_\perp (U')^\dagger \end{pmatrix} = O_{U'}^{\mathsf{SEQ}}. \end{aligned}$$

Thus $O_{U'}^{\mathsf{SEQ}} = S O_U^{\mathsf{SEQ}} S^\dagger$ with S acting only on the hidden registers $(\mathcal{C}, \mathcal{P})$ and trivially on $(\mathcal{A}, \mathcal{Q})$.

Consider any t-query adversary that interleaves local operations V_i on $(\mathcal{A}, \mathcal{Q})$ with oracle calls. Writing $O_U^{\mathrm{op}} := O_U^{\mathsf{SEQ}} \otimes I_{\mathcal{A}}$ and similarly for $O_{U'}^{\mathsf{SEQ}}$, we have

$$\begin{aligned} \mathcal{U}_{U'} &= V_t O_{U'}^{\mathrm{op}} \cdots V_1 O_{U'}^{\mathrm{op}} = V_t (S O_U^{\mathrm{op}} S^\dagger) \cdots V_1 (S O_U^{\mathrm{op}} S^\dagger) \\ &= S \left(V_t O_U^{\mathrm{op}} \cdots V_1 O_U^{\mathrm{op}} \right) S^\dagger = S\, \mathcal{U}_U\, S^\dagger, \end{aligned}$$

since S acts only on $(\mathcal{C}, \mathcal{P})$ and commutes with each V_i (which acts on $(\mathcal{A}, \mathcal{Q})$). The joint initial state is $|\psi\rangle_{\mathcal{A}} |0\rangle_{\mathcal{C}} |0^m\rangle_{\mathcal{P}} |\phi\rangle_{\mathcal{Q}}$, and S fixes it because S acts as identity on the $\mathcal{C}{=}0$ subspace. Therefore $\mathcal{U}_{U'} |\mathrm{init}\rangle = S\, \mathcal{U}_U |\mathrm{init}\rangle$. Tracing out the inaccessible registers $(\mathcal{C}, \mathcal{P})$ yields identical reduced states on $(\mathcal{A}, \mathcal{Q})$ in the two worlds (partial trace is invariant under local conjugation on the traced-out subsystem). Hence no test on $(\mathcal{A}, \mathcal{Q})$ can distinguish O_U^{SEQ} from $O_{U'}^{\mathsf{SEQ}}$, even across multiple queries. □

Lemmas 9 to 11 imply that oracle access to $\mathsf{Sim}'(P)$ and O_Φ^{SEQ} are perfectly indistinguishable. □

Giving the SEQ-security simulator Sim query access to $\mathsf{Sim}'(P)$, instead of O_Φ^{SEQ}, completes the proof of the theorem.

Relativization to unitary oracles. The construction of $\mathsf{Sim}'(P)$ and the indistinguishability argument are black-box. Consequently, if a fixed family of unitary oracles $\mathcal{O}$ is given to all parties, each hybrid and equality above holds verbatim with all machines making identical relative use of $\mathcal{O}$. The theorem therefore holds relative to $\mathcal{O}$. □

5 Stateful Quantum State Obfuscation: Towards a Plain Model Construction

Now that we have a strong definition for OTPs which makes sense in the plain model, we would like to actually construct it there. For a moment, let us suppose that we had a candidate construction.

How would we show that any functionally equivalent program reveals no more information than the candidate? In the classical setting, we can prove that iO is best-possible by using iO to obfuscate the other, allegedly "better" program. However, in the quantum setting, the other program may use a quantum state - so at a minimum we need quantum state iO [BBV24, CG24]. Further than that, the other program *might change its behavior as it is queried* – for example by refusing to answer a second query. So it seems that any security proof would need the ability to obfuscate such programs.

In this section, we define and investigate the feasibility of quantum state obfuscation for programs which may modify their behavior as they are queried.

Definition 18 (Stateful Quantum Programs)**.** *A stateful quantum program is specified by a tuple $(U, |\psi\rangle)$ consisting of a unitary U and an initial state $|\psi\rangle$ contained in register $\mathcal{R}$. To evaluate a stateful quantum program on a state contained in register $\mathcal{Q}$, apply U to register $(\mathcal{Q}, \mathcal{R})$ and output register $\mathcal{Q}$.*

We define oracle access to P as follows. We initialize an internal register $\mathcal{R}$ with the state $|\psi\rangle$. A query can be either a forward or inverse query. On a forward query in register $\mathcal{Q}$, the operation U is performed on registers $(\mathcal{Q}, \mathcal{R})$, and the query register $\mathcal{Q}$ is returned. On an inverse query in register $\mathcal{Q}$, the operation U^{-1} is performed on registers $(\mathcal{Q}, \mathcal{R})$ and $\mathcal{Q}$ is returned. For a quantum oracle algorithm A, we denote oracle access to program P as A^P.

Definition 19 (Functional Equivalence Between Stateful Programs)**.** *Two stateful quantum programs $P_1 = (U_1, |\psi_1\rangle)$ and $P_2 = (U_2, |\psi_2\rangle)$ are (s, ε)-functionally equivalent if for every quantum oracle algorithm A_s making at most s oracle queries, the trace distance between the state of A_s given P_1 and its state when its given P_2 is at most ε, that is,* [9]

$$\frac{1}{2}\left\|A_s^{P_1} - A_s^{P_2}\right\|_1 \leq \varepsilon$$

If this holds for every $s = \mathsf{poly}(\lambda)$, we say they are $(\mathsf{poly}(\lambda), \varepsilon)$-functionally equivalent.

A stateful quantum program essentially implements a quantum channel *with memory*. As the program is queried, the state on register $\mathcal{R}$ may evolve, changing the behavior on future queries. As an example, imagine the "counting program" given by $|\psi\rangle = |0^n\rangle$ and U which increments the contents of $\mathcal{R}$ by 1 modulo 2^n, then classical-copies the result to register $\mathcal{Q}$. If one were to query this program twice on $|0\rangle$, the program would return $|1\rangle$ the first time and $|2\rangle$ the second time.

5.1 Stateful Quantum Indistinguishability Obfuscation

Using the notion of stateful functional equivalence, the definition of stateful iO is quite natural: if two programs have stateful functional equivalence, they can be indistinguishably obfuscated.

Definition 20 (Stateful Quantum Indistinguishability Obfuscation)**.** *A stateful quantum state indistinguishability obfuscator is a QPT compiler Obfs which takes in a stateful quantum program $(U, |\psi\rangle)$, then outputs another stateful quantum program $(\widetilde{U}, |\widetilde{\psi}\rangle)$:*

$$(\widetilde{U}, |\widetilde{\psi}\rangle) \leftarrow \mathsf{Obfs}(1^\lambda, (U, |\psi\rangle))$$

It must satisfy the following two properties.

- ***Correctness:** There exists some negligible function μ such that for every stateful program $(U, |\psi\rangle)$, the obfuscation $(\widetilde{U}, |\widetilde{\psi}\rangle) \leftarrow \mathsf{Obfs}(1^\lambda, (U, |\psi\rangle))$ is $(\mathsf{poly}(\lambda), \mu(\lambda))$-functionally equivalent to $(U, |\psi\rangle)$ with probability $1 - \mathsf{negl}(\lambda)$.*

[9] Note that the order of quantifiers permits A_s to depend on the programs. For example, A_s could evaluate P_1 on a description of U_1.

- ***Security:*** *For every pair of stateful programs* $(U_1, |\psi_1\rangle)$ *and* $(U_2, |\psi_2\rangle)$ *which are* $(\mathsf{poly}(\lambda), \mathsf{negl}(\lambda))$*-functionally equivalent and satisfy* $|(U_1, |\psi_1\rangle)| = |(U_2, |\psi_2\rangle)|$*,*

$$\{\mathsf{Obfs}(1^\lambda, (U_1, |\psi_1\rangle))\} \approx \{\mathsf{Obfs}(1^\lambda, (U_2, |\psi_2\rangle))\}$$

It is useful to compare this to the classical notion of indistinguishability obfuscation. In the classical setting, two programs P_1 and P_2 are considered to be functionally equivalent if $P_1(x) = P_2(x)$ for *every* x. In the quantum setting, the functional equivalence requirement is usually relaxed to allow negligible error in implementing the program [BK21, HT25], because it is generally quite difficult to exactly implement a quantum program.

This behavior can be seen as requiring that once P_1 and P_2 are fixed, *no* possible query x exists that would allow distinguishing black-box access to P_1 from access to P_2. We emphasize that in this view, x is still allowed to depend on the code of the two program even though the adversary otherwise can only make black-box queries to the selected program. As a sanity check, allowing the adversary to depend on the code ensures that this view still avoids impossibility results like [BGI+01b, ABDS21] which are based on "feeding the program to itself".

In the stateful setting, the definition needs to account for the ability of the program to change over the course of several queries. We allow this by considering an adversary which may submit a polynomial number of queries in sequence. If the program did not update its state, then this would be equivalent to quantifying over all single queries, since sequential queries would have the same effect as single queries (up to negligible error due to allowing imperfect implementations). On the other hand, it might be the case that two stateful programs have identical outputs for, say, 10 queries, before suddenly changing behavior on the 11'th. Since even an adversary with black-box access could perform 11 queries and notice the difference on the last query, it does not make sense to claim that the two programs can be indistinguishably obfuscated.

The restriction to a polynomial number of queries is also quite natural. Although any polynomial-time adversary could certainly reach a polynomially-late program state by simply performing that many queries, to reach a superpolynomially-late program state they would need to interact with the program in a non-black-box manner. Thus, we argue that the ability to skip to a superpolynomially-late program state should be considered as a major break in the security of the scheme. At that point, we should simply consider obfuscating a state*less* program which allows anyone to skip to any point in the future.

Nonetheless, we do note that strengthening the functional equivalence requirement to a super-polynomial number of queries also makes sense as a (weaker) definition, if obfuscation for polynomial functional equivalence turns out to be impossible. We provide evidence in Section 5.3 that obfuscation is possible for programs which are only polynomially-equivalent by constructing such a scheme in an idealized model.

5.2 Stateful Quantum iO Implies Best-Possible Testable One-Time Programs

We show that such stateful quantum iO is actually sufficient to achieve best-possible testable quantum one-time programs. Thus, if one were to construct stateful quantum indistinguishability obfuscation in the plain model, they would also obtain good one-time programs.

Theorem 10. *Assuming stateful quantum indistinguishability obfuscation, there exists a best-possible testable quantum one-time program compiler for quantum channels (Definition 16).*

Proof. Let $\Phi : L(\mathcal{H}_\mathcal{Q}) \to L(\mathcal{H}_\mathcal{Q})$ be a quantum channel and let $P = (|0^m\rangle, \mathsf{Eval})$ be a canonical Stinespring dilation of Φ. Without loss of generality, we can equip P with a reflection oracle by reflecting around $|0^m\rangle$. The construction is $\mathsf{sqiO}(\mathsf{SEQ}(P))$, where $\mathsf{SEQ}(P)$ is a program implementing the SEQ oracle on P (see Section 4).

To show best-possible security among testable programs, we must show a simulator that is indistinguishable from $\mathsf{sqiO}(\mathsf{SEQ}(P))$. Given a program P' that implements Φ, the simulator is $\mathsf{sqiO}(\mathsf{SEQ}(P'))$. By Lemma 11, $\mathsf{SEQ}(P)$ and $\mathsf{SEQ}(P')$ are (s,ε)-functionally equivalent for all $s \in \mathbb{N}$ and $\varepsilon = 0$. Therefore stateful iO security implies that $\mathsf{sqiO}(\mathsf{SEQ}(P))$ is computationally indistinguishable from $\mathsf{sqiO}(\mathsf{SEQ}(P'))$.

□

5.3 Stateful Oracles in the Classical Oracle Model

Next, we provide evidence that stateful obfuscation is possible by constructing it in the classical oracle model. Unfortunately, we do not yet know how to construct stateful quantum state indistinguishability obfuscation in the plain model. Even giving a plausible candidate would resolve a major open question since it implies weaker notions of obfuscation which have not yet been constructed, such as iO for pseudodeterministic circuits or for unitaries.

Another interpretation of our construction is that *oracle models with memory are just as good as stateless oracle models.* For example, our Single Effective Query (SEQ) security is a query interface that maintains internal state across queries. Although it might seem somewhat odd to allow an oracle to maintain state, our result shows that this behavior can be achieved in the more standard classical oracle model.

Definition 21 (Ideal Stateful Quantum State Obfuscation)**.** *A ideal stateful quantum state obfuscator is a QPT compiler* Obfs *which takes as input a stateful quantum program* $(U, |\psi\rangle)$ *and outputs another stateful quantum program* $(\widetilde{U}, |\widetilde{\psi}\rangle)$*.*

$$(\widetilde{U}, |\widetilde{\psi}\rangle) \leftarrow \mathsf{Obfs}(1^\lambda, (U, |\psi\rangle))$$

In an oracle model, Obfs *may also output a description of an efficient algorithm matching the oracle model (e.g. a unitary for the unitary oracle model) and all parties are given oracle access to that unitary.*

It must satisfy the following two properties.

- ***Correctness:*** *There exists some* $\varepsilon = \mathsf{negl}(\lambda)$ *such that for every stateful program* $(U, |\psi\rangle)$*, the obfuscation* $(\widetilde{U}, |\widetilde{\psi}\rangle) \leftarrow \mathsf{Obfs}(1^\lambda, (U, |\psi\rangle))$ *is* $(\mathsf{poly}(\lambda), \varepsilon)$*-functionally equivalent to* $(U, |\psi\rangle)$ *with probability* $1 - \mathsf{negl}(\lambda)$*.*

- ***Security:*** *There exists a simulator* Sim *such that for every stateful program* $P = (U, |\psi\rangle)$*, every QPT adversary* $\mathcal{A}$*, and every QPT distinguisher* D*,*

$$\left|\Pr[1 \leftarrow D(\mathcal{A}(\mathsf{Obfs}(1^\lambda, P)] - \Pr[D(\mathsf{Sim}^P(1^\lambda, \mathcal{A}, n(\lambda), m(\lambda))]\right| = \mathsf{negl}(\lambda)$$

where U *has at most* $n(\lambda)$ *gates,* $|\psi\rangle$ *has at most* $n(\lambda)$ *qubits, and* P *takes as input an* $m(\lambda)$*-sized register.*

A straightforward hybrid argument shows that ideal stateful obfuscator is also a stateful indistinguishability obfuscator. First, ideal obfuscation allows replacing an obfuscation of P_1 by a simulator who only has oracle access to P_1. Then, since the simulator makes only a polynomial number of queries, it cannot distinguish between oracle access to P_1 or P_2. Finally, ideal obfuscation allows switching back from the simulator with oracle access to P_2 to an obfuscation of P_2.

Claim 3. *Any ideal stateful quantum state obfuscator is also a stateful quantum state indistinguishability obfuscator.*

We now show how to construct ideal stateful obfuscation in the unitary oracle model. As a corollary of [HT25], it can also be constructed in the classical oracle model.

Construction 1. *The construction uses a publicly verifiable quantum authentication scheme* QAS *(e.g. the coset authentication scheme from [BBV24]) and an ideal unitary obfuscation* Obfs' *(e.g. Theorem 6 [HT25]).*

Without loss of generality, the QAS scheme supports Pauli key updates since we can always attach a Pauli correction to the key. [BBV24]'s authentication scheme also supports Pauli key updates natively. We implement $\mathsf{QAS.Dec}_k$ *and* $\mathsf{QAS.Enc}_k$ *as reversible circuits that act unitarily on an enlarged work space and return all ancillas to* $|0\rangle$ *(i.e., they are coherently implemented isometries), so the following overall procedure is unitary. Assume that* U *does not make any use of oracles.*

On input $P = (U, |\psi\rangle)$ *and security parameter* 1^λ*:*

1. *Sample an authentication key* $k \leftarrow \mathsf{QAS.KeyGen}(1^\lambda)$.
2. *Compute* $|\widetilde{\psi}\rangle := \mathsf{QAS.Enc}_k(|\psi\rangle)$ *on register* $\mathcal{R}$.
3. *Let* $\widetilde{U}$ *be the following unitary:*
   (a) *Take as input two registers* $(\mathcal{Q},\ \mathcal{R})$
   (b) *Apply* $\mathsf{QAS.Dec}_k$ *to register* $\mathcal{R}$*. If the result is* $\perp$*, uncompute* $\mathsf{QAS.Dec}_k$ *and return the input registers immediately.*
   (c) *Otherwise, apply* U *to* $(\mathcal{Q}, \mathcal{R})$.
   (d) *Apply* $\mathsf{QAS.Enc}_k$ *to register* $\mathcal{R}$.
   (e) *Return* $(\mathcal{Q}, \mathcal{R})$.
4. *Compute the unitary obfuscation* $(|\widetilde{\phi}\rangle, C) \leftarrow \mathsf{Obfs}'(\widetilde{U})$.
5. *Output* $|\widetilde{\psi}\rangle, \mathsf{Obfs}'(\widetilde{U})$.

Theorem 11. *Ideal stateful quantum obfuscation exists in the classical oracle model.*

Proof. Let n be the size of the internal state register $\mathcal{R}$, in qubits. Let Adv be the adversary. Let Sim' be the simulator of the ideal unitary obfuscation scheme Obfs'. Let Adv' be the adversary obtained as $\mathsf{Adv}' = \mathsf{Sim}'(\mathsf{Adv}, \cdot)$.

The simulator $\mathsf{Sim}^P(\mathsf{Adv}')$ initializes itself by sampling $k \leftarrow \mathsf{Auth.KeyGen}(1^\lambda)$. Prepare n EPR pairs in registers $(\mathcal{A}_1, \mathcal{B}_1),\ \ldots,\ (\mathcal{A}_n, \mathcal{B}_n)$ and apply $\mathsf{Auth.Enc}_k$ to register $\mathcal{B} = (\mathcal{B}_1, \ldots, \mathcal{B}_n)$, expanding the register as necessary, to obtain

$$\propto \sum_{x \in \{0,1\}^n} |x\rangle_{\mathcal{A}} \otimes \mathsf{Auth.Enc}_k(|x\rangle)_{\mathcal{B}}$$

It runs $\mathsf{Adv}'^{(\cdot)}(\mathcal{B})$, answering its oracle queries as follows.

1. Parse the query register as $(\mathcal{Q}, \mathcal{B}')$.
2. Run $\mathsf{Auth.Ver}_k(\mathcal{B}')$. If the result is $\perp$, uncompute $\mathsf{Auth.Ver}_k$ and return immediately. Otherwise continue.
3. Query P on $\mathcal{Q}$. Return the result.

We show that $\mathsf{Sim}^P(\mathsf{Adv})$ is indistinguishable from $\mathsf{Adv}^{\widetilde{U}}(|\widetilde{\psi}\rangle)$ for any $q = \mathsf{poly}(\lambda)$ queries via a series of hybrid experiments.

- $\mathsf{Hyb}_{-1} = \mathsf{Adv}(|\widetilde{\psi}\rangle, \mathsf{Obfs}'(\widetilde{U}))$.
- $\mathsf{Hyb}_0 = \mathsf{Sim}'^{\widetilde{U}, \widetilde{U}^\dagger}(\mathsf{Adv}, |\widetilde{\psi}\rangle) = \mathsf{Adv}'^{\widetilde{U}, \widetilde{U}^\dagger}(|\widetilde{\psi}\rangle)$.

- Hyb_1: Prepare the half-authenticated EPR pairs

$$\propto \sum_{x\in\{0,1\}^n} |x\rangle_{\mathcal{A}} \otimes \mathsf{Auth.Enc}_k(|x\rangle)_{\mathcal{B}}$$

as in the simulator. Teleport $|\psi\rangle$ into the authenticated register $\mathcal{B}$ using the EPR halves in register $\mathcal{A}$. Run $\mathsf{Sim}'^{\widetilde{U}_{k'},\widetilde{U}^\dagger_{k'}}(\mathsf{Adv},\mathcal{B}) = \mathsf{Adv}'^{\widetilde{U}_{k'},\widetilde{U}^\dagger_{k'}}(\mathcal{B})$. Here, $\widetilde{U}_{k'}$ denotes that the oracle uses key k'.

- Hyb_2: Instead of performing the teleportation and key update $k \mapsto k'$ immediately after the teleportation, perform it when Sim' submits its first query (before answering the query). Until that point, store the internal state, which is initialized to $|\psi\rangle$, in register $\mathcal{R}$.

- Hyb_{2+i} for $i = 1$ to q: Initialize Sim' as in Hyb_2. For the first i queries, answer the query as in Sim. To do this, perform P internally using register $\mathcal{R}$. Then, upon receiving query $i+1$, teleport the state of register $\mathcal{R}$ into register $\mathcal{B}$ and update the key $k \mapsto k'$. Then continue answering queries as in the real scheme.

- $\mathsf{Sim} = \mathsf{Hyb}_{2+q}$.

$\mathsf{Hyb}_{-1} \approx \mathsf{Hyb}_0$ by the security of the ideal obfuscation of unitary $\widetilde{U}$ (see Definition 9).

$\mathsf{Hyb}_0 = \mathsf{Hyb}_1$ by the correctness of teleportation and the ability to perform Pauli key updates on the authentication scheme. $\mathsf{Hyb}_1 = \mathsf{Hyb}_2$ since the teleportation and key updates are performed on disjoint registers from those hold by Adv. The main step is to show that $\mathsf{Hyb}_{2+i} \approx \mathsf{Hyb}_{2+i+1}$.

Claim 4. *For every $i < q$,*

$$\mathsf{Hyb}_{2+i} \approx \mathsf{Hyb}_{2+i+1}$$

Proof. We further divide the transition into a sequence of hybrid experiments.

- $\mathsf{Hyb}_{2+i,1}$: For the first $i-1$ queries, answer the query as in Sim. The difference from Hyb_{2+1} happens upon receiving query i.

  Upon receiving query i, first compute $\mathsf{QAS.Dec}_k$ on register $\mathcal{B}'$. Then measure whether the result of decoding is $\neq \bot$ and registers $(\mathcal{A},\mathcal{B}')$ contain a tensor of EPR pairs

$$\propto \sum_x |x\rangle_{\mathcal{A}} \otimes |x\rangle_{\mathcal{B}}$$

  If not, abort the experiment and output $\bot$. Otherwise, continue as in Hyb_{2+i}, starting from teleporting the contents of $\mathcal{R}$ into register $\mathcal{B}$ using $\mathcal{A}$.

- $\mathsf{Hyb}_{2+i,2}$: This is the same as $\mathsf{Hyb}_{2+i,1}$ except that after performing the early abort measurement on query i, the order of computation and teleportation is swapped. Specifically, if the experiment does not abort, compute U on registers $(\mathcal{Q},\mathcal{R})$. Then, teleport the contents of $\mathcal{R}$ into $\mathcal{B}$ using $\mathcal{A}$. Re-authenticate $\mathcal{B}$ using k and continue as in $\mathsf{Hyb}_{2+i,1}$.

- $\mathsf{Hyb}_{2+i,3}$: This is the same as $\mathsf{Hyb}_{2+i,2}$, except that the early abort measurement on query i is *not* performed. Now the computation of U on query i occurs immediately after computing $\mathsf{Auth.Dec}_k$ on register $\mathcal{B}$, controlled on the result not being $\bot$.

  Note that the result of $\mathsf{Auth.Dec}_k$ is not used for query i except to check whether the result is $\bot$.

- Hyb_{2+i+1}: The only difference from $\mathsf{Hyb}_{2+i,3}$ is that performing $\mathsf{Auth.Dec}_k$ to $\mathcal{B}$, checking whether the result is $\bot$, and finally applying $\mathsf{Auth.Enc}_k$ to $\mathcal{B}$ is replaced by performing $\mathsf{Auth.Ver}_k$ to register $\mathcal{B}$ and checking whether the result is $\bot$.

$\mathsf{Hyb}_{2+i} \approx \mathsf{Hyb}_{2+i,1}$ follows from the observation that the abort check is a measurement corresponding to the publicly-verifiable security of QAS. Note that public verifiability is necessary because oracle access to Ver_k is used to answer queries prior to i.

$\mathsf{Hyb}_{2+i,1} = \mathsf{Hyb}_{2+i,2}$ because conditioned on not aborting, the state across register $\mathcal{A}$ and the queried register $\mathcal{B}'$ is n EPR pairs, so teleportation is completely correct. Therefore teleporting from register $\mathcal{R}$ into $\mathcal{B}'$ then performing computation on $\mathcal{B}'$ is equivalent to performing the same computation on register $\mathcal{R}$, then teleporting from $\mathcal{R}$ to $\mathcal{B}'$.

$\mathsf{Hyb}_{2+i,2} \approx \mathsf{Hyb}_{2+i,3}$ because the abort check is a gentle measurement by the publicly verifiable security of QAS.

Finally, $\mathsf{Hyb}_{2+i,3} = \mathsf{Hyb}_{2+i+1}$ because Ver_k outputs $\bot$ precisely when Dec_k would output $\bot$, and otherwise $\mathsf{Hyb}_{2+i,3}$ does not use the result of Dec_k. □

□

By using Theorem 11 and obfuscating the SEQ oracle, which is a stateful quantum program, we achieve one-time programs with SEQ security for all quantum channels in the classical oracle model.

Corollary 2. *There exist one-time programs with SEQ security for all quantum channels in the classical oracle model.*

Acknowledgements

During the preparation of this work, the authors used LLMs to generate initial draft text for certain sections. The authors reviewed, revised, and take full responsibility for all content, ensuring its correctness and integrity. AG was supported in part by DARPA under Agreement No. HR00112020023, NSF CNS-2154149 and a Simons Investigator Award. This work was done in part while AG was at the Simons Institute and participating in the Challenge Institute for Quantum Computation at UC Berkeley.

References

[ABDS21] Gorjan Alagic, Zvika Brakerski, Yfke Dulek, and Christian Schaffner. Impossibility of quantum virtual black-box obfuscation of classical circuits. In Tal Malkin and Chris Peikert, editors, *Advances in Cryptology - CRYPTO 2021 - 41st Annual International Cryptology Conference, CRYPTO 2021, Virtual Event, August 16-20, 2021, Proceedings, Part I*, volume 12825 of *Lecture Notes in Computer Science*, pages 497–525. Springer, 2021.

[ADFMP20] Navid Alamati, Luca De Feo, Hart Montgomery, and Sikhar Patranabis. Cryptographic group actions and applications. In Shiho Moriai and Huaxiong Wang, editors, *Advances in Cryptology – ASIACRYPT 2020*, pages 411–439, Cham, 2020. Springer International Publishing.

[AR19a] Scott Aaronson and Guy N. Rothblum. Gentle measurement of quantum states and differential privacy. In *Proceedings of the 51st Annual ACM SIGACT Symposium on Theory of Computing*, STOC 2019, page 322–333, New York, NY, USA, 2019. Association for Computing Machinery.

[AR19b] Scott Aaronson and Guy N. Rothblum. Gentle measurement of quantum states and differential privacy, 2019. Full version used in this paper (arXiv:1904.08747v1, 18 Apr 2019).

[BBV24] James Bartusek, Zvika Brakerski, and Vinod Vaikuntanathan. Quantum state obfuscation from classical oracles. In Bojan Mohar, Igor Shinkar, and Ryan O'Donnell, editors, *Proceedings of the 56th Annual ACM Symposium on Theory of Computing, STOC 2024, Vancouver, BC, Canada, June 24-28, 2024*, pages 1009–1017. ACM, 2024.

[BDS23] Shalev Ben-David and Or Sattath. Quantum tokens for digital signatures. *Quantum*, 7:901, jan 2023.

[BGI+01a] Boaz Barak, Oded Goldreich, Rusell Impagliazzo, Steven Rudich, Amit Sahai, Salil Vadhan, and Ke Yang. On the (im) possibility of obfuscating programs. In *Annual International Cryptology Conference*, pages 1–18. Springer, 2001.

[BGI+01b] Boaz Barak, Oded Goldreich, Russell Impagliazzo, Steven Rudich, Amit Sahai, Salil P. Vadhan, and Ke Yang. On the (im)possibility of obfuscating programs. In Joe Kilian, editor, *Advances in Cryptology - CRYPTO 2001, 21st Annual International Cryptology Conference, Santa Barbara, California, USA, August 19-23, 2001, Proceedings*, volume 2139 of *Lecture Notes in Computer Science*, pages 1–18. Springer, 2001.

[BGI+12] Boaz Barak, Oded Goldreich, Russell Impagliazzo, Steven Rudich, Amit Sahai, Salil Vadhan, and Ke Yang. On the (im) possibility of obfuscating programs. *Journal of the ACM (JACM)*, 59(2):6, 2012.

[BGMZ18] James Bartusek, Jiaxin Guan, Fermi Ma, and Mark Zhandry. Preventing zeroizing attacks on ggh15. In *Proceedings of TCC 2018*, 2018.

[BGS13a] Anne Broadbent, Gus Gutoski, and Douglas Stebila. Quantum one-time programs. In *Advances in Cryptology – CRYPTO 2013*, pages 344–360. Springer Berlin Heidelberg, 2013.

[BGS13b] Anne Broadbent, Gus Gutoski, and Douglas Stebila. Quantum one-time programs. In *Annual Cryptology Conference*, pages 344–360. Springer, 2013.

[BJ15] Anne Broadbent and Stacey Jeffery. Quantum homomorphic encryption for circuits of low t-gate complexity. In *Annual Cryptology Conference*, pages 609–629. Springer, 2015.

[BK21] Anne Broadbent and Raza Ali Kazmi. Constructions for quantum indistinguishability obfuscation. In Patrick Longa and Carla Ràfols, editors, *Progress in Cryptology - LATINCRYPT 2021 - 7th International Conference on Cryptology and Information Security in Latin America, Bogotá, Colombia, October 6-8, 2021, Proceedings*, volume 12912 of *Lecture Notes in Computer Science*, pages 24–43. Springer, 2021.

[BKNY23] James Bartusek, Fuyuki Kitagawa, Ryo Nishimaki, and Takashi Yamakawa. Obfuscation of pseudo-deterministic quantum circuits. In *Proceedings of the 55th Annual ACM Symposium on Theory of Computing*, pages 1567–1578, 2023.

[CD08] Ran Canetti and Ronny Ramzi Dakdouk. Obfuscating point functions with multibit output. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 489–508. Springer, 2008.

[CG24] Andrea Coladangelo and Sam Gunn. How to use quantum indistinguishability obfuscation. In Bojan Mohar, Igor Shinkar, and Ryan O'Donnell, editors, *Proceedings of the 56th Annual ACM Symposium on Theory of Computing, STOC 2024, Vancouver, BC, Canada, June 24-28, 2024*, pages 1003–1008. ACM, 2024.

[DS05] Yevgeniy Dodis and Adam Smith. Correcting errors without leaking partial information. In *Proceedings of the Thirty-Seventh Annual ACM Symposium on Theory of Computing*, STOC '05, page 654–663, New York, NY, USA, 2005. Association for Computing Machinery.

[GGH+16] Sanjam Garg, Craig Gentry, Shai Halevi, Mariana Raykova, Amit Sahai, and Brent Waters. Candidate indistinguishability obfuscation and functional encryption for all circuits. *SIAM Journal on Computing*, 45(3):882–929, 2016.

[GKR08] Shafi Goldwasser, Yael Tauman Kalai, and Guy N Rothblum. One-time programs. In *Advances in Cryptology–CRYPTO 2008: 28th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 17-21, 2008. Proceedings 28*, pages 39–56. Springer, 2008.

[GKW17] Rishab Goyal, Venkata Koppula, and Brent Waters. Lockable obfuscation. In *2017 IEEE 58th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 612–621. IEEE, 2017.

[GLR+25] Aparna Gupte, Jiahui Liu, Justin Raizes, Bhaskar Roberts, and Vinod Vaikuntanathan. Quantum one-time programs, revisited. In *Proceedings of the 57th Annual ACM Symposium on Theory of Computing*, pages 213–221, 2025.

[GM24] Sam Gunn and Ramis Movassagh. Quantum one-time protection of any randomized algorithm. *private communication*, 2024.

[GR07] Shafi Goldwasser and Guy N Rothblum. On best-possible obfuscation. In *Theory of Cryptography Conference*, pages 194–213. Springer, 2007.

[HJL25] Yao-Ching Hsieh, Aayush Jain, and Huijia Lin. Lattice-based post-quantum io from circular security with random opening assumption. In *Advances in Cryptology – CRYPTO 2025: 45th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 17–21, 2025, Proceedings, Part VII*, page 3–38, Berlin, Heidelberg, 2025. Springer-Verlag.

[HLOV11] Brett Hemenway, Benoît Libert, Rafail Ostrovsky, and Damien Vergnaud. Lossy encryption: Constructions from general assumptions and efficient selective opening chosen ciphertext security. In *ASIACRYPT*, volume 7073 of *Lecture Notes in Computer Science*, pages 70–88. Springer, 2011.

[HT25] Mi-Ying Huang and Er-Cheng Tang. Obfuscation of unitary quantum programs. *CoRR*, abs/2507.11970, 2025.

[JLS20] Aayush Jain, Huijia Lin, and Amit Sahai. Indistinguishability obfuscation from well-founded assumptions. Cryptology ePrint Archive, Report 2020/1003, 2020. https://eprint.iacr.org/2020/1003.

[LPS04] Benjamin Lynn, Manoj Prabhakaran, and Amit Sahai. Positive results and techniques for obfuscation. In *International conference on the theory and applications of cryptographic techniques*, pages 20–39. Springer, 2004.

[MW05] Chris Marriott and John Watrous. Quantum arthur–merlin games. *Computational Complexity*, 14(2):122–152, 2005.

[Pei15] Chris Peikert. A decade of lattice cryptography. Cryptology ePrint Archive, Paper 2015/939, 2015.

[PW07] Chris Peikert and Brent Waters. Lossy trapdoor functions and their applications. Cryptology ePrint Archive, Paper 2007/279, 2007.

[Reg05] Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. In *Proceedings of the Thirty-Seventh Annual ACM Symposium on Theory of Computing*, STOC '05, page 84–93, New York, NY, USA, 2005. Association for Computing Machinery.

[WZ17] Daniel Wichs and Giorgos Zirdelis. Obfuscating compute-and-compare programs under lwe. In *2017 IEEE 58th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 600–611. IEEE, 2017.

[Zha25] Mark Zhandry. How to model unitary oracles. In Yael Tauman Kalai and Seny F. Kamara, editors, *Advances in Cryptology – CRYPTO 2025*, pages 237–268, Cham, 2025. Springer Nature Switzerland.

A Lossy Public-Key Encryption

Here we define and construct lossy public-key encryption (lossy PKE), which will be used in our impossibility result. In injective mode, lossy PKE functions like normal PKE, in which Enc hides the message computationally, but not statistically. In lossy mode, Enc actually hides the message statistically or perfectly, so the output distributions of $\mathsf{Enc}(0)$ and $\mathsf{Enc}(1)$ are (respectively) statistically close or identical. Furthermore, the injective and lossy modes are indistinguishable to an adversary who is given the public encryption key.

The following definition is based on [HLOV11], definition 2, but with some differences. For instance, [HLOV11]'s definition required perfect correctness whereas we allow a negligible but non-zero decryption error.

Definition 22 (Statistically/Perfectly Lossy Public-Key Encryption)**.** *A* ***statistically/perfectly lossy public-key encryption*** *(lossy PKE) scheme is a tuple of PPT algorithms* $(\mathsf{Gen}, \mathsf{Enc}, \mathsf{Dec})$ *with the following syntax.*

Let $\mathcal{M}$ *be the message space, and let* $\mathcal{R}_{\mathsf{Gen}}, \mathcal{R}_{\mathsf{Enc}}, \mathcal{R}_{\mathsf{Dec}}$ *be the sample space of the randomness for* $\mathsf{Gen}, \mathsf{Enc}, \mathsf{Dec}$*, respectively. The sizes of these sets may depend on* λ*. Also let* $r_{\mathsf{Gen}} \xleftarrow{\$} \mathcal{R}_{\mathsf{Gen}}, r_{\mathsf{Enc}} \xleftarrow{\$} \mathcal{R}_{\mathsf{Enc}}, r_{\mathsf{Dec}} \xleftarrow{\$} \mathcal{R}_{\mathsf{Dec}}$*. Next,*

- $\mathsf{Gen}(1^\lambda, \mathsf{mode}; r_{\mathsf{Gen}})$*: Takes a security parameter* $\lambda \in \mathbb{N}$ *and a* $\mathsf{mode} \in \{\mathsf{inj}, \mathsf{lossy}\}$ *and outputs keys* $(\mathsf{pk}, \mathsf{sk})$*. Additionally, let* $\mathsf{GenPK}(1^\lambda, \mathsf{mode}; r_{\mathsf{Gen}})$ *compute* $\mathsf{Gen}(1^\lambda, \mathsf{mode}; r_{\mathsf{Gen}})$ *but only output* pk*.*
- $\mathsf{Enc}(\mathsf{pk}, m; r_{\mathsf{Enc}})$*: Takes a public key* pk *and a message* $m \in \mathcal{M}$ *and outputs a ciphertext* c*.*
- $\mathsf{Dec}(\mathsf{sk}, c; r_{\mathsf{Dec}})$*: Takes a secret key* sk *and a ciphertext* c*, decrypts* c*, and outputs the message* m*.*

We will often omit the random inputs in our notation, for instance, writing $\mathsf{Enc}(\mathsf{pk}, m)$ *instead of* $\mathsf{Enc}(\mathsf{pk}, m; r_{\mathsf{Enc}})$*.*

Next, the encryption scheme satisfies the following properties:

- ***Correctness:*** *With overwhelming probability over the sampling of* $(\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{Gen}(1^\lambda, \mathsf{inj})$*, the following is true for every* $m \in \mathcal{M}$*:*

$$\Pr_{r_{\mathsf{Enc}}, r_{\mathsf{Dec}}} [m = \mathsf{Dec}[\mathsf{sk}, \mathsf{Enc}(\mathsf{pk}, m)]] = 1$$

- ***Statistical/Perfect Lossiness:*** *There is a function $\mu(\lambda)$ that is (respectively) negligible/identically zero such that with overwhelming probability over the sampling of $(\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{Gen}(1^\lambda, \mathsf{lossy})$, the following is true for every $m_0, m_1 \in \mathcal{M}$: the statistical distance between $\mathsf{Enc}(\mathsf{pk}, m_0)$ and $\mathsf{Enc}(\mathsf{pk}, m_1)$ is $\leq \mu(\lambda)$.*

- ***Indistinguishability of Modes:*** *The output distributions of $\mathsf{GenPK}(1^\lambda, \mathsf{inj})$ and $\mathsf{GenPK}(1^\lambda, \mathsf{lossy})$ are indistinguishable to any QPT adversary. Formally, we require that for any QPT adversary $\mathcal{A}$, there is a negligible function $\nu(\lambda)$ such that:*

$$\left|\Pr\left[\mathcal{A}(1^\lambda, \mathsf{GenPK}(1^\lambda, \mathsf{inj})) \to 1\right] - \Pr\left[\mathcal{A}(1^\lambda, \mathsf{GenPK}(1^\lambda, \mathsf{lossy})) \to 1\right]\right| \leq \nu(\lambda)$$

A.1 Construction from Lossy Trapdoor Functions

We will construct statistically lossy PKE from lossy trapdoor functions (LTFs).[10]

Lossy Trapdoor Functions: We will start with the primitive of lossy trapdoor functions, which can be constructed from LWE.

Definition 23 (Almost Always (n, k)-Lossy Trapdoor Functions ([PW07] Section 3.1))**.** *Let $n(\lambda) = \mathsf{poly}(\lambda)$ represent the input length, and let $k(\lambda) \leq n(\lambda)$ represent the lossiness.*

*An **almost always (n, k)-lossy trapdoor function** scheme is a tuple of PPT algorithms $(\mathsf{Gen}, F, F^{-1})$ with the following syntax:*

- *$\mathsf{Gen}(1^\lambda, \mathsf{mode})$: Takes a security parameter $\lambda \in \mathbb{N}$ and a mode $\mathsf{mode} \in \{\mathsf{inj}, \mathsf{lossy}\}$ and outputs keys $(\mathsf{pk}, \mathsf{sk})$. Additionally, let $\mathsf{GenPK}(1^\lambda, \mathsf{mode})$ compute $\mathsf{Gen}(1^\lambda, \mathsf{mode})$ but only output pk.*

- *$F(\mathsf{pk}, x)$: Takes a public key pk and an input $x \in \{0,1\}^n$ and computes a string y that is a deterministic function of (pk, x).*

- *$F^{-1}(\mathsf{sk}, y)$: Given an injective-mode key sk and an image value y, it outputs $x \in \{0,1\}^n$.*

The scheme satisfies the following properties:

- ***Injectivity:*** *With probability overwhelming in λ, $\mathsf{Gen}(1^\lambda, \mathsf{inj})$ outputs a $(\mathsf{pk}, \mathsf{sk})$ such that:*

  1. *$F(\mathsf{pk}, \cdot)$ is injective, and*
  2. *for any $x \in \{0,1\}^n$, $F^{-1}[\mathsf{sk}, F(\mathsf{pk}, x)] = x$.*

- ***Lossiness:*** *With probability overwhelming in λ, $\mathsf{Gen}(1^\lambda, \mathsf{lossy})$ outputs a $(\mathsf{pk}, \mathsf{sk})$ such that $F(\mathsf{pk}, \cdot)$ has an image of size $\leq 2^{n-k}$.*

- ***Indistinguishability of Modes:*** *The output distributions of $\mathsf{GenPK}(1^\lambda, \mathsf{inj})$ and $\mathsf{GenPK}(1^\lambda, \mathsf{lossy})$ are indistinguishable to any QPT adversary. Formally, we require that for any QPT adversary $\mathcal{A}$, there is a negligible function $\nu(\lambda)$ such that:*

$$\left|\Pr\left[\mathcal{A}(1^\lambda, \mathsf{GenPK}(1^\lambda, \mathsf{inj})) \to 1\right] - \Pr\left[\mathcal{A}(1^\lambda, \mathsf{GenPK}(1^\lambda, \mathsf{lossy})) \to 1\right]\right| \leq \nu(\lambda)$$

Theorem 12 (Adapted from [PW07, Theorem 6.4])**.** *Let $n(\lambda) = \lambda^2$, $q = 4\lambda^{10}$, $\alpha = \frac{1}{32\lambda^{10}}$, $\chi = \overline{\Psi}_\alpha$. Then assuming the post-quantum hardness of $\mathsf{LWE}_{q,\chi}$, there exists a lossiness parameter $k(\lambda) \sim \frac{n(\lambda)}{2}$ and an almost always (n, k)-lossy trapdoor function scheme (Definition 23).*

[10] One implication of our construction is that LWE implies lossy PKE. There is another route to the same claim. [Reg05]'s encryption scheme from LWE can be adapted to support lossy encryption, as [Pei15] noted. Our result is stronger since we only need to assume the existence of LTFs, rather than the hardness of LWE.

Proof. Theorem 12 is obtained by renaming the variables of [PW07] Theorem 6.4 or giving them concrete values. We do this as follows: $d \to \lambda$, $c_1 = 4$, $c_2 = c_3 = 2$, $p = n^{c_1}$, $q = 4pn$, $\alpha = \frac{1}{32pn}$. Then we obtain:

$$\begin{aligned} n &= d^{c_3} = \lambda^2 \\ r &\leq \left(\frac{c_2}{c_1} + o(1)\right) \cdot n = \left(\frac{1}{2} + o(1)\right) \cdot n \\ k &= n - r \geq n - \left(\frac{1}{2} + o(1)\right) \cdot n = \frac{n}{2} - o(n) \sim \frac{n}{2} \end{aligned}$$

Additionally,

$$\begin{aligned} pn &= n^{c_1} \cdot n = n^5 = \left(\lambda^2\right)^5 = \lambda^{10} \\ q &= 4pn = 4\lambda^{10} \\ \alpha &= \frac{1}{32pn} = \frac{1}{32\lambda^{10}} \end{aligned}$$

With these choices of parameters, our theorem matches the parameter regime and lossiness bound of [PW07] Theorem 6.4. The theorem in [PW07] is stated for classical PPT adversaries; here we use the stronger assumption that $\mathsf{LWE}_{q,\chi}$ is hard for QPT adversaries. □

Pairwise-Independent Permutations: Here we will construct a family of pairwise-independent permutations.

Let the domain and range be a field $\mathbb{F}$ of size $2^{(\lambda^2)}$.

- $\pi_{a,b}(x)$: The function π is defined by any values $a \in \mathbb{F}\backslash\{0\}$ and $b \in \mathbb{F}$. It takes an input $x \in \mathbb{F}$, then computes and outputs
$$y = a \cdot x + b$$
- $\pi^{-1}_{a,b}(y)$: The inverse function π^{-1} is parametrized by the same values (a, b). It takes an input $y \in \mathbb{F}$, then computes and outputs
$$x' = \frac{y - b}{a}$$

Lemma 12 says that π is indeed a permutation. Then Lemma 13 says that the function family defined above is pairwise-independent.

Lemma 12 (π is a permutation)**.** *For any $(a, b) \in \mathbb{F}\backslash\{0\} \times \mathbb{F}$, and any $x \in \mathbb{F}$, $\pi^{-1}_{a,b} \circ \pi_{a,b}(x) = x$.*

Proof.

$$\begin{aligned} \pi^{-1}_{a,b} \circ \pi_{a,b}(x) &= \frac{(a \cdot x + b) - b}{a} = \frac{a \cdot x}{a} \\ &= x \end{aligned}$$

We used the fact that $a \neq 0$. □

Lemma 13 (Pairwise Independence)**.** *For any values $x, x', y, y' \in \mathbb{F}$ such that $x \neq x'$ and $y \neq y'$,*

$$\Pr_{(a,b) \xleftarrow{\$} \mathbb{F}\backslash\{0\} \times \mathbb{F}} \left[\pi_{a,b}(x) = y \wedge \pi_{a,b}(x') = y'\right] = \frac{1}{|\mathbb{F}| \cdot (|\mathbb{F}| - 1)}$$

Proof. The event $\pi_{a,b}(x) = y \wedge \pi_{a,b}(x') = y'$ is equivalent to each of the following lines:

$$\begin{aligned} y = a \cdot x + b \quad &\wedge \quad y' = a \cdot x' + b \\ a = \tfrac{y-y'}{x-x'} \quad &\wedge \quad b = y - \tfrac{y-y'}{x-x'} \cdot x \end{aligned}$$

Since $x \neq x'$ and $y \neq y'$, the fraction $\frac{y-y'}{x-x'} \in \mathbb{F}\backslash\{0\}$. Furthermore, a and b are sampled independently. Therefore,

$$\begin{aligned} \Pr_{(a,b) \xleftarrow{\$} \mathbb{F}\backslash\{0\} \times \mathbb{F}} \left[\pi_{a,b}(x) = y \wedge \pi_{a,b}(x') = y'\right] &= \Pr_{(a,b) \xleftarrow{\$} \mathbb{F}\backslash\{0\} \times \mathbb{F}} \left[a = \frac{y-y'}{x-x'} \wedge b = y - \frac{y-y'}{x-x'} \cdot x\right] \\ &= \Pr_{a \xleftarrow{\$} \mathbb{F}\backslash\{0\}} \left[a = \frac{y-y'}{x-x'}\right] \cdot \Pr_{b \xleftarrow{\$} \mathbb{F}} \left[b = y - \frac{y-y'}{x-x'} \cdot x\right] \\ &= \frac{1}{|\mathbb{F}| \cdot (|\mathbb{F}| - 1)} \end{aligned}$$

□

Construction of Lossy PKE: Here we will construct lossy PKE from the almost-always lossy trapdoor function scheme $\mathsf{LTF}.(\mathsf{Gen}, F, F^{-1})$ and the pairwise independent permutation family defined above.

We will treat bitstrings as field elements and vice versa. Values in $\{0,1\}^{\lambda^2}$ will be treated as elements in a field $\mathbb{F}$ of size $2^{(\lambda^2)}$, and elements of $\mathbb{F}$ will be treated as values in $\{0,1\}^{\lambda^2}$.

- Let $\mathcal{M} = \{0,1\}^{\lambda}$, $n = \lambda^2$, and $k \sim \frac{\lambda^2}{2}$. Let LTF be the almost always (n,k)-lossy trapdoor function scheme given by Theorem 12.
- $\mathsf{Gen}(1^{\lambda}, \mathsf{mode})$: Same as $\mathsf{LTF}.\mathsf{Gen}(1^{\lambda}, \mathsf{mode})$
- $\mathsf{Enc}(\mathsf{pk}, m)$:
  1. Sample $r \xleftarrow{\$} \{0,1\}^{\lambda^2 - \lambda}$ and $(a,b) \xleftarrow{\$} \mathbb{F}\backslash\{0\} \times \mathbb{F}$ independently.
  2. Compute
  $$\begin{aligned} x &= m||r \in \mathbb{F} \\ y &= \mathsf{LTF}.F\left[\mathsf{pk}, \pi_{a,b}(x)\right] \\ c &= (a,b,y) \end{aligned}$$
  and output c.
- $\mathsf{Dec}(\mathsf{sk}, c)$:
  1. Parse c as (a,b,y).
  2. Compute $x' = \pi_{a,b}^{-1}\left[\mathsf{LTF}.F^{-1}(\mathsf{sk}, y)\right]$.
  3. Parse x' as $x' = m'||r' \in \{0,1\}^{\lambda} \times \{0,1\}^{\lambda^2 - \lambda}$.
  4. Output m'.

Theorem 13. *Assuming the post-quantum hardness of LWE for the parameters given in Theorem 12, the construction above is a statistically lossy PKE scheme (Definition 22) with message space $\mathcal{M} = \{0,1\}^{\lambda}$.*

Proof. First, by Theorem 12, the scheme LTF used in our construction is an almost always (n, k)-lossy trapdoor function scheme for $n = \lambda^2$ and $k \sim \frac{\lambda^2}{2}$.

Next, it suffices to show that our lossy PKE construction satisfies the three properties of interest: correctness, statistical lossiness, and indistinguishability of modes.

Lemma 14. *The lossy PKE construction satisfies correctness.*

Proof. The injectivity property of LTF (Definition 23) guarantees that with overwhelming probability, $\mathsf{Gen}(1^\lambda, \mathsf{inj})$ outputs keys $(\mathsf{pk}, \mathsf{sk})$ such that for all $x \in \{0,1\}^n$,

$$\mathsf{LTF}.F^{-1}\left[\mathsf{sk}, \mathsf{LTF}.F(\mathsf{pk}, x)\right] = x$$

We will prove that in this case,

$$\mathsf{Dec}\left[\mathsf{sk}, \mathsf{Enc}(\mathsf{pk}, m)\right] = m$$

Let us compute $m' = \mathsf{Dec}\left[\mathsf{sk}, \mathsf{Enc}(\mathsf{pk}, m)\right]$. In so doing, we compute the intermediate values y and x' as follows.

$$\begin{aligned} y &= \mathsf{LTF}.F[\mathsf{pk}, \pi_{a,b}(m||r)] \\ x' &= \pi_{a,b}^{-1}\left[\mathsf{LTF}.F^{-1}(\mathsf{sk}, y)\right] \\ &= \pi_{a,b}^{-1}\left[\pi_{a,b}(m||r)\right] \\ &= m||r \end{aligned}$$

We used the fact that $\mathsf{LTF}.F^{-1}(\mathsf{sk}, \cdot)$ inverts $\mathsf{LTF}.F(\mathsf{pk}, \cdot)$ and $\pi_{a,b}^{-1}$ inverts $\pi_{a,b}$ (Lemma 12). Finally, since $x' = m||r$, $\mathsf{Dec}(\mathsf{sk}, c)$ outputs $m' = m$.

In summary, we've shown that with probability $\geq 1 - \mathsf{negl}(\lambda)$, $\mathsf{Gen}(1^\lambda, \mathsf{inj})$ outputs keys $(\mathsf{pk}, \mathsf{sk})$ such that for any message $m \in \{0,1\}^\lambda$, $m' = m$. Therefore, the lossy PKE construction satisfies correctness. □

Lemma 15. *The lossy PKE construction satisfies statistical lossiness.*

Proof. First, the lossiness property of LTF (Definition 23) guarantees that with overwhelming probability, $\mathsf{Gen}(1^\lambda, \mathsf{lossy})$ outputs keys $(\mathsf{pk}, \mathsf{sk})$ such that $\mathsf{LTF}.F(\mathsf{pk}, \cdot)$ has an image of size $\leq 2^{n-k}$. We will prove that in this case, for any $m_0, m_1 \in \mathcal{M}$, the distributions of $\mathsf{Enc}(\mathsf{pk}, m_0)$ and $\mathsf{Enc}(\mathsf{pk}, m_1)$, over the randomness of Enc, are statistically close.

Second, since the image of $\mathsf{LTF}.F(\mathsf{pk}, \cdot)$ has size $\leq 2^{n-k}$, then there exists a (possibly inefficient) compression function C that maps each value in the image of $\mathsf{LTF}.F(\mathsf{pk}, \cdot)$ to a unique bitstring $\in \{0,1\}^{n-k}$. C is invertible.

Third, let us introduce the crooked leftover hash lemma (Lemma 16) to do the heavy lifting.

Lemma 16 (Crooked Leftover Hash Lemma ([DS05] Lemma 12))**.** *Let $f : \{0,1\}^N \to \{0,1\}^\ell$ be an arbitrary function, and let $\{h_i\}_{i \in \mathcal{I}}$ be a pairwise independent hash family, where each h_i maps $\{0,1\}^n \to \{0,1\}^N$.*

Let X be a random variable with sample space $\{0,1\}^n$ and with min-entropy $\geq \ell + 2\log\left(\frac{1}{\varepsilon}\right) + 1$. Let I be a random variable sampled uniformly at random from $\mathcal{I}$, the keyspace of the hash family. Let U_N be a random variable sampled uniformly at random from $\{0,1\}^N$. X, I, U_N are independent.

Then the distributions of $[I, f \circ h_I(X)]$ and $[I, f(U_N)]$ are ε-close in statistical distance.

Fourth, let's give concrete values to the variables in Lemma 16. Let $N = n = \lambda^2$. Let $\ell = n - k$. Let $\varepsilon = 2^{-\frac{\lambda^2}{8}}$. Let $f(x) = C \circ \mathsf{LTF}.F(\mathsf{pk}, x)$. Let $\mathcal{I} = \mathbb{F}\backslash\{0\} \times \mathbb{F}$, and for each $(a, b) \in \mathcal{I}$, let $h_{a,b} = \pi_{a,b}$. Let I be a random variable $(A, B) \xleftarrow{\$} \mathbb{F}\backslash\{0\} \times \mathbb{F}$. For any messages $m_0, m_1 \in \mathcal{M}$, let

$$\begin{aligned} X_0 &= m_0||R \\ X_1 &= m_1||R \end{aligned}$$

for $R \xleftarrow{\$} \{0,1\}^{\lambda^2-\lambda}$.

Fifth, we claim that all the conditions of Lemma 16 are satisfied. Note that $f : \{0,1\}^N \to \{0,1\}^\ell$. Note that $\{h_{a,b}\}_{(a,b)\in\mathcal{I}}$ is a pairwise-independent hash family (Lemma 13), where each $h_{a,b}$ maps $\{0,1\}^n \to \{0,1\}^N$. Also I is sampled uniformly at random from $\mathcal{I}$ and independently of X_0, X_1. Next, X_0 and X_1 have sample space $\{0,1\}^{\lambda^2} = \{0,1\}^n$. The min-entropy of X_0 (and also X_1) is $\lambda^2 - \lambda$. For sufficiently large λ,

$$\begin{aligned}
H_{\min}(X_0) &= \lambda^2 - \lambda \\
&\geq \frac{3\lambda^2}{4} + 1 \\
&= \left(\lambda^2 - \frac{\lambda^2}{2}\right) + 2 \cdot \frac{\lambda^2}{8} + 1 \\
&\geq (n-k) + 2 \cdot \log\left(\frac{1}{2^{-\lambda^2/8}}\right) + 1 \\
&= \ell + 2 \cdot \log\left(\frac{1}{\varepsilon}\right) + 1
\end{aligned}$$

Then X_0 and X_1 satisfy the min-entropy condition of Lemma 16.

Sixth, we can apply Lemma 16 to say that the distributions of

$$[A, B, C \circ \mathsf{LTF}.F(\mathsf{pk}, \pi_{A,B}(m_0||R))] \quad \text{and} \quad [A, B, C \circ \mathsf{LTF}.F(\mathsf{pk}, U_N)]$$

are ε-close. Likewise, the distributions of

$$[A, B, C \circ \mathsf{LTF}.F(\mathsf{pk}, \pi_{A,B}(m_1||R))] \quad \text{and} \quad [A, B, C \circ \mathsf{LTF}.F(\mathsf{pk}, U_N)]$$

are ε-close. Then by the triangle inequality, the distributions of

$$[A, B, C \circ \mathsf{LTF}.F(\mathsf{pk}, \pi_{A,B}(m_0||R))] \quad \text{and} \quad [A, B, C \circ \mathsf{LTF}.F(\mathsf{pk}, \pi_{A,B}(m_1||R))]$$

are (2ε)-close.

Additionally, since C is invertible, the distributions of

$$[A, B, \mathsf{LTF}.F(\mathsf{pk}, \pi_{A,B}(m_0||R))] \quad \text{and} \quad [A, B, \mathsf{LTF}.F(\mathsf{pk}, \pi_{A,B}(m_1||R))]$$

are (2ε)-close.

Seventh, note that the distribution of $[A, B, \mathsf{LTF}.F(\mathsf{pk}, \pi_{A,B}(m_0||R))]$ is simply the distribution of $\mathsf{Enc}(\mathsf{pk}, m_0)$, and the distribution of $[A, B, \mathsf{LTF}.F(\mathsf{pk}, \pi_{A,B}(m_1||R))]$ is the distribution of $\mathsf{Enc}(\mathsf{pk}, m_1)$.

In summary, we've shown that for sufficiently large λ, with overwhelming probability, $\mathsf{Gen}(1^\lambda, \mathsf{lossy})$ outputs keys $(\mathsf{pk}, \mathsf{sk})$ such that for any $m_0, m_1 \in \mathcal{M}$, the distributions of $\mathsf{Enc}(\mathsf{pk}, m_0)$ and $\mathsf{Enc}(\mathsf{pk}, m_1)$ have statistical distance $2 \cdot 2^{-\lambda^2/8} = \mathsf{negl}(\lambda)$. This proves the lossiness property. □

Lemma 17. *The lossy PKE construction satisfies indistinguishability of modes.*

Proof. This immediately follows from the fact that Gen is the same as $\mathsf{LTF}.\mathsf{Gen}$, and $\mathsf{LTF}.\mathsf{Gen}$ satisfies the same indistinguishability of modes property (Definition 23). □

□

A.2 Construction from Group Actions

In this section, we describe a perfectly lossy PKE assuming the weak pseudorandomness of effective group actions. This construction is folklore, but we will describe it here for the sake of completeness.

Notation. For a regular and abelian group action $\star : \mathbb{G} \times \mathbb{X} \to \mathbb{X}$, we use additive notation to denote the group operation in $\mathbb{G}$.

Definition 24 (Effective Group Action)**.** *A regular and abelian group action $(\mathbb{G}, \mathbb{X}, \star)$ is effective if it satisfies the following properties.*

1. *The group $\mathbb{G}$ is finite and there exist efficient p.p.t. algorithms for membership testing (deciding whether a binary string represents a group element), equality testing and sampling uniformly in $\mathbb{G}$, and group operation and computing the inverse of any element.*
2. *The set $\mathbb{X}$ is finite and there exist efficient algorithms for membership testing (to check if a binary string represents a valid set element), and unique representation.*
3. *There exists a distinguished element $x_0 \in \mathbb{X}$ with known representation.*
4. *There exists an efficient algorithm that given any $g \in \mathbb{G}$ and any $x \in \mathbb{X}$, outputs $g \star x$.*

Assumption 1 (Weak Pseudorandomness Assumption of an Effective Group Action [ADFMP20])**.** *Suppose $(\mathbb{G}, \mathbb{X}, \star)$ is an effective group action. The weak pseudorandomness assumption states that there is no p.p.t. adversary that can distinguish tuples of the form $(x_i, g \star x_i)$ from (x_i, y_i) where $g \leftarrow \mathbb{G}$ and each $x_i, y_i \leftarrow \mathbb{X}$ are sampled uniformly at random.*

Theorem 14. *Suppose there exists an effective group action for which Assumption 1 holds. Then, there exists a perfectly lossy public key encryption scheme (Definition 22).*

Construction 2. *Let $(\mathbb{G}, \mathbb{X}, \star)$ be a group action for which Assumption 1 holds. Let $x_0 \in \mathbb{X}$ be a distinguished point that is specified as a public parameter.*

- $\mathsf{Gen}(1^\lambda, \mathsf{mode})$*:*
  - *If* $\mathsf{mode} = \mathsf{lossy}$*, then sample random $g, h \leftarrow \mathbb{G}$ and set* $\mathsf{pk} := (x_0, g \star x_0, h \star x_0, (g+h) \star x_0)$.
  - *If* $\mathsf{mode} = \mathsf{inj}$*, then sample random $g, h, k \leftarrow \mathbb{G}$ conditioned on $k \neq g + h$. Set* $\mathsf{pk} = (x_0, g \star x_0, h \star x_0, k \star x_0)$ *and* $\mathsf{sk} = (g, k - h)$.
- $\mathsf{Enc}(\mathsf{pk}, m)$*:*
  - *Sample a random $u \leftarrow \mathbb{G}$. Parse* $\mathsf{pk} = (x_0, x_1, x_2, x_3)$.
  - *If $m = 0$, output $c = (u \star x_0, u \star x_1)$.*
  - *If $m = 1$, output $c = (u \star x_2, u \star x_3)$.*
- $\mathsf{Dec}(\mathsf{sk}, c)$*:*
  - *Parse injective secret key as* $\mathsf{sk} = (g, g')$ *and ciphertext $c = (y_1, y_2)$.*
  - *If $g \star y_1 = y_2$ then output $m = 0$.*
  - *Else if $g' \star y_1 = y_2$ then output $m = 1$.*
  - *Otherwise, output $\perp$.*

Proof. We will prove that the scheme described in Construction 2 is a perfectly lossy PKE scheme, assuming the weak pseudorandomness assumption of the group action.

Correctness. This encryption scheme satisfies perfect decryption correctness on injective keys. In the injective mode, an encryption of 0 looks like

$$(u \star x_0, u \star x_1) = (u \star x_0, (u+g) \star x_0),$$

for a random $u \leftarrow \mathbb{G}$, while an encryption of 1 looks like

$$(u \star x_2, u \star x_3) = ((u+h) \star x_0, (u+k) \star x_0).$$

In order for there to be a collision between encryptions of 0 and 1, we would need some w, v satisfying

$$w \star x_0 = (v+h) \star x_0 \quad \text{and} \quad (w+g) \star x_0 = (v+k) \star x_0.$$

Because we are working with a regular group action, this implies $w = v + h$ and $w + g = v + k$, hence $g + h = k$, which contradicts the injectivity assumption. Therefore, decryption is perfectly correct.

Perfect Lossiness. In the lossy mode, note that an encryption of 1 takes the form

$$(u \star x_2, u \star x_3) = ((u+h) \star x_0,\ (u+g+h) \star x_0) = (u' \star x_0, (u'+g) \star x_0) = (u' \star x_0, u' \star x_1).$$

Here $u' = u + h$. Since u is sampled uniformly at random, so is u', and thus the distribution of an encryption of 1 is identical to that of an encryption of 0. Hence, encryptions of 0 and 1 are identically distributed, and the scheme is perfectly lossy.

Indistinguishability of modes. This follows immediately from Assumption 1. □

B Classical Single Effective Query (CSEQ)

B.1 Classical Single Effective Query (CSEQ) Model

In this section, we recall the *single effective query* security definition from [GLR+25] and, to distinguish it from our generalized notion, refer to this classical variant as CSEQ.

Definition 25 (The Single Effective Query Oracle)**.** *For a randomized function $f : \mathcal{X} \times \mathcal{R} \to \mathcal{Y}$, we define the classical single effective query oracle O_f^{CSEQ} as implementing the following algorithm:*

- *We assume it has oracle access to O_f, which maps $|x, r, u\rangle \mapsto |x, r, u \oplus f(x; r)\rangle$.*
- *The oracle maintains two internal registers: a workspace register $\mathcal{W}$ to perform intermediate computations, and a database register $\mathcal{D}$ to implement the database for the compressed random oracle. These registers are initialized to $|0\rangle_{\mathcal{W}} \otimes |\emptyset\rangle_{\mathcal{D}}$.*
- *To describe its behavior on queries, all we need to do is describe its behavior on basis states. We will maintain the invariant that the workspace register is always $|0\rangle_{\mathcal{W}}$. On query $|x, u, b\rangle_{\mathcal{Q}}$ on query register $\mathcal{Q}$, the oracle implements an isometry specified by the following steps on each basis state $|x, u, b\rangle_{\mathcal{Q}} \otimes |D\rangle_{\mathcal{D}} \otimes |0\rangle_{\mathcal{W}}$:*
  1. *If there exists some $x' \neq x$ such that $(x', r) \in D$ for some $r \in \mathcal{R}$, skip the following steps.*
  2. *Copy x into the workspace register*

$$|x, u, b\rangle_{\mathcal{Q}} \otimes |D\rangle_{\mathcal{D}} \otimes |0\rangle_{\mathcal{W}} \mapsto |x, u, b\rangle_{\mathcal{Q}} \otimes |D\rangle_{\mathcal{D}} \otimes |x, 0\rangle_{\mathcal{W}}.$$

  3. *Apply a compressed random oracle query isometry with query register $\mathcal{W}$ and database register $\mathcal{D}$.*

4. *On the basis states apply the following unitary map that acts on registers $\mathcal{Q}, \mathcal{W}$:*

$$|x, u, b\rangle_{\mathcal{Q}} \otimes |D\rangle_{\mathcal{D}} \otimes |x, r\rangle_{\mathcal{W}} \mapsto |x, u \oplus f(x; r), b \oplus 1\rangle_{\mathcal{Q}} \otimes |D\rangle_{\mathcal{D}} \otimes |x, r\rangle_{\mathcal{W}}$$

5. *Uncompute the workspace register: first, query the compressed oracle again with the query register being $\mathcal{W}$ and the database register being $\mathcal{D}$, and then copy the input x.*

Definition 26 (CSEQ-based simulation security for one-time programs)**.** *A one-time program compiler* OTP *satisfies* CSEQ*-based simulation security for a class $\mathcal{F}$ of randomized functions if there exists a q.p.t. simulator* Sim *such that for every $f \in \mathcal{F}$ and for every q.p.t. distinguisher D, there exists a negligible function* $\mathsf{negl}(\lambda)$ *such that*

$$\left|\Pr\left[1 \leftarrow D(1^\lambda, f, \mathsf{OTP}(1^\lambda, f))\right] - \Pr\left[1 \leftarrow D(1^\lambda, f, \mathsf{Sim}^{O_f^{\mathsf{CSEQ}}}(1^\lambda))\right]\right| \leq \mathsf{negl}(\lambda).$$

By giving f as input to the distinguishing algorithm, we mean that D gets access to f in a way that it can evaluate on any (x, r) of its choice. Note that this is actually redundant, because the order of quantifiers is such that D is allowed to depend on the choice of f.

B.2 When is CSEQ a special case of SEQ

We will show that for any classical functionality f with a domain size $|\mathcal{X}| > 1$, there is a channel Φ_f such that the classical SEQ oracle O_f^{CSEQ} reveals the same information as the (channel) SEQ oracle $O_{\Phi_f}^{\mathsf{SEQ}}$.

The channel Φ_f computes f on any input x and uses a (compressed) random oracle H to sample r. Given a function $f : \mathcal{X} \times \mathcal{R} \to \mathcal{Y}$ and a random oracle $H : \mathcal{X} \to \mathcal{R}$, Φ_f maps $x \to f(x; H(x))$ and also flips a bit b to indicate that it has executed. Φ_f is essentially the same as the classical SEQ oracle O_f^{CSEQ}, except without step 1 of O_f^{CSEQ}.

Definition 27 (Channel Φ_f)**.** *Given a randomized classical function $f : \mathcal{X} \times \mathcal{R} \to \mathcal{Y}$, let Φ_f be the following quantum channel.*

- *The channel's internal register has two parts $\mathcal{P} = \mathcal{D} \times \mathcal{W}$. $\mathcal{D}$ is a database register for the compressed random oracle, and $\mathcal{W}$ is a workspace register to perform intermediate computations. $\mathcal{D}$ is initialized to $|\emptyset\rangle_{\mathcal{D}}$, and we maintain the invariant that $\mathcal{W}$ is in the state $|0, 0\rangle_{\mathcal{W}}$ at the start of each query.*
- *The channel's Stinespring unitary U_{Φ_f} acts as follows on each basis state $|x, u, b\rangle_{\mathcal{Q}} \otimes |D\rangle_{\mathcal{D}} \otimes |0, 0\rangle_{\mathcal{W}}$:*

  1. *Copy x into the workspace register:*

  $$|x, u, b\rangle_{\mathcal{Q}} \otimes |D\rangle_{\mathcal{D}} \otimes |0, 0\rangle_{\mathcal{W}} \mapsto |x, u, b\rangle_{\mathcal{Q}} \otimes |D\rangle_{\mathcal{D}} \otimes |x, 0\rangle_{\mathcal{W}}$$

  2. *Apply the query operation of the compressed oracle with $\mathcal{W}$ as the query register and $\mathcal{D}$ as the database register.*
  3. *Apply the unitary that acts as follows on the basis states:*

  $$|x, u, b\rangle_{\mathcal{Q}} \otimes |D\rangle_{\mathcal{D}} \otimes |x, r\rangle_{\mathcal{W}} \mapsto |x, u \oplus f(x; r), b \oplus 1\rangle_{\mathcal{Q}} \otimes |D\rangle_{\mathcal{D}} \otimes |x, r\rangle_{\mathcal{W}}$$

  4. *Uncompute step 2 and then step 1.*

Note that f and Φ_f provide no one-time guarantee. They never reject queries, so the user may make many effective queries. There are two ways to limit the user's queries: we can implement f with the classical SEQ oracle O_f^{CSEQ} and implement Φ_f with the SEQ oracle $O_{\Phi_f}^{\mathsf{SEQ}}$. The purpose of both oracles is to restrict the user to just one effective query.

Which oracle reveals more information about f? In fact, they reveal the same information. We formalize this by showing that O_f^{CSEQ} can be used to simulate $O_{\Phi_f}^{\mathsf{SEQ}}$, and vice versa as long as $|\mathcal{X}| > 1$ (theorem 15).

To prove the first simulation indistinguishability in theorem 15, we first define the CSEQ-wrapper simulator Sim'.

The simulator Sim' below has the following internal registers. Let $\mathcal{P}$ store the program state of P, which is initialized to $|\psi\rangle$. Let $\mathcal{D}$ be an internal database register that stores a query $x \in \mathcal{X} \cup \{\emptyset\}$ and is initialized to $|\emptyset\rangle$. Let $\mathcal{W}_x \times \mathcal{W}_y \times \mathcal{W}_b$ be work registers that store values $(x, y, b) \in (\mathcal{X} \cup \{\emptyset\}) \times (\mathcal{Y} \cup \{\emptyset\}) \times \{0, 1\}$. $\mathcal{W}_x \times \mathcal{W}_y \times \mathcal{W}_b$ are initialized to $|\emptyset, \emptyset, 0\rangle$.

We represent elements of $\mathcal{X} \cup \{\emptyset\}$ as the following binary strings. Each $x \in \mathcal{X}$ becomes $x\|1$ and $\emptyset$ becomes $0^n\|0$. In particular, $x = 0^n$ is represented as $0^n\|1$, which is different from the representation $0^n\|0$ of $\emptyset$. Therefore, we have that $\emptyset + x = x$, and $x + x = \emptyset$. The same kind of representation is used for $\mathcal{Y} \cup \{\emptyset\}$.

$\mathsf{Sim}'(P)$ works as follows. First, it uses the EvalAndCopy operation to evaluate $P(x)$. Rather than evaluating the program on registers $\mathcal{Q}_x \times \mathcal{Q}_u$ directly, EvalAndCopy evaluates it on work registers $\mathcal{W}_x \times \mathcal{W}_y$ instead, and copies the inputs and outputs between $\mathcal{Q}_x \times \mathcal{Q}_u$ and $\mathcal{W}_x \times \mathcal{W}_y$. This provides insulation from any pathological behavior of Eval. Even if Eval modifies or reads from its query register arbitrarily, we are guaranteed that EvalAndCopy only reads from $\mathcal{Q}_x$ and only writes to $\mathcal{Q}_y$.

Next, $\mathsf{Sim}'(P)$ stores a query x' in the database $\mathcal{D}$ and rejects any new query x if $x' \notin \{x, \emptyset\}$. This is analogous O_f^{CSEQ}'s database. The ReflectAndCopy operation updates $\mathsf{Sim}'(P)$'s database by copying x from $\mathcal{Q}_x$ to $\mathcal{D}$ if and only if the remaining internal registers $\mathcal{P} \times \mathcal{W}_x \times \mathcal{W}_y \times \mathcal{W}_b$ are in their initial state (lemma 18).

Definition 28 (Simulator Sim' for O_f^{CSEQ})**.**

- Sim' *takes as input a testable quantum program* $P = (|\psi\rangle, \mathsf{Eval}, R)$.
- *Initialize the internal state to* $|\psi\rangle_{\mathcal{P}} \otimes |\emptyset\rangle_{\mathcal{D}} \otimes |\emptyset\rangle_{\mathcal{W}_x} \otimes |\emptyset\rangle_{\mathcal{W}_y} \otimes |0\rangle_{\mathcal{W}_b}$.
- *Given a query* $|x, u, b\rangle_{\mathcal{Q}}$ *and database* $|x'\rangle_{\mathcal{D}}$, Sim' *acts as follows.*
  1. *If* $x' \notin \{x, \emptyset\}$*, do nothing and ignore the remaining steps.*
  2. ReflectAndCopy*: If* $\mathcal{W}_x \times \mathcal{W}_y$ *contains* $(\emptyset, \emptyset)$*, then do the following:*
     (a) *Apply the reflection oracle* R *to registers* $(\mathcal{W}_b, \mathcal{P})$.
     (b) *If* $\mathcal{W}_b$ *contains* 1*, then* CNOT *the value* x *from* $\mathcal{Q}_x$ *onto* $\mathcal{D}$.
     (c) *Apply the reflection oracle* R *to registers* $(\mathcal{W}_b, \mathcal{P})$.
  3. EvalAndCopy*:*
     (a) CNOT *the value* x *from* $\mathcal{Q}_x$ *to* $\mathcal{W}_x$.
     (b) *Apply* Eval *to registers* $(\mathcal{W}_x, \mathcal{W}_y, \mathcal{P})$.
     (c) CopyOutput*:* CNOT *the contents of* $\mathcal{W}_y$ *onto* $\mathcal{Q}_u$.
     (d) *Apply* $\mathsf{Eval}^\dagger$ *to registers* $(\mathcal{W}_x, \mathcal{W}_y, \mathcal{P})$.
     (e) CNOT *the value* x *from* $\mathcal{Q}_x$ *to* $\mathcal{W}_x$.
  4. *Apply* ReflectAndCopy *(step 2) again.*
  5. Bitflip*: Apply* X *to the* $\mathcal{Q}_b$ *register.*

In summary, the simulator acts as follows: it acts as the identity on states $|x, u, b\rangle\,|D\rangle\,|\cdots\rangle$ if $x' \in D$ for some $x' \neq x$, and on the rest of the state space it acts as the unitary

$$\mathsf{Bitflip} \circ \mathsf{ReflectAndCopy} \circ \mathsf{EvalAndCopy} \circ \mathsf{ReflectAndCopy}.$$

The next lemma says that ReflectAndCopy acts similarly to a reflection oracle on registers $\mathcal{P} \times \mathcal{W}_x \times \mathcal{W}_y \times \mathcal{W}_b$. It CNOTS x from $\mathcal{Q}_x$ to $\mathcal{D}$ if and only if $\mathcal{P} \times \mathcal{W}_x \times \mathcal{W}_y \times \mathcal{W}_b$ is in its initial state.

Lemma 18. *If P is a testable quantum program, then* ReflectAndCopy *(step 2 of* $\mathsf{Sim}'(P)$*) is equivalent to the operation that applies* CNOT *from $\mathcal{Q}_x$ to $\mathcal{D}$ controlled on $\mathcal{P} \times \mathcal{W}_x \times \mathcal{W}_y \times \mathcal{W}_b$ being in their initial state $|\psi, \emptyset, \emptyset, 0\rangle$.*

Additionally, after any number of queries to $\mathsf{Sim}'(P)$*, $\mathcal{W}_b$ is in state $|0\rangle$.*

Proof. First, $\mathcal{W}_b$ is initialized to $|0\rangle$. Next, note that $\mathcal{W}_b$ is only modified during the ReflectAndCopy operation. Let us assume that at the start of a given ReflectAndCopy operation, $\mathcal{W}_b$ contains $|0\rangle$. We will show that at the end of this operation, $\mathcal{W}_b$ still contains $|0\rangle$. This establishes that $\mathcal{W}_b = |0\rangle$ after every query to $\mathsf{Sim}'(P)$.

Let us consider how ReflectAndCopy acts on the following basis states.

Case 1: $\mathcal{W}_x \times \mathcal{W}_y$ contains a state orthogonal to $|\emptyset, \emptyset\rangle$. Then ReflectAndCopy acts as the identity, so at the end of ReflectAndCopy, $\mathcal{W}_b = |0\rangle$, and $\mathcal{Q}_x \times \mathcal{D} \times \mathcal{P} \times \mathcal{W}_x \times \mathcal{W}_y$ are unchanged.

Case 2: $\mathcal{W}_x \times \mathcal{W}_y = |\emptyset, \emptyset\rangle$, but $\mathcal{P}$ contains a state that is orthogonal to $|\psi\rangle$. ReflectAndCopy first applies R to $(\mathcal{W}_b, \mathcal{P})$, resulting in $\mathcal{W}_b = |0\rangle$. Then the CNOT operation from $\mathcal{Q}_x$ to $\mathcal{D}$ is controlled on 0, so we do not copy x to $\mathcal{D}$, and we can replace this operation with the identity. Finally, the second application of R also leaves the $\mathcal{W}_b = |0\rangle$. At the end of ReflectAndCopy, $\mathcal{W}_b = |0\rangle$, and $\mathcal{Q}_x \times \mathcal{D} \times \mathcal{P} \times \mathcal{W}_x \times \mathcal{W}_y$ are unchanged.

Case 3: $\mathcal{W}_x \times \mathcal{W}_y = |\emptyset, \emptyset\rangle$, and $\mathcal{P} = |\psi\rangle$. ReflectAndCopy first applies R to $(\mathcal{W}_b, \mathcal{P})$, resulting in $\mathcal{W}_b = |1\rangle$. Then, the CNOT operation copies x from $\mathcal{Q}_x$ to $\mathcal{D}$. Since the CNOT step acts only on $\mathcal{W}_b \times \mathcal{Q}_x \times \mathcal{D}$, this does not affect the $\mathcal{P}$ register, which will still be in the state $|\psi\rangle$. Finally, the second application of the reflection oracle R will XOR the $\mathcal{W}_b$ register with 1, returning it to $|0\rangle$. At the end of ReflectAndCopy, $\mathcal{W}_b = |0\rangle$, and $\mathcal{P} \times \mathcal{W}_x \times \mathcal{W}_y$ are unchanged.

In summary, we've shown that for each type of basis state listed above, ReflectAndCopy keeps $\mathcal{W}_b = |0\rangle$, and it performs a CNOT from $\mathcal{Q}_x \to \mathcal{D}$ controlled on $\mathcal{P} \times \mathcal{W}_x \times \mathcal{W}_y = |\psi, \emptyset, \emptyset\rangle$. Any state on registers $\mathcal{Q}_x \times \mathcal{D} \times \mathcal{P} \times \mathcal{W}_x \times \mathcal{W}_y$ is in the span of the basis states above, so we have completely characterized the behvaior of ReflectAndCopy. □

Next, let us define a testable quantum program $P_f = (|+\rangle, \mathsf{Eval}_f, R_+)$ as follows:

- $|+\rangle = \frac{1}{\sqrt{|\mathcal{R}|}} \cdot \sum_{r \in \mathcal{R}} |r\rangle$
- Eval_f is a unitary that maps $|x, u\rangle \otimes |r\rangle \mapsto |x, u \oplus f(x; r)\rangle \otimes |r\rangle$
- R_+ is the reflection about $|+\rangle$

Note that P_f is a testable quantum program that implements f with 0 error.

Lemma 19. *Oracles for* $\mathsf{Sim}'(P_f)$ *and* O_f^{CSEQ} *are perfectly indistinguishable to any adversary making an unbounded number of quantum queries.*

Proof. The following hybrids transform O_f^{CSEQ} into $\mathsf{Sim}'(P_f)$.

Hybrid 0. This is the CSEQ oracle, except that the size of the compressed oracle database is fixed to record at most 1 query. This is equivalent to the CSEQ oracle because the CSEQ oracle holds at most one entry in the database at any point in time, by Claim 4.7 of [GLR+25].

Recall that StdDecomp applied to $\mathcal{W} \times \mathcal{D}$ swaps the states $|x, r\rangle_{\mathcal{W}} \otimes |\emptyset, \emptyset\rangle_{\mathcal{D}}$ and $|x, r\rangle_{\mathcal{W}} \otimes |x, +\rangle_{\mathcal{D}}$, and acts as the identity on the rest of the space. Additionally, CStO' applied to $\mathcal{W} \times \mathcal{D}$ maps $|x, r\rangle_{\mathcal{W}} \otimes |D\rangle_{\mathcal{D}} \mapsto |x, r \oplus D(x)\rangle \otimes |D\rangle$.

The oracle initializes its internal registers $\mathcal{W} \times \mathcal{D} = |0\rangle \times |\emptyset\rangle$. On query $|x, u, b\rangle_{\mathcal{Q}}$, the oracle acts as follows:

1. If there exists some $x' \neq x$ such that $(x', r) \in D$ for some $r \in \mathcal{R}$, skip the following steps.
2. Copy x into the workspace register
$$|x\rangle_{\mathcal{Q}_x} \otimes |0\rangle_{\mathcal{W}} \mapsto |x\rangle_{\mathcal{Q}_x} \otimes |x, 0\rangle_{\mathcal{W}}.$$
3. Apply the compressed random oracle to registers $\mathcal{W} \times \mathcal{D}$ by executing the following unitaries:
$$\mathsf{StdDecomp} \circ \mathsf{CStO}' \circ \mathsf{StdDecomp}$$
4. Apply the following unitary map:
$$|u, b\rangle_{\mathcal{Q}_u \times \mathcal{Q}_b} \otimes |x, r\rangle_{\mathcal{W}} \mapsto |u \oplus f(x; r), b \oplus 1\rangle_{\mathcal{Q}_u \times \mathcal{Q}_b} \otimes |x, r\rangle_{\mathcal{W}}$$
5. Query the compressed oracle again:
$$\mathsf{StdDecomp} \circ \mathsf{CStO}' \circ \mathsf{StdDecomp}$$
6. Copy x into the workspace register
$$|x\rangle_{\mathcal{Q}_x} \otimes |x, 0\rangle_{\mathcal{W}} \mapsto |x\rangle_{\mathcal{Q}_x} \otimes |0\rangle_{\mathcal{W}}$$

Hybrid 1. We've canceled out operations with their inverse whereever possible. We have also moved the bitflip on $\mathcal{Q}_b$ to the final step.

The oracle initializes its internal registers $\mathcal{W} \times \mathcal{D} = |0\rangle \times |\emptyset\rangle$. On query $|x, u, b\rangle_{\mathcal{Q}}$, the oracle acts as follows:

1. If there exists some $x' \neq x$ such that $(x', r) \in D$ for some $r \in \mathcal{R}$, skip the following steps.
2. Apply StdDecomp to registers $\mathcal{Q}_x \times \mathcal{D}$.
3. Apply the following unitary map:
$$|x, u, b\rangle_{\mathcal{Q}} \otimes |x, r\rangle_{\mathcal{D}} \mapsto |x, u \oplus f(x; r), b\rangle_{\mathcal{Q}} \otimes |x, r\rangle_{\mathcal{D}}$$
4. Apply StdDecomp to registers $\mathcal{Q}_x \times \mathcal{D}$.
5. Apply X (bitflip) to $\mathcal{Q}_b$.

Claim 5. *The oracles in hybrids 0 and 1 are perfectly indistinguishable after any number of queries.*

Proof. First, in hybrid 0, StdDecomp applies $\mathsf{StdDecomp}_x$ to $\mathcal{D}$ controlled on the value of x written on $\mathcal{W}$. In hybrid 1, StdDecomp applies $\mathsf{StdDecomp}_x$ to $\mathcal{D}$ controlled on the value of x written on $\mathcal{Q}_x$. These operations are equivalent because during any query in hybrid 0, the value of x written on $\mathcal{Q}_x$ matches the value written on $\mathcal{W}$ whenever StdDecomp is called.

Let us modify hybrid 0 so that StdDecomp is controlled on $\mathcal{Q}_x$ instead of $\mathcal{W}$.

Second, StdDecomp now commutes with items 2, 4 and 6 of hybrid 0. This is because StdDecomp applies $\mathsf{StdDecomp}_x$ to $\mathcal{D}$ controlled on the computational-basis value of $\mathcal{Q}_x$. Step 2 applies a unitary to $\mathcal{W}$ controlled on the computational-basis value of $\mathcal{Q}_x$. StdDecomp and step 2 commute because they are controlled by the computational-basis value of the same register $\mathcal{Q}_x$, and they apply a controlled unitary to disjoint registers, $\mathcal{D}$ and $\mathcal{W}$ respectively. The same reasoning shows that StdDecomp commutes with step 6. Step 4 does not act on $\mathcal{Q}_x$ or D, so it commutes with StdDecomp because they act on disjoint registers.

Let us switch the order of step 2 and the first StdDecomp in step 3. Let us also switch the order of step 6 and the last StdDecomp in step 5.

Third, the final application of StdDecomp in step 3 cancels out with the first application of StdDecomp in step 5. This is because $\mathsf{StdDecomp}^{-1} = \mathsf{StdDecomp}$, and the only operation that occurs in between these two applications of StdDecomp is step 4, which commutes with StdDecomp.

Fourth, after making the changes above, we have that hybrid 0 is equivalent to the following operations:

1. If there exists some $x' \neq x$ such that $(x', r) \in D$ for some $r \in \mathcal{R}$, skip the following steps.
2. Apply StdDecomp to $\mathcal{Q}_x \times \mathcal{D}$.
3. Step 2: $|x\rangle_{\mathcal{Q}_x} \otimes |0\rangle_{\mathcal{W}} \to |x\rangle_{\mathcal{Q}_x} \otimes |x, 0\rangle_{\mathcal{W}}$
4. CStO': $|x, 0\rangle_{\mathcal{W}} \otimes |x, r\rangle_{\mathcal{D}} \to |x, r\rangle_{\mathcal{W}} \otimes |x, r\rangle_{\mathcal{D}}$
5. Step 4: $|u, b\rangle_{\mathcal{Q}_u \times \mathcal{Q}_b} \otimes |x, r\rangle_{\mathcal{W}} \mapsto |u \oplus f(x; r), b \oplus 1\rangle_{\mathcal{Q}_u \times \mathcal{Q}_b} \otimes |x, r\rangle_{\mathcal{W}}$
6. CStO': $|x, r\rangle_{\mathcal{W}} \otimes |x, r\rangle_{\mathcal{D}} \to |x, 0\rangle_{\mathcal{W}} \otimes |x, r\rangle_{\mathcal{D}}$
7. Step 6: $|x\rangle_{\mathcal{Q}_x} \otimes |x, 0\rangle_{\mathcal{W}} \to |x\rangle_{\mathcal{Q}_x} \otimes |0\rangle_{\mathcal{W}}$
8. Apply StdDecomp to $\mathcal{Q}_x \times \mathcal{D}$.

Note that the middle steps, Step 6 $\circ$ CStO' $\circ$ Step 4 $\circ$ CStO' $\circ$ Step 2, are all classical operations, and they are equivalent to the following operation:

$$|x, u, b\rangle_{\mathcal{Q}} \otimes |0\rangle_{\mathcal{W}} \otimes |x, r\rangle_{\mathcal{D}} \to |x, u \oplus f(x; r), b \oplus 1\rangle_{\mathcal{Q}} \otimes |0\rangle_{\mathcal{W}} \otimes |x, r\rangle_{\mathcal{D}}$$

Note that $\mathcal{W}$ is returned to the $|0\rangle$ state by the end of this sequence of operations, so we can ignore it.

Fifth, let us delay applying the bitflip to $\mathcal{Q}_b$ until after all the other steps have finished. This is equivalent to the steps above because the bitflip commutes with all other steps.

Now we have that hybrid 0 is equivalent to the following steps:

1. If there exists some $x' \neq x$ such that $(x', r) \in D$ for some $r \in \mathcal{R}$, skip the following steps.
2. Apply StdDecomp to $\mathcal{Q}_x \times \mathcal{D}$.
3. Apply the following unitary map:
$$|x, u, b\rangle_{\mathcal{Q}} \otimes |x, r\rangle_{\mathcal{D}} \to |x, u \oplus f(x; r), b\rangle_{\mathcal{Q}} \otimes |x, r\rangle_{\mathcal{D}}$$
4. Apply StdDecomp to $\mathcal{Q}_x \times \mathcal{D}$.
5. Apply X (bitflip) to $\mathcal{Q}_b$.

This is exactly hybrid 1, so we've shown that hybrids 0 and 1 are perfectly indistinguishable. □

Hybrid 2. Let us split the database register $\mathcal{D}$ into two registers, called $\mathcal{D} \times \mathcal{P}$ to match the eventual notation of $\mathsf{Sim}'(P_f)$. $\mathcal{D}$ will store the database's x-value, and $\mathcal{P}$ will store its r-value.

Let Record be the unitary that swaps the states $|x\rangle_{\mathcal{Q}_x} \otimes |\emptyset\rangle_{\mathcal{D}} |+\rangle_{\mathcal{P}}$ and $|x\rangle_{\mathcal{Q}_x} \otimes |x\rangle_{\mathcal{D}} |+\rangle_{\mathcal{P}}$, and acts as the identity on the rest of the space. Additionally, let Decomp be the unitary that swaps $|\emptyset\rangle_{\mathcal{P}}$ and $|+\rangle_{\mathcal{P}}$.

In this hybrid, the oracle initializes database registers $\mathcal{D} \times \mathcal{P} = |\emptyset, \emptyset\rangle$ and work registers $\mathcal{W}_x \times \mathcal{W}_y \times \mathcal{W}_b = |\emptyset, \emptyset, 0\rangle$, although the oracle never operates on $\mathcal{W}_x \times \mathcal{W}_y \times \mathcal{W}_b$.

Next, the oracle answers queries on $|x, u, b\rangle_{\mathcal{Q}} \otimes |x'\rangle_{\mathcal{D}}$ as follows:

1. If $x' \notin \{x, \emptyset\}$, then skip the following steps.
2. Apply Record $\circ$ Decomp to the $(\mathcal{Q}_x, \mathcal{D}, \mathcal{P})$ registers.
3. Apply Eval_f on the $(\mathcal{Q}_x, \mathcal{Q}_u, \mathcal{P})$ registers:
$$|x, u\rangle_{\mathcal{Q}_x \times \mathcal{Q}_u} \otimes |r\rangle_{\mathcal{P}} \to |x, u \oplus f(x; r)\rangle_{\mathcal{Q}_x \times \mathcal{Q}_u} \otimes |r\rangle_{\mathcal{P}}$$
4. Apply Decomp $\circ$ Record on the $(\mathcal{Q}_x, \mathcal{D}, \mathcal{P})$ registers.
5. Apply X (bitflip) to $\mathcal{Q}_b$.

Claim 6. *The oracles in hybrids 1 and 2 are perfectly indistinguishable after any number of quantum queries.*

Proof. We claim that the unitaries defined in hybrids 1 and 2 are identical. We argue this by choosing a suitable basis for states on registers $\mathcal{Q}_x \times \mathcal{D}$ (hybrid 1) or $\mathcal{Q} \times \mathcal{P} \times \mathcal{D}$ (hybrid 2) and then tracing how the unitaries StdDecomp, Eval_f, Record $\circ$ Decomp and Decomp $\circ$ Record transform these basis states.

The unitaries in hybrids 1 and 2 both act as the identity on basis states of the form $|x\rangle_{\mathcal{Q}_x} |x'\rangle_{\mathcal{D}} |\cdots\rangle$ for $x' \notin \{x, \emptyset\}$, so we will restrict our attention to states supported on $|x\rangle_{\mathcal{Q}_x} |x\rangle_{\mathcal{D}}$ and $|x\rangle_{\mathcal{Q}_x} |\emptyset\rangle_{\mathcal{D}}$. For the $\mathcal{P}$ register, our orthogonal "basis" will be $|\emptyset\rangle$, $|+\rangle$ and $|-\rangle$, where, in an abuse of notation, we write $|-\rangle$ to denote any state in the span of $\{\frac{1}{\sqrt{|\mathcal{R}|}} \cdot \sum_{r \in \mathcal{R}} (-1)^{r \cdot u} |r\rangle\}_{u \neq 0}$.

- The starting state of the first query is $|x\rangle_{\mathcal{Q}_x} |\emptyset\rangle_{\mathcal{D}} |\emptyset\rangle_{\mathcal{P}}$, and both StdDecomp and Record $\circ$ Decomp map it to $|x\rangle |x\rangle |+\rangle$.
- Eval_f maps $|x\rangle |x\rangle |+\rangle$ and $|x\rangle |x\rangle |-\rangle$ to superpositions of $|x\rangle |x\rangle |+\rangle$ and $|x\rangle |x\rangle |-\rangle$ (where $|-\rangle$ does not necessarily refer to the same state as before the application of Eval_f).
- Both StdDecomp and Decomp $\circ$ Record map these states as follows:
$$|x\rangle |x\rangle |+\rangle \mapsto |x\rangle |\emptyset\rangle |\emptyset\rangle \,, \text{ and}$$
$$|x\rangle |x\rangle |-\rangle \mapsto |x\rangle |x\rangle |-\rangle \,.$$
- Note that StdDecomp, Record$\circ$Decomp, and Decomp$\circ$Record each leave $|x\rangle |x\rangle |-\rangle$ invariant.

Let us define the set of *reachable* states to be states whose database is supported on eigenstates of the form: $|x, -\rangle, |\emptyset, \emptyset\rangle$. We claim that the database is in a reachable state at the start of any query. First, the state of the database at the start of the first query is reachable. Second, the observations above imply that if the database is in a reachable state at the start of a query, then it will end the query in a reachable state.

The only difference in the operations applied by hybrids 1 and 2 is that hybrid 1 uses StdDecomp twice, and hybrid 2 uses Record $\circ$ Decomp and Decomp $\circ$ Record instead.

Let us step through a query in hybrids 1 or 2 when the starting state is reachable. The first time that StdDecomp or Record$\circ$Decomp is called during the query, the database is supported on states of the form $|\emptyset,\emptyset\rangle, |x,-\rangle$. We've already argued that StdDecomp, Record$\circ$Decomp act equivalently on such states. After this operation, the database is supported on states of the form $|x,+\rangle, |x,-\rangle$. Then Eval_f maps these states to superpositions of $|x,+\rangle, |x,-\rangle$. Next, StdDecomp or Decomp$\circ$Record is called. We've argued that StdDecomp and Decomp$\circ$Record act equivalently on database states of the form $|x,+\rangle, |x,-\rangle$.

This shows that the unitaries in hybrids 1 and 2 act equivalently on any reachable state, so hybrids 1 and 2 are perfectly indistinguishable. □

Hybrid 3. This is $\mathsf{Sim}'(P_f)$.

In this hybrid, the oracle initializes internal registers $\mathcal{D}\times\mathcal{P}\times\mathcal{W}_x\times\mathcal{W}_y\times\mathcal{W}_b = |\emptyset\rangle\otimes|+\rangle\otimes|\emptyset,\emptyset,0\rangle$, and answers queries on $|x,u,b\rangle\otimes|x'\rangle_{\mathcal{D}}$ as follows:

1. If $x'\notin\{x,\emptyset\}$, do nothing and ignore the remaining steps.
2. Apply ReflectAndCopy to the $(\mathcal{Q}_x,\mathcal{D},\mathcal{P},\mathcal{W}_x,\mathcal{W}_y,\mathcal{W}_b)$ registers.
3. Apply EvalAndCopy on the $(\mathcal{Q}_x,\mathcal{Q}_u,\mathcal{P},\mathcal{W}_x,\mathcal{W}_y)$ registers.
4. Apply ReflectAndCopy to the $(\mathcal{Q}_x,\mathcal{D},\mathcal{P},\mathcal{W}_x,\mathcal{W}_y,\mathcal{W}_b)$ registers.
5. Apply Bitflip to $\mathcal{Q}_b$.

Claim 7. *The oracles in hybrids 2 and 3 are perfectly indistinguishable after any number of quantum queries.*

Proof. First, Record is equivalent to ReflectAndCopy for program P_f. ReflectAndCopy copies x from $\mathcal{Q}_x$ to $\mathcal{D}$ controlled on $\mathcal{P}\times\mathcal{W}_x\times\mathcal{W}_y\times\mathcal{W}_b$ being in the initial state (lemma 18). Record copies x from $\mathcal{Q}_x$ to $\mathcal{D}$ controlled on $\mathcal{P}$ being in its initial state $|+\rangle$. Furthermore, every time that Record is called in hybrid 2, the $\mathcal{W}_x\times\mathcal{W}_y\times\mathcal{W}_b$ registers are in their initial state because hybrid 2 does not operate on these registers after initializing them. Therefore, we can replace Record with ReflectAndCopy.

Second, Eval_f is equivalent to EvalAndCopy for program P_f. If $\mathcal{W}_x\times\mathcal{W}_y$ are in the state $|\emptyset,\emptyset\rangle$ at the start of EvalAndCopy, then EvalAndCopy acts as follows:

$$\begin{aligned}
|x,u\rangle_{\mathcal{Q}_x\times\mathcal{Q}_u}\otimes|\emptyset,\emptyset\rangle_{\mathcal{W}_x\times\mathcal{W}_y}\otimes|r\rangle_{\mathcal{P}} &\to |x,u\rangle_{\mathcal{Q}_x\times\mathcal{Q}_u}\otimes|x,\emptyset\rangle_{\mathcal{W}_x\times\mathcal{W}_y}\otimes|r\rangle_{\mathcal{P}}\\
&\to |x,u\rangle_{\mathcal{Q}_x\times\mathcal{Q}_u}\otimes|x,f(x;r)\rangle_{\mathcal{W}_x\times\mathcal{W}_y}\otimes|r\rangle_{\mathcal{P}}\\
&\to |x,u\oplus f(x;r)\rangle_{\mathcal{Q}_x\times\mathcal{Q}_u}\otimes|x,f(x;r)\rangle_{\mathcal{W}_x\times\mathcal{W}_y}\otimes|r\rangle_{\mathcal{P}}\\
&\to |x,u\oplus f(x;r)\rangle_{\mathcal{Q}_x\times\mathcal{Q}_u}\otimes|x,\emptyset\rangle_{\mathcal{W}_x\times\mathcal{W}_y}\otimes|r\rangle_{\mathcal{P}}\\
&\to |x,u\oplus f(x;r)\rangle_{\mathcal{Q}_x\times\mathcal{Q}_u}\otimes|\emptyset,\emptyset\rangle_{\mathcal{W}_x\times\mathcal{W}_y}\otimes|r\rangle_{\mathcal{P}}
\end{aligned}$$

We claim that $\mathcal{W}_x\times\mathcal{W}_y = |\emptyset,\emptyset\rangle$ at the start of every EvalAndCopy query. Before any queries have been made, $\mathcal{W}_x\times\mathcal{W}_y = |\emptyset,\emptyset\rangle$. Next, the computational-basis values of $\mathcal{W}_x\times\mathcal{W}_y$ are only modified during EvalAndCopy. We showed above that EvalAndCopy returns $\mathcal{W}_x\times\mathcal{W}_y$ to $|\emptyset,\emptyset\rangle$, so $\mathcal{W}_x\times\mathcal{W}_y = |\emptyset,\emptyset\rangle$ at the start of every EvalAndCopy query.

We also showed that EvalAndCopy is equivalent to

$$|x,u\rangle_{\mathcal{Q}_x\times\mathcal{Q}_u}\otimes|r\rangle_{\mathcal{P}}\to|x,u\oplus f(x;r)\rangle_{\mathcal{Q}_x\times\mathcal{Q}_u}\otimes|r\rangle_{\mathcal{P}}$$

which is the application of Eval_f to $\mathcal{Q}_x\times\mathcal{Q}_u\times\mathcal{P}$.

Third, let us remove all the Decomp operations from hybrid 2 as follows. The Decomp operation at the end of one query cancels with the Decomp operation at the start of the next one. This is because Decomp$\circ$Decomp $= I$, and Decomp operates only on the internal registers of the oracle.

Next, we can remove the first Decomp from the first query. This Decomp operation acts on $|\emptyset\rangle_{\mathcal{P}}$ to produce the state $|+\rangle_{\mathcal{P}}$. In hybrid 3, we remove this Decomp operation and instead have the oracle start with state $|+\rangle_{\mathcal{P}}$.

Finally, we can remove the final Decomp from the final query. Decomp acts only on the internal registers of the oracle, so the view of a distinguisher that has query access to the oracle is unchanged.

In summary, we have transformed hybrid 2 into hybrid 3 and shown that the two hybrids are equivalent. □

□

Lemma 20. *If P_1 and P_2 are testable quantum programs that implement the same functionality with 0 error, then oracles for $\mathsf{Sim}'(P_1)$ and $\mathsf{Sim}'(P_2)$ are perfectly indistinguishable to any adversary making an unbounded number of quantum queries.*

The intuition for this proof is that given two program states $|\psi^1\rangle, |\psi^2\rangle$ that implement the same sampling functionality, for each x, we can decompose each program state into a superposition over an orthonormal basis of programs that give deterministic output values. Since the two programs implement the same functionality, these bases are equivalent up to a rotation/labeling.

This nice picture breaks down when you consider querying the program on multiple x values, but the database that Sim' uses to record queries allows us to get around this issue and still make the argument go through.

Proof. First, for any program P and any $(x, y) \in \mathcal{X} \times \mathcal{Y}$, let $\alpha_{x,y} = \sqrt{\Pr[P(x) \to y]}$.

In $\mathsf{Sim}'(P)$, the initial state of $\mathcal{P} \times \mathcal{W}_x \times \mathcal{W}_y$ is $|\psi, \emptyset, \emptyset\rangle$. The following claim says that we can decompose $|\psi, \emptyset, \emptyset\rangle$ into a basis defined by the y-values that the program outputs. For each y-value, $|\psi_{x,y}\rangle$ is the component of $|\psi, \emptyset, \emptyset\rangle$ that produces output y when EvalAndCopy is called.

Claim 8. *For any $x \in \mathcal{X}$, there exists a set of orthonormal states $\{|\psi_{x,y}\rangle\}_{y\in\mathcal{Y}}$ on registers $\mathcal{P} \times \mathcal{W}_x \times \mathcal{W}_y$ such that for any $u \in \mathcal{Y} \cup \{\emptyset\}$ and any y in the support of $P(x)$,*

$$\mathsf{EvalAndCopy}\,|x, u\rangle_{\mathcal{Q}_x\times\mathcal{Q}_u} \otimes |\psi_{x,y}\rangle = |x, u + y\rangle_{\mathcal{Q}_x\times\mathcal{Q}_u} \otimes |\psi_{x,y}\rangle$$

Additionally, $|\psi, \emptyset, \emptyset\rangle$ is in the span of $\{|\psi_{x,y}\rangle\}_y$:

$$|\psi, \emptyset, \emptyset\rangle = \sum_{y\in\mathcal{Y}} \alpha_{x,y} |\psi_{x,y}\rangle$$

Proof. First, note that EvalAndCopy does not modify the computational-basis value of $\mathcal{Q}_x$. It just applies an operation that is controlled by $\mathcal{Q}_x$. When $\mathcal{Q}_x$ contains value x, the EvalAndCopy step acts as follows on registers $\mathcal{Q}_u \times \mathcal{W}_x \times \mathcal{W}_y \times \mathcal{P}$:

$$\mathsf{Eval}_x^\dagger \circ \mathsf{CopyOutput} \circ \mathsf{Eval}_x$$

where we define Eval_x to be the operation that CNOTs x onto register $\mathcal{W}_x$ and then applies Eval to $\mathcal{W}_x \times \mathcal{W}_y \times \mathcal{P}$.

Second, for each $x \in \mathcal{X}$ and each y in the support of $P(x)$, let $|\psi_{x,y}\rangle$ be the component of $|\psi, \emptyset, \emptyset\rangle$ that produces output y when the program is queried on x. Formally, for each $x \in \mathcal{X}$ and each y in the support of $P(x)$,

$$\text{let } |\psi_{x,y}\rangle = \frac{1}{\alpha_{x,y}} \cdot \mathsf{Eval}_x^\dagger \cdot (\mathbb{I}_{\mathcal{P}\times\mathcal{W}_x} \otimes |y\rangle \langle y|_{\mathcal{W}_y}) \cdot \mathsf{Eval}_x \cdot (|\psi, \emptyset, \emptyset\rangle_{\mathcal{P}\times\mathcal{W}_x\times\mathcal{W}_y})$$

This state is well-defined because for all y in the support of $P(x)$, $\alpha_{x,y} \neq 0$.

The state $|\psi_{x,y}\rangle$ defined above has unit norm. We know that when P is evaluated on x, the value written to $\mathcal{W}_y$ will be y with probability $\alpha_{x,y}^2 = \Pr[P(x) \to y]$. That means

$$\left\| (\mathbb{I}_{\mathcal{P}\times\mathcal{W}_x} \otimes |y\rangle \langle y|_{\mathcal{W}_y}) \cdot \mathsf{Eval}_x \cdot |\psi, \emptyset, \emptyset\rangle \right\| = \alpha_{x,y}$$

Next, since Eval_x is a unitary,

$$\begin{aligned}
\||\psi_{x,y}\rangle\| &= \left\| \frac{1}{\alpha_{x,y}} \cdot \mathsf{Eval}_x^\dagger \cdot (\mathbb{I}_{\mathcal{P}\times\mathcal{W}_x} \otimes |y\rangle \langle y|_{\mathcal{W}_y}) \cdot \mathsf{Eval}_x \cdot |\psi, \emptyset, \emptyset\rangle \right\| \\
&= \frac{1}{\alpha_{x,y}} \cdot \left\| (\mathbb{I}_{\mathcal{P}\times\mathcal{W}_x} \otimes |y\rangle \langle y|_{\mathcal{W}_y}) \cdot \mathsf{Eval}_x \cdot |\psi, \emptyset, \emptyset\rangle \right\| \\
&= \frac{1}{\alpha_{x,y}} \cdot \alpha_{x,y} = 1
\end{aligned}$$

Third, let us define $|\psi_{x,y}\rangle$ for any y not in the support of $P(x)$. The definition above will not suffice here because $\alpha_{x,y} = 0$.

$$\text{Let } |\psi_{x,y}\rangle = \mathsf{Eval}_x^\dagger \cdot (|\psi\rangle_{\mathcal{P}} |\emptyset\rangle_{\mathcal{W}_x} |y\rangle_{\mathcal{W}_y})$$

This state is well-defined and has unit norm because $\mathsf{Eval}_x^\dagger$ is a unitary.

Now we will prove some properties of $\{|\psi_{x,y}\rangle\}_{y\in\mathcal{Y}}$.

Fourth, for any $x \in \mathcal{X}$, the states $\{|\psi_{x,y}\rangle\}_{y\in\mathcal{Y}}$ are orthogonal. This is because for any $y \in \mathcal{Y}$, $\mathsf{Eval}_x |\psi_{x,y}\rangle$ is in the span of $\mathbb{I}_{\mathcal{P}\times\mathcal{W}_x} \otimes |y\rangle \langle y|_{\mathcal{W}_y}$. For any two distinct values $y, y' \in \mathcal{Y}$, $\mathbb{I}_{\mathcal{P}\times\mathcal{W}_x} \otimes |y\rangle \langle y|_{\mathcal{W}_y}$ and $\mathbb{I}_{\mathcal{P}\times\mathcal{W}_x} \otimes |y'\rangle \langle y'|_{\mathcal{W}_y}$ project onto orthogonal subspaces, so $\mathsf{Eval}_x |\psi_{x,y}\rangle$ and $\mathsf{Eval}_x |\psi_{x,y'}\rangle$ are orthogonal. Finally, Eval_x is a unitary, so it preserves inner products, and $|\psi_{x,y}\rangle$ and $|\psi_{x,y'}\rangle$ are orthogonal as well.

Fifth, $|\psi, \emptyset, \emptyset\rangle$ is in the span of $\{|\psi_{x,y}\rangle\}_{y\in\mathcal{Y}}$.

$$\begin{aligned}
|\psi, \emptyset, \emptyset\rangle &= \mathsf{Eval}_x^\dagger \cdot \mathsf{Eval}_x \cdot |\psi, \emptyset, \emptyset\rangle \\
&= \mathsf{Eval}_x^\dagger \cdot (\mathbb{I}_{\mathcal{P}\times\mathcal{W}_x} \otimes \mathbb{I}_{\mathcal{W}_y}) \cdot \mathsf{Eval}_x \cdot |\psi, \emptyset, \emptyset\rangle \\
&= \mathsf{Eval}_x^\dagger \cdot \left(\mathbb{I}_{\mathcal{P}\times\mathcal{W}_x} \otimes \sum_{y\in\mathcal{Y}\cup\{\emptyset\}} |y\rangle \langle y| \right) \cdot \mathsf{Eval}_x \cdot |\psi, \emptyset, \emptyset\rangle \\
&= \sum_{y\in\mathcal{Y}\cup\{\emptyset\}} \mathsf{Eval}_x^\dagger \cdot (\mathbb{I}_{\mathcal{P}\times\mathcal{W}_x} \otimes |y\rangle \langle y|) \cdot \mathsf{Eval}_x \cdot |\psi, \emptyset, \emptyset\rangle \\
&= \sum_{y\in\mathcal{Y}} \alpha_{x,y} |\psi_{x,y}\rangle
\end{aligned}$$

We used the fact that

$$\mathbb{I}_{\mathcal{P}\times\mathcal{W}_x} \otimes \left(\sum_{y\in\mathcal{Y}\cup\{\emptyset\}} |y\rangle \langle y| \right) = \sum_{y\in\mathcal{Y}\cup\{\emptyset\}} \mathbb{I}_{\mathcal{P}\times\mathcal{W}_x} \otimes |y\rangle \langle y|$$

We also used the fact that $(\mathbb{I}_{\mathcal{P}\times\mathcal{W}_x} \otimes |\emptyset\rangle \langle\emptyset|) \cdot \mathsf{Eval}_x \cdot |\psi, \emptyset, \emptyset\rangle = \mathbf{0}$ because $P(x)$ never outputs $\emptyset$. Likewise, for any y not in the support of $P(x)$, $(\mathbb{I}_{\mathcal{P}\times\mathcal{W}_x} \otimes |y\rangle \langle y|) \cdot \mathsf{Eval}_x \cdot |\psi, \emptyset, \emptyset\rangle = \mathbf{0}$.

Sixth, let us show that for any y in the support of $P(x)$,

$$\mathsf{EvalAndCopy} |x, u\rangle_{\mathcal{Q}_x\times\mathcal{Q}_u} \otimes |\psi_{x,y}\rangle = |x, u + y\rangle_{\mathcal{Q}_x\times\mathcal{Q}_u} \otimes |\psi_{x,y}\rangle$$

$\mathsf{EvalAndCopy}$ does not modify the computational basis value of $\mathcal{Q}_x$ and acts as $\mathsf{Eval}_x^\dagger \circ \mathsf{CopyOutput} \circ \mathsf{Eval}_x$ on the remaining registers. Let us apply Eval_x:

$$\begin{aligned}\mathsf{Eval}_x(|u\rangle_{\mathcal{Q}_u} \otimes |\psi_{x,y}\rangle) &= |u\rangle_{\mathcal{Q}_u} \otimes \mathsf{Eval}_x \cdot \left(\frac{1}{\alpha_{x,y}} \cdot \mathsf{Eval}_x^\dagger \cdot (\mathbb{I}_{\mathcal{P}\times\mathcal{W}_x} \otimes |y\rangle \langle y|_{\mathcal{W}_y}) \cdot \mathsf{Eval}_x \cdot |\psi,\emptyset,\emptyset\rangle\right) \\ &= \frac{1}{\alpha_{x,y}} \cdot |u\rangle_{\mathcal{Q}_u} \otimes \left((\mathbb{I}_{\mathcal{P}\times\mathcal{W}_x} \otimes |y\rangle \langle y|_{\mathcal{W}_y}) \cdot \mathsf{Eval}_x \cdot |\psi,\emptyset,\emptyset\rangle\right)\end{aligned}$$

Next, if we apply $\mathsf{CopyOutput}$, this copies y from $\mathcal{W}_y$ to $\mathcal{Q}_u$:

$$\mathsf{CopyOutput} \cdot \mathsf{Eval}_x \cdot (|u\rangle \otimes |\psi_{x,y}\rangle) = \frac{1}{\alpha_{x,y}} \cdot |u+y\rangle \otimes \left((\mathbb{I}_{\mathcal{P}\times\mathcal{W}_x} \otimes |y\rangle \langle y|_{\mathcal{W}_y}) \cdot \mathsf{Eval}_x \cdot |\psi,\emptyset,\emptyset\rangle\right)$$

Finally, let us apply $\mathsf{Eval}_x^\dagger$:

$$\begin{aligned}\mathsf{Eval}_x^\dagger \cdot \mathsf{CopyOutput} \cdot \mathsf{Eval}_x \cdot (|u\rangle \otimes |\psi_{x,y}\rangle) &= \frac{1}{\alpha_{x,y}} \cdot |u+y\rangle \otimes \left(\mathsf{Eval}_x^\dagger \cdot (\mathbb{I}_{\mathcal{P}\times\mathcal{W}_x} \otimes |y\rangle \langle y|_{\mathcal{W}_y}) \cdot \mathsf{Eval}_x \cdot |\psi,\emptyset,\emptyset\rangle\right) \\ &= |u+y\rangle \otimes |\psi_{x,y}\rangle\end{aligned}$$

In summary,

$$\mathsf{EvalAndCopy}\,|x,u\rangle_{\mathcal{Q}_x\times\mathcal{Q}_u} \otimes |\psi_{x,y}\rangle = |x,u+y\rangle_{\mathcal{Q}_x\times\mathcal{Q}_u} \otimes |\psi_{x,y}\rangle$$

□

Let S_x be the span of $\{|\psi_{x,y}\rangle\}_{y\in\mathcal{Y}}$, and let $S_{\perp,x}$ be the subspace of S_x that is orthogonal to $|\psi,\emptyset,\emptyset\rangle$. Since $|\psi_{x,y}\rangle \in S_x$, $|\psi_{x,y}\rangle$ can be written as a superposition of $|\psi,\emptyset,\emptyset\rangle$ and a state $|\psi_{x,y}^\perp\rangle \in S_{\perp,x}$.

For programs $P_1 = (|\psi^1\rangle, \mathsf{Eval}_1, R_1)$ and $P_2 = (|\psi^2\rangle, \mathsf{Eval}_2, R_2)$, let us use superscripts or subscripts for the variables defined above. For example, let $|\psi_{x,y}^1\rangle$ and $|\psi_{x,y}^2\rangle$ be the state $|\psi_{x,y}\rangle$ defined for programs P_1 and P_2, respectively. Let $\mathsf{ReflectAndCopy}_1, \mathsf{EvalAndCopy}_1, \mathsf{ReflectAndCopy}_2, \mathsf{EvalAndCopy}_2$ be defined analogously.

For each x, let us define a unitary U_x that maps

$$U_x : |\psi_{x,y}^1\rangle \mapsto |\psi_{x,y}^2\rangle$$

for all $y \in \mathcal{Y}$. Such a unitary exists because $\{|\psi_{x,y}^1\rangle\}_y$ and $\{|\psi_{x,y}^2\rangle\}_y$ are each orthonormal. Next, let us define CU to be a controlled version of U_x that reads the value of x from $\mathcal{Q}_x$ and applies U_x to $\mathcal{P} \times \mathcal{W}_x \times \mathcal{W}_y$.

Claim 9. *For any $x \in \mathcal{X}$, $U_x \cdot |\psi^1,\emptyset,\emptyset\rangle = |\psi^2,\emptyset,\emptyset\rangle$.*

Proof. First, for any (x,y), $\alpha_{x,y}^1 = \sqrt{\Pr[P_1(x)\to y]} = \sqrt{\Pr[P_2(x)\to y]} = \alpha_{x,y}^2$.

Second,

$$\begin{aligned}U_x \cdot |\psi^1,\emptyset,\emptyset\rangle &= U_x \cdot \sum_y \alpha_{x,y}^1 |\psi_{x,y}^1\rangle \\ &= \sum_y \alpha_{x,y}^1 \cdot U_x \cdot |\psi_{x,y}^1\rangle \\ &= \sum_y \alpha_{x,y}^1 \cdot |\psi_{x,y}^2\rangle \\ &= \sum_y \alpha_{x,y}^2 |\psi_{x,y}^2\rangle \\ &= |\psi^2,\emptyset,\emptyset\rangle\end{aligned}$$

□

Now let us define some hybrids to transform $\mathsf{Sim}'(P_1)$ into $\mathsf{Sim}'(P_2)$.

Hybrid 1. $\mathsf{Sim}'(P_1)$

Hybrid 2. At a high level, in this hybrid we operate on initial state $|\psi^1\rangle$ but on each query it transforms the state into $|\psi^2\rangle$ and acts with the P_2 evaluation unitary. An important detail is that transforming from $|\psi^1\rangle$ to $|\psi^2\rangle$ can only be done with respect to some x – the x-value given as input to the query.

- Initialize the program register with $|\psi^1\rangle$.
- On queries of the form $|x, u, b\rangle\,|D\rangle$ with $x' \in D$ such that $x' \neq x$, act as the identity and skip the following steps.
- Apply CU, which applies U_x to $\mathcal{P} \times \mathcal{W}_x \times \mathcal{W}_y$ controlled on the value x written on $\mathcal{Q}_x$.
- Apply the unitary

$$\mathsf{Bitflip} \circ \mathsf{ReflectAndCopy}_2 \circ \mathsf{EvalAndCopy}_2 \circ \mathsf{ReflectAndCopy}_2$$

- Apply CU†.

Hybrid 3. $\mathsf{Sim}'(P_2)$

Before showing the indistinguishability of these hybrids, we make the following claim.

Claim 10. *At the end of any query in hybrids 1 and 2, the database and program registers maintain the following invariant: $D = \emptyset$ if and only if $\mathcal{P} \times \mathcal{W}_x \times \mathcal{W}_y$ contain the initial program state $|\psi^1, \emptyset, \emptyset\rangle$, and $D = x$ if and only if $\mathcal{P} \times \mathcal{W}_x \times \mathcal{W}_y$ are in some state $|\phi\rangle \in S^1_{\perp,x}$.*

Proof. We first prove this invariant for hybrid 1 using induction. For the base case: the oracle is initialized in the state $|\emptyset\rangle_{\mathcal{D}}\,|\psi^1, \emptyset, \emptyset\rangle_{\mathcal{P}\times\mathcal{W}_x\times\mathcal{W}_y}$, so the invariant is satisfied to begin with. For the inductive case, let us assume that at the start of a query, the invariant is satisfied. We will show that the invariant is still satisfied at the end of the query.

First, if $\mathcal{Q}_x = |x\rangle$, $\mathcal{D} = |x'\rangle$, and $x' \notin \{\emptyset, x\}$, then this query aborts, and the state on $\mathcal{D} \times \mathcal{P} \times \mathcal{W}_x \times \mathcal{W}_y$ still satisfy the invariant.

Next, let us consider the case where $\mathcal{Q}_x = |x\rangle$, and $\mathcal{D}$ contains either x or $\emptyset$. Let us also assume that the state satisfies the invariant at the start of the query. First, by lemma 18, ReflectAndCopy acts as follows.

$$\begin{aligned}\mathsf{ReflectAndCopy} : |\emptyset\rangle\,|\psi^1, \emptyset, \emptyset\rangle &\mapsto |x\rangle\,|\psi^1, \emptyset, \emptyset\rangle \\ |x\rangle\,|\phi\rangle &\mapsto |x\rangle\,|\phi\rangle\end{aligned}$$

where $|\phi\rangle \in S^1_{\perp,x}$. Therefore, at the end of ReflectAndCopy, $\mathcal{D} = |x\rangle$, and $\mathcal{P} \times \mathcal{W}_x \times \mathcal{W}_y$ are in S^1_x.

Second, EvalAndCopy is applied. When EvalAndCopy is applied to $|x\rangle_{\mathcal{D}}\,|\psi^1_{x,y}\rangle_{\mathcal{P}\times\mathcal{W}_x\times\mathcal{W}_y}$, it does not change the state on the $\mathcal{D} \times \mathcal{P} \times \mathcal{W}_x \times \mathcal{W}_y$ registers (by claim 8). This implies that EvalAndCopy maps states in S^1_x to states in S^1_x. Additionally, EvalAndCopy does not touch $\mathcal{D}$. Therefore, at the end of EvalAndCopy, $\mathcal{D}$ still contains x, and $\mathcal{P} \times \mathcal{W}_x \times \mathcal{W}_y$ still lies in S^1_x.

Third, we apply ReflectAndCopy again. At the start of this operation, $\mathcal{D} \times \mathcal{P} \times \mathcal{W}_x \times \mathcal{W}_y$ is in the span of $|x\rangle\,|\psi^1, \emptyset, \emptyset\rangle$ and $|x\rangle\,|\phi\rangle$ for states $|\phi\rangle \in S^1_{\perp,x}$. ReflectAndCopy acts on these basis states as follows:

$$\begin{aligned}\mathsf{ReflectAndCopy} : |x\rangle\,|\psi^1, \emptyset, \emptyset\rangle &\mapsto |\emptyset\rangle\,|\psi^1, \emptyset, \emptyset\rangle \\ |x\rangle\,|\phi\rangle &\mapsto |x\rangle\,|\phi\rangle\end{aligned}$$

Therefore, the state at the end of this operation is in the span of states that satisfy the invariant.

Fourth, we apply Bitflip. This operation acts only on $\mathcal{Q}_b$, so the state at the end of this operation will still satisfy the invariant.

A very similar argument also shows that this invariant holds for hybrid 2. □

Claim 11. *At the start of any invocation of* $\mathsf{EvalAndCopy}_1$ *in hybrid 1, the state of* $\mathcal{P} \times \mathcal{W}_x \times \mathcal{W}_y$ *is in* S^1_x*, where* x *is the value written on* $\mathcal{Q}_x$*.*

Proof. Claim 10 says that at the start of any query, the state of $\mathcal{D} \times \mathcal{P} \times \mathcal{W}_x \times \mathcal{W}_y$ is in $|\emptyset\rangle\,|\psi^1, \emptyset, \emptyset\rangle$ or $|x'\rangle\,|\phi\rangle$ for some $|\phi\rangle \in S^1_{\perp,x'}$. In the second case, we can assume that $x' = x$ because otherwise, the query will immediately abort, and $\mathsf{EvalAndCopy}_1$ will not be executed.

Next, the query to $\mathsf{Sim}'(P)$ applies $\mathsf{ReflectAndCopy}_1$, which acts as follows (lemma 18):

$$\begin{aligned}\mathsf{ReflectAndCopy}_1 : |\emptyset\rangle\,|\psi^1, \emptyset, \emptyset\rangle &\mapsto |x\rangle\,|\psi^1, \emptyset, \emptyset\rangle \\ |x\rangle\,|\phi\rangle &\mapsto |x\rangle\,|\phi\rangle\end{aligned}$$

for any $|\phi\rangle \in S^1_{\perp,x}$. In either case, $\mathsf{ReflectAndCopy}_1$ maps states in S^1_x to states in S^1_x.

Next, the only time that $\mathsf{EvalAndCopy}_1$ is invoked is right after the first invocation of ReflectAndCopy. We've shown that at the start of $\mathsf{EvalAndCopy}_1$, the state of $\mathcal{P} \times \mathcal{W}_x \times \mathcal{W}_y$ is in S^1_x. □

Claim 12. *Hybrids 1 and 2 are perfectly indistinguishable.*

Proof. First, we make the following observations:

1. $\mathsf{CU}^\dagger \circ \mathsf{Bitflip} \circ \mathsf{CU} = \mathsf{Bitflip}$. This is because Bitflip acts only on $\mathcal{Q}_b$, and CU acts only on $\mathcal{Q}_x \times \mathcal{P} \times \mathcal{W}_x \times \mathcal{W}_y$. Since they act on disjoint registers, they commute, so $\mathsf{CU}^\dagger \circ \mathsf{Bitflip} \circ \mathsf{CU} = \mathsf{CU}^\dagger \circ \mathsf{CU} \circ \mathsf{Bitflip} = \mathsf{Bitflip}$.

2. $\mathsf{CU}^\dagger \circ \mathsf{ReflectAndCopy}_2 \circ \mathsf{CU} = \mathsf{ReflectAndCopy}_1$.

   First, $\mathsf{ReflectAndCopy}_2$ is equivalent to the operation that applies CNOT from $\mathcal{Q}_x$ to $\mathcal{D}$ controlled on $\mathcal{P} \times \mathcal{W}_x \times \mathcal{W}_y \times \mathcal{W}_b$ being in the state $|\psi^2, \emptyset, \emptyset, 0\rangle$ (lemma 18). Second, CU acts as follows (claim 9):

   $$|x\rangle_{\mathcal{Q}_x}\,|\psi^1, \emptyset, \emptyset, 0\rangle \to |x\rangle_{\mathcal{Q}_x}\,|\psi^2, \emptyset, \emptyset, 0\rangle$$

   That implies that $\mathsf{CU}^\dagger \circ \mathsf{ReflectAndCopy}_2 \circ \mathsf{CU}$ applies CNOT from $\mathcal{Q}_x$ to $\mathcal{D}$ controlled on the state $|\psi^1, \emptyset, \emptyset, 0\rangle$. This is the same as $\mathsf{ReflectAndCopy}_1$.

3. For every x, y,

   $$\begin{aligned}\mathsf{EvalAndCopy}_1(|x, u\rangle\,|\psi^1_{x,y}\rangle) &= |x, u + y\rangle\,|\psi^1_{x,y}\rangle \\ &= \mathsf{CU}^\dagger \circ \mathsf{EvalAndCopy}_2(|x, u\rangle\,|\psi^2_{x,y}\rangle) \\ &= \mathsf{CU}^\dagger \circ \mathsf{EvalAndCopy}_2 \circ \mathsf{CU}(|x, u\rangle\,|\psi^1_{x,y}\rangle)\end{aligned}$$

4. For every x, y, and every state $|\phi\rangle \in S^1_x$, we have that

   $$\mathsf{EvalAndCopy}_1(|x, u\rangle\,|\phi\rangle) = \mathsf{CU}^\dagger \circ \mathsf{EvalAndCopy}_2 \circ \mathsf{CU}(|x, u\rangle\,|\phi\rangle)$$

   This follows from the previous observation (Item 3).

By claim 11, in hybrid 1, $\mathsf{EvalAndCopy}_1$ is only invoked on states in S^1_x, and on these states, $\mathsf{EvalAndCopy}_1$ acts the same as $\mathsf{CU}^\dagger \circ \mathsf{EvalAndCopy}_2 \circ \mathsf{CU}$. Therefore, in hybrid 1, we can replace $\mathsf{EvalAndCopy}_1$ with $\mathsf{CU}^\dagger \circ \mathsf{EvalAndCopy}_2 \circ \mathsf{CU}$, and the change will be perfectly indistinguishable.

Let us put everything together. We can replace

$$\mathsf{Bitflip} \circ \mathsf{ReflectAndCopy}_1 \circ \mathsf{EvalAndCopy}_1 \circ \mathsf{ReflectAndCopy}_1$$

from hybrid 1 with

$$\mathsf{CU}^\dagger \circ \mathsf{Bitflip} \circ \mathsf{CU} \circ \mathsf{CU}^\dagger \circ \mathsf{ReflectAndCopy}_2 \circ \mathsf{CU} \circ \mathsf{CU}^\dagger \circ \mathsf{EvalAndCopy}_2 \circ \mathsf{CU} \circ \mathsf{CU}^\dagger \circ \mathsf{ReflectAndCopy}_2 \circ \mathsf{CU}$$

Then when we cancel out adjacent applications of CU and $\mathsf{CU}^\dagger$, the operation becomes:

$$\mathsf{CU}^\dagger \circ \mathsf{Bitflip} \circ \mathsf{ReflectAndCopy}_2 \circ \mathsf{EvalAndCopy}_2 \circ \mathsf{ReflectAndCopy}_2 \circ \mathsf{CU}$$

This is the sequence of operations found in hybrid 2. Therefore, hybrids 1 and 2 are perfectly indistinguishable.

□

Claim 13. *Hybrids 2 and 3 are perfectly indistinguishable.*

Proof. In hybrid 2, the first application of CU during the first query to the simulator converts $|\psi^1, \emptyset, \emptyset\rangle$ into $|\psi^2, \emptyset, \emptyset\rangle$, which is consistent with the initial state of $|\psi^2, \emptyset, \emptyset\rangle$ in hybrid 3. We would now like to argue that the applications of CU and $\mathsf{CU}^\dagger$ cancel each other, but this is not true in general: in between queries, the adversary can act on the input query register, so that we are effectively applying a $U_{x'} \circ U_x^\dagger$ operation on the internal state of the oracle, which is not the identity operation in general.

We make use of the invariant from Claim 10: In every branch of the superposition of hybrid 2, after the end of a query, either $D = \emptyset$ and $\mathcal{P} \times \mathcal{W}_x \times \mathcal{W}_y = |\psi^1, \emptyset, \emptyset\rangle$, or D contains x, and $\mathcal{P} \times \mathcal{W}_x \times \mathcal{W}_y$ contains $|\phi\rangle \in S^1_{\perp,x}$.

In the branch with $|\psi^1, \emptyset, \emptyset\rangle$, CU acts the same no matter what the control register is, and always transforms $|\psi^1, \emptyset, \emptyset\rangle$ to $|\psi^2, \emptyset, \emptyset\rangle$, In this branch, the applications of CU and $\mathsf{CU}^\dagger$ cancel each other. In the branch with $|\phi\rangle\,|x\rangle$, where $|\phi\rangle \in S^1_{\perp,x}$, if $x' \neq x$, the next query acts as the identity on this branch, so we can ignore it. □

This shows that $\mathsf{Sim}'(P_1)$ and $\mathsf{Sim}'(P_2)$ are perfectly indistinguishable and completes the proof of Lemma 20. □

Lemma 21 (Simulation of the CSEQ oracle)**.** *Given any (possibly oracle-aided) testable quantum program P that implements f with 0 error, an oracle for $\mathsf{Sim}'(P)$ (definition 28) is perfectly indistinguishable from O_f^{CSEQ} to any adversary making an unbounded number of quantum queries.*

Proof. First, Sim' only needs black-box access to the Eval and R operations of P. It does not access $|\psi\rangle$ directly. P may even be oracle-aided, and the oracle may maintain an internal pure state, which is considered part of $|\psi\rangle$.

Second, P_f (defined in lemma 19) and P are testable quantum programs. P_f implements f with 0 error. Lemma 19 says that $\mathsf{Sim}'(P_f)$ and O_f^{CSEQ} are perfectly indistinguishable after an unbounded number of quantum queries.

Third, since P also implements f with 0 error, lemma 20 implies that oracles for $\mathsf{Sim}'(P_f)$ and $\mathsf{Sim}'(P)$ are perfectly indistinguishable after an unbounded number of quantum queries. Therefore, $\mathsf{Sim}'(P)$ and O_f^{CSEQ} are also perfectly indistinguishable after an unbounded number of quantum queries. □

Remark 2. *For classical functionalities, any testable OTP compiler that satisfies CSEQ security is best-possible among testable programs, in the sense of definition 16. This follows from Lemma 21.*

Theorem 15.

1. *For any sets of bitstrings $\mathcal{X}, \mathcal{R}, \mathcal{Y}$, there exists a Q.P.T. simulator Sim_1 such that for every randomized function $f : \mathcal{X} \times \mathcal{R} \to \mathcal{Y}$, O_f^{CSEQ} and $\mathsf{Sim}_1^{O_{\Phi_f}^{\mathsf{SEQ}}}$ are perfectly indistinguishable.*
2. *For any sets of bitstrings $\mathcal{X}, \mathcal{R}, \mathcal{Y}$, **for which** $|\mathcal{X}| > 1$, there exists a Q.P.T. simulator Sim_2 such that for every randomized function $f : \mathcal{X} \times \mathcal{R} \to \mathcal{Y}$, $\mathsf{Sim}_2^{O_f^{\mathsf{CSEQ}}}$ and $O_{\Phi_f}^{\mathsf{SEQ}}$ are perfectly indistinguishable.*

Proof.

Simulating O_f^{CSEQ} with Sim_1: Let Sim' be the simulator from definition 28. By lemma 21, if Sim' is given black-box access to any testable quantum program P that implements f with 0 error, then $\mathsf{Sim}'(P)$ is perfectly indistinguishable from O_f^{CSEQ}. The program P may be oracle-aided, and the oracle may maintain an internal pure state.

Next, $O_{\Phi_f}^{\mathsf{SEQ}}$ can be used to construct such a testable quantum program that implements f with 0 error (lemma 22). Composing these two procedures yields Sim_1.

Simulating $O_{\Phi_f}^{\mathsf{SEQ}}$ with Sim_2: Sim_2 mainly uses the simulator Sim' from definition 17 (this is a different Sim' than the one used to construct Sim_1). This Sim' takes any testable quantum program that implements Φ_f and perfectly simulates $O_{\Phi_f}^{\mathsf{SEQ}}$ (lemma 8). The program can be oracle-aided, and the oracle can even maintain an internal state. Sim' only needs black-box access to the program's Eval and R operations.

Next, we can use O_f^{CSEQ} to construct an (oracle-aided) testable quantum program that implements Φ_f as long as $|\mathcal{X}| > 1$ (lemma 23). Then if we run Sim' on this program, it will perfectly simulate $O_{\Phi_f}^{\mathsf{SEQ}}$. □

Lemma 22. *$O_{\Phi_f}^{\mathsf{SEQ}}$ can be used to construct an (oracle-aided) testable quantum program that implements f with 0 error.*

The program is oracle-aided since it makes queries to $O_{\Phi_f}^{\mathsf{SEQ}}$, and the program state includes the internal state of the oracle.

Proof. Let us construct a testable quantum program $P = (|\psi\rangle, \mathsf{Eval}, \mathsf{R})$ that implements f.

- Program State $|\psi\rangle$: Let the program state be the internal state of $O_{\Phi_f}^{\mathsf{SEQ}}$ concatenated with a cache register $\mathcal{B}$ that is initialized to $|0\rangle$. Note that the internal state of $O_{\Phi_f}^{\mathsf{SEQ}}$ is defined as a pure state, so the initial program state $|\psi\rangle$ is indeed pure.
- Eval: The program's Eval operation simply queries $O_{\Phi_f}^{\mathsf{SEQ}}$. Eval takes an external query register $\mathcal{Q}$ with basis states of the form $|x, u\rangle$ where $(x, u) \in \mathcal{X} \times \mathcal{Y}$. Then Eval queries $O_{\Phi_f}^{\mathsf{SEQ}}$ on $\mathcal{Q} \times \mathcal{B}$:

  $$|x, u\rangle_{\mathcal{Q}} \otimes |b\rangle_{\mathcal{B}} \to |x, u \oplus y\rangle_{\mathcal{Q}} \otimes |b \oplus b'\rangle_{\mathcal{B}}$$

  Finally, Eval outputs $\mathcal{Q}$.
- R: The reflection operation R takes an external register $\mathcal{E}$. If $\mathcal{B}$ stores 0, then R applies X to $\mathcal{E}$.

Claim 14. *The program P defined above implements f with 0 error (definition 10).*

Proof. Given a query of the form $|x, 0\rangle$, the program's Eval operation queries $O_{\Phi_f}^{\mathsf{SEQ}}$ on $|x, 0, b\rangle$. The first time $O_{\Phi_f}^{\mathsf{SEQ}}$ is queried, it applies the unitary U_{Φ_f}, which uses a compressed oracle to compute $y = f(x; r)$ for a uniformly random $r \xleftarrow{\$} \mathcal{R}$. This is exactly the output distribution of $f(x)$. □

The following claim shows that P is indeed testable.

Claim 15. *After any number of* Eval *and R operations, applying R is equivalent to applying the function $\mathsf{X} \otimes |\psi\rangle\langle\psi| + I \otimes (I - |\psi\rangle\langle\psi|)$ to $\mathcal{E}$ and the current program state.*

Proof. Here is some intuition for why R works as the reflection oracle. First, $O_{\Phi_f}^{\mathsf{SEQ}}$'s internal state includes a register $\mathcal{C}$ that is $|0\rangle$ if and only if $O_{\Phi_f}^{\mathsf{SEQ}}$ is in its initial state. Second, any time that $\mathcal{C}$ is changed, $O_{\Phi_f}^{\mathsf{SEQ}}$ flips the output bit b. This implies that $\mathcal{C}$ and $\mathcal{B}$ always store the same value after any query, and that $\mathcal{B} = |0\rangle$ if and only if the program state is in its initial state. Next, we will make this argument formal.

The program state comprises three registers: $\mathcal{C}$ and $\mathcal{P}$, which contain $O_{\Phi_f}^{\mathsf{SEQ}}$'s state, and $\mathcal{B}$. The initial state of $\mathcal{B} \times \mathcal{C} \times \mathcal{P}$ is $|\psi\rangle := |0\rangle \otimes |0\rangle \otimes |0^m\rangle$.

We will show that after any number of Eval and R operations, $\mathcal{B} = |0\rangle$ if and only if $\mathcal{B} \times \mathcal{C} \times \mathcal{P} = |\psi\rangle$.

After any number of Eval and R operations, $\mathcal{B}$ and $\mathcal{C}$ store the same computational-basis value. First, $\mathcal{B} \times \mathcal{C}$ are initialized to $|0\rangle \otimes |0\rangle$. Next, the only time that $\mathcal{C}$'s computational-basis value changes is potentially during an Eval operation, specifically during step 2 of the query to $O_{\Phi_f}^{\mathsf{SEQ}}$. If $\mathcal{C}$ is not flipped in step 2, then this call to $O_{\Phi_f}^{\mathsf{SEQ}}$ acts as the identity, and $\mathcal{B}$ is not flipped either. On the other hand, if $\mathcal{C}$ is flipped in step 2, then this call to $O_{\Phi_f}^{\mathsf{SEQ}}$ applies U_{Φ_f} or $U_{\Phi_f}^\dagger$, which are equal according to claim 16. U_{Φ_f} flips the value of b. Since the program records the value of b on register $\mathcal{B}$, that means that $\mathcal{B}$ is flipped on any query that flips $\mathcal{C}$. Therefore, after any number of Eval or R operations, $\mathcal{B} \times \mathcal{C}$ store $(0, 0)$ or $(1, 1)$.

After any number of Eval and R operations, if $\mathcal{C} = |0\rangle$ then $\mathcal{P} = |0^m\rangle$. First, $\mathcal{C} \times \mathcal{P}$ are initialized to $|0\rangle \otimes |0^m\rangle$, so the invariant is initially true. Second, R does not change the state of $\mathcal{C} \times \mathcal{P}$. Third, the first Eval query flips $\mathcal{C}$ to $|1\rangle$ with certainty. If any future Eval query flips $\mathcal{C}$ back to $|0\rangle$, then the check in step 2 of $O_{\Phi_f}^{\mathsf{SEQ}}$ must have found that $\mathcal{P} = |0^m\rangle$ and flipped $\mathcal{C}$ to $|0\rangle$. Then step 3 would have acted as the identity because $\mathcal{C} = |0\rangle$, so at the end of the query, it is still true that $\mathcal{P} = |0^m\rangle$. This argument extends to show that after any number of queries, either $\mathcal{C} = |1\rangle$ or $\mathcal{C} \times \mathcal{P} = |0\rangle \otimes |0^m\rangle$.

The previous discussion implies that after any number of Eval and R operations, if $\mathcal{B} = |0\rangle$, then $\mathcal{C} = |0\rangle$, and $\mathcal{P} = |0^m\rangle$, so $\mathcal{B} \times \mathcal{C} \times \mathcal{P} = |\psi\rangle$. Next, if $\mathcal{B} = |1\rangle$, then $\mathcal{B} \times \mathcal{C} \times \mathcal{P} \neq |\psi\rangle$. Therefore, $\mathcal{B} = |0\rangle$ if and only if $\mathcal{B} \times \mathcal{C} \times \mathcal{P} = |\psi\rangle$.

Finally, R applies X to $\mathcal{E}$ if $\mathcal{B} = |0\rangle$. This is equivalent to applying X to $\mathcal{E}$ if $\mathcal{B} \times \mathcal{C} \times \mathcal{P}$ is in state $|\psi\rangle$.

□

Claim 16. *For the unitary U_{Φ_f} defined in definition 27, $U_{\Phi_f}^\dagger = U_{\Phi_f}$.*

Proof. First, steps 1 - 3 of U_{Φ_f} are each their own inverses. Step 1 CNOTs x from $\mathcal{Q}$ to $\mathcal{W}$, which can be uncomputed by applying step 1 a second time.

Step 2 queries the compressed oracle. The compressed oracle is perfectly indistinguishable from a random oracle when queried as a black box, and any query to a random oracle can be uncomputed by querying the random oracle a second time. Therefore, step 2 can be uncomputed by computing step 2 a second time.

Step 3 CNOTs some values that are computed from $\mathcal{W}$ onto $\mathcal{Q}$. This can be uncomputed by applying step 3 a second time.

Second, U_{Φ_f} and $U_{\Phi_f}^\dagger$ apply the following sequences of operations:

$$\begin{aligned}
U_{\Phi_f} &= \text{step } 1^\dagger \cdot \text{step } 2^\dagger \cdot \text{step } 3 \cdot \text{step } 2 \cdot \text{step } 1 \\
U_{\Phi_f}^\dagger &= \left(\text{step } 1^\dagger \cdot \text{step } 2^\dagger \cdot \text{step } 3 \cdot \text{step } 2 \cdot \text{step } 1\right)^\dagger \\
&= \text{step } 1^\dagger \cdot \text{step } 2^\dagger \cdot \text{step } 3^\dagger \cdot \left(\text{step } 2^\dagger\right)^\dagger \cdot \left(\text{step } 1^\dagger\right)^\dagger \\
&= \text{step } 1^\dagger \cdot \text{step } 2^\dagger \cdot \text{step } 3 \cdot \text{step } 2 \cdot \text{step } 1 \\
&= U_{\Phi_f}
\end{aligned}$$

□

□

Lemma 23. *If $|\mathcal{X}| > 1$, then O_f^{CSEQ} can be used to construct an (oracle-aided) testable quantum program that implements Φ_f.*

Proof. Let us construct a testable quantum program $P = (|\psi\rangle, \mathsf{Eval}, \mathsf{R})$ that implements Φ_f.

- Program State $|\psi\rangle$: Let the program state $|\psi\rangle$ be the internal state of O_f^{CSEQ} concatenated with several work registers. Let $x_0, x_1 \in \mathcal{X}$ be two distinct values, and then initialize the work registers as follows:

  $$\mathcal{Q}_0 = |x_0, 0, 0\rangle, \mathcal{Q}_1 = |x_1, 0, 0\rangle, \mathcal{B}_0 = |0\rangle, \mathcal{B}_1 = |0\rangle$$

  Note that the internal state of O_f^{CSEQ} is defined as a pure state, so $|\psi\rangle$ is indeed pure.

- Eval: The program's Eval operation simply queries O_f^{CSEQ}.

- R: The reflection operation R should test whether the program state is in its initial state. Part of the program state is contained in the oracle O_f^{CSEQ}, so our construction cannot access it directly. Instead, we can make queries to O_f^{CSEQ} to test if its internal state is in its initial state. Our strategy is to query O_f^{CSEQ} on two different inputs, while uncomputing between queries. If the oracle answers both queries, then its state began in the initial state. If it rejects at least one query, then its database must already record a query.

  Formally, R takes as input a single-qubit register $\mathcal{E}$ and acts as follows.

  1. Query O_f^{CSEQ} on $\mathcal{Q}_0$. The resulting state is $|x_0, y_0, b_0\rangle_{\mathcal{Q}_0}$ for some $y_0 \in \mathcal{Y}$ and $b_0 \in \{0, 1\}$.
  2. Copy b_0 onto $\mathcal{B}_0$.
  3. Uncompute item 1.
  4. Query O_f^{CSEQ} on $\mathcal{Q}_1$. The resulting state is $|x_1, y_1, b_1\rangle_{\mathcal{Q}_1}$ for some $y_1 \in \mathcal{Y}$ and $b_1 \in \{0, 1\}$.
  5. Copy b_1 onto $\mathcal{B}_1$.
  6. Uncompute item 4.
  7. If $|b_0, b_1\rangle_{\mathcal{B}_0 \times \mathcal{B}_1} = |1, 1\rangle$, then apply X (bitflip) to register $\mathcal{E}$.
  8. Uncompute steps 1 - 6.

Claim 17. *P implements Φ_f (with 0 error).*

Proof. When we apply Eval to the user's query register $\mathcal{Q}$ and the initial program state $|\psi\rangle$, it queries O_f^{CSEQ}. Since the database of O_f^{CSEQ} is initialized to $|\emptyset\rangle$, step 1 of O_f^{CSEQ} does nothing during this query.

If we omit step 1 of O_f^{CSEQ}, then the remaining steps are exactly the same as U_{Φ_f}. Therefore, applying Eval with program state $|\psi\rangle$ computes the same operation as Φ_f. □

The following claim shows that P is indeed testable.

Claim 18. *After any number of* Eval *and R operations, applying R is equivalent to applying the function $\mathsf{X} \otimes |\psi\rangle\langle\psi| + I \otimes (I - |\psi\rangle\langle\psi|)$ to $\mathcal{E}$ and the current program state.*

Proof. Let us consider the case where P's program state is in its initial state $|\psi\rangle$. We will show that in this case, R applies X to $\mathcal{E}$ and restores the program state to its initial state.

Step 1 of R queries O_f^{CSEQ} on input $|x_0, 0, 0\rangle$. Since we are in the initial state, O_f^{CSEQ} responds to the query and flips b_0 to 1 with certainty. Step 2 copies b_0 to another register $\mathcal{B}_0$, but this step does not change the joint state on $\mathcal{Q}_0$ and the internal registers of O_f^{CSEQ} because the value $b_0 = 1$ is deterministic. Step 3 uncomputes step 1. Steps 1 and 3 act only on $\mathcal{Q}_0$ and the internal state of O_f^{CSEQ}, and the state on these registers is unchanged by step 2. Therefore the state of $\mathcal{Q}_0$ and the internal state of O_f^{CSEQ} is the same at the end of step 3 as it was at the start of step 1. The only difference in the program's state is that at the end of step 3, $\mathcal{B}_0$ contains $|1\rangle$.

Next, steps 4 - 6 are the same as steps 1 - 3 except the query register is $\mathcal{Q}_1 = |x_1, 0, 0\rangle$. By the same argument as before, the state at the end of step 6 is the same as the state at the start of step 1 except that $\mathcal{B}_0 = |1\rangle$ and $\mathcal{B}_1 = |1\rangle$.

Next, step 7 applies X to $\mathcal{E}$ with certainty. This step does not change the state on the program's registers because the application of X occurs with probability 1. Finally, step 8 uncomputes all steps except for the application of X. At the end of R, the program state is the initial state $|\psi\rangle$, and the only change from the beginning of R is the application of X to $\mathcal{E}$.

Next, let us consider the case where the program state is orthogonal to $|\psi\rangle$. We only need to consider states that are reachable from a sequence of Eval and R operations. We will show that in this case, R acts as the identity.

If the program state is orthogonal to $|\psi\rangle$, then the database register $\mathcal{D}$ must contain a non-empty database D. The program state comprises the registers: $\mathcal{W} \times \mathcal{D}$, which are the internal state of O_f^{CSEQ}, as well as $\mathcal{Q}_0 \times \mathcal{Q}_1 \times \mathcal{B}_0 \times \mathcal{B}_1$. After any query to O_f^{CSEQ}, the $\mathcal{W}$ register is in the $|0\rangle$ state because any values written to it have been uncomputed during the query. Likewise, the state of $\mathcal{Q}_0 \times \mathcal{Q}_1 \times \mathcal{B}_0 \times \mathcal{B}_1$ at the start of any R operation is the same as its initial state (claim 19). After any number of queries to Eval and R, the only register that might not be in its initial state is $\mathcal{D}$. If the program state is orthogonal to $|\psi\rangle$, then $\mathcal{D}$ must contain a non-empty database. We will consider two types of non-empty databases: (1) every entry of D (the only entry, really) is of the form (x_0, r) for some $r \in \mathcal{R}$, and (2) D contains (x', r) for some $x' \neq x_0$ and some $r \in \mathcal{R}$.

Now let us step through R in the first case, where every entry of D is of the form (x_0, r) for some $r \in \mathcal{R}$. Step 1 queries O_f^{CSEQ} on input $|x_0, 0, 0\rangle$, and the query is answered ($b_0 = 1$). Step 2 copies b_0 to $\mathcal{B}_0$, and step 3 uncomputes step 1. The state of the program at the end of step 3 is the same as the state at the beginning of step 1 except that $\mathcal{B}_0 = |1\rangle$. Next, step 4 queries O_f^{CSEQ} on x_1. This query is rejected ($b_1 = 0$) because D contains (x_0, r). By step 7, $\mathcal{B}_0 \times \mathcal{B}_1 = |1, 0\rangle$, so step 7 acts as the identity. Finally, step 8 uncomputes steps 1 - 6, so R acts as the identity on the program state.

Finally, let us step through R in the second case, where D contains (x', r) for some $x' \neq x_0$ and some $r \in \mathcal{R}$. Then the query to O_f^{CSEQ} in step 1 will be rejected ($b_0 = 0$). Skipping ahead, at the start of step 7, $\mathcal{B}_0 = |0\rangle$, so this step acts as the identity. Finally, step 8 uncomputes steps 1 - 6, so R acts as the identity on the program state.

In summary, we've shown that for every program state that is reachable by a sequence of Eval and R operations, if the program state is orthogonal to $|\psi\rangle$, then R acts as the identity on the program state. □

Claim 19. *After any number of* Eval *and R operations, the work registers are in their initial state:*

$$\mathcal{Q}_0 = |x_0, 0, 0\rangle, \mathcal{Q}_1 = |x_1, 0, 0\rangle, \mathcal{B}_0 = |0\rangle, \mathcal{B}_1 = |0\rangle$$

Proof. Before any Eval and R operations have been executed, the work registers $\mathcal{Q}_0 \times \mathcal{Q}_1 \times \mathcal{B}_0 \times \mathcal{B}_1$ are in their initial state. Next, Eval does not modify the work registers. Finally, it suffices to analyze an R operation and show that if the work registers are in their initial state at the start of the operation, then they will return to their initial state at the end of the operation.

Let us split up the program's registers into two groups. Let register $\mathcal{R}$ comprise all components of the work registers $\mathcal{Q}_0 \times \mathcal{Q}_1 \times \mathcal{B}_0 \times \mathcal{B}_1$ except the values x_0 and x_1 written on $\mathcal{Q}_0$ and $\mathcal{Q}_1$ respectively. Let $\mathcal{S}$ comprise the internal registers of O_f^{CSEQ} and the values x_0 and x_1 written on $\mathcal{Q}_0$ and $\mathcal{Q}_1$.

Next, no step of R or O_f^{CSEQ} modifies the computational basis value of x_0 and x_1 written on $\mathcal{Q}_0$ and $\mathcal{Q}_1$. Therefore these values remain in unchanged by R.

Steps 1 - 6 of R may modify register $\mathcal{R}$, but they only do so by CNOT-ing a value onto $\mathcal{R}$ that was computed from the state on register $\mathcal{S}$. Next, step 7 applies an X operation to $\mathcal{E}$ controlled on the value of $\mathcal{R}$. However, step 7 does not change the state of $\mathcal{S}$. Finally, step 8 uncomputes steps 1 - 6. Again, this entails CNOT-ing a value onto $\mathcal{R}$ that is computed from the state on $\mathcal{S}$. The value that is CNOT-ed in step 8 is the same as the value that was CNOT-ed during steps 1 - 6 because the state on $\mathcal{S}$ is unchanged. Therefore step 8 returns the state of $\mathcal{R}$ to its initial state.

In summary, all components of $\mathcal{Q}_0 \times \mathcal{Q}_1 \times \mathcal{B}_0 \times \mathcal{B}_1$ are returned to their initial state at the end of R. □

□

C Impossibility of one-time correct sampling QSIO

In this section, we show that our impossibility for a best-possible one-time compiler in Section 3 rules out the existence of obfuscation schemes that satisfy a natural definition of quantum state indistinguishability obfuscation for *sampling* programs. Previously, quantum state indistinguishability obfuscation has been studied only in the setting where the quantum program implements a deterministic classical functionality with negligible error [BKNY23, GM24].

A natural generalization of the definition of quantum state IO to quantum programs designed for sampling tasks is the following. It requires obfuscations of any two programs that implement the same sampling task to be indistinguishable. Crucially, for two programs to be equal in this sense, they need to evaluate the same distributions on the first query, but there are no guarantees on their equivalence on subsequent queries. A priori, one can image that it might be possible to indistinguishably obfuscate such a pair of programs by enforcing some kind of one-time guarantee, so both obfuscations stop working after one evaluation. The impossibility in this section (Corollary 3) says that this is not possible. Indeed, if it was possible, then it would be a best-possible one-time program. This is formalized in Lemma 24.

We also give a more direct proof of this impossibility, without going through the intermediate primitive of best-possible one-time programs. This proof is simpler to describe and will hopefully shed light on the core idea of the impossibility.

We leave open the question of coming up with a feasible notion of quantum state IO for sampling programs. Our notion of stateful obfuscation in Section 5.3 (a notion of quantum state IO where the two programs must have the same behavior on polynomially many (forward and inverse) queries, instead of just the first one) can be seen as an attempt to make progress in this direction.

Definition 29 (Quantum State Obfuscation for Quantum Sampling Programs)**.** *A quantum state obfuscator for quantum sampling programs is a q.p.t. algorithm* QObf *with the following syntax:*

$$\mathsf{QObf}(1^\lambda, \rho, C) \to (|\tilde{\psi}\rangle, \tilde{C}).$$

The obfuscator takes as input a security parameter 1^λ *and a quantum program* $(|\psi\rangle, C)$ *and outputs an obfuscated circuit* $(|\tilde{\psi}\rangle, \tilde{C})$*.*

- ***Correctness:*** *Suppose a family of quantum sampling programs* $\{|\psi_\lambda\rangle, C_\lambda\}_{\lambda\in\mathbb{N}}$ *implements a family of sampling functionalities* $\{D_\lambda : \{0,1\}^{n(\lambda)} \to \{0,1\}^{m(\lambda)}\}_{\lambda\in\mathbb{N}}$ *for every* $x \in \{0,1\}^{n(\lambda)}$ *up to negligible error* $\mathsf{negl}(\lambda)$*. Then, the family of quantum sampling programs* $\{|\tilde{\psi}_\lambda\rangle, \tilde{C}_\lambda\}$ *where*

  $$(|\tilde{\psi}_\lambda\rangle, \tilde{C}_\lambda) \leftarrow \mathsf{QObf}(1^\lambda, |\psi_\lambda\rangle, C_\lambda)$$

  also implements the same sampling functionality up to negligible error.

- ***Indistinguishability Obfuscation:*** *For every pair of families of quantum sampling programs* $\{|\psi_{\lambda,0}\rangle, C_{\lambda,0}\}_{\lambda\in\mathbb{N}}$ *and* $\{|\psi_{\lambda,1}\rangle, C_{\lambda,1}\}_{\lambda\in\mathbb{N}}$ *that both implement the same sampling functionality* $\{D_\lambda : \{0,1\}^{n(\lambda)} \to \{0,1\}^{m(\lambda)}\}_{\lambda\in\mathbb{N}}$ *up to negligible error in* λ*, and furthermore satisfy that the program descriptions have the same length* $||\psi_{\lambda,0}\rangle, C_{\lambda,0}| = ||\psi_{\lambda,1}\rangle, C_{\lambda,1}|$ *for every* λ*, for every q.p.t. adversary* $\{A_\lambda\}_{\lambda\in\mathbb{N}}$*,*

  $$\left|\Pr\left[1 \leftarrow A_\lambda\left(1^\lambda, \mathsf{QObf}(1^\lambda, |\psi_{\lambda,0}\rangle, C_{\lambda,0})\right)\right] - \Pr\left[1 \leftarrow A_\lambda\left(1^\lambda, \mathsf{QObf}(1^\lambda, |\psi_{\lambda,1}\rangle, C_{\lambda,1})\right)\right]\right| \leq \mathsf{negl}(\lambda).$$

Lemma 24. *Suppose there exists a quantum state indistinguishability obfuscator for quantum sampling programs for a class of classical functionalities randomized* $\mathcal{F}$*. Then, there must exist a one-time compiler that satisfies the best-possible one-time security guarantee of Definition 12 for* $\mathcal{F}$*.*

Proof. By assumption, let QObf be an obfuscator that satisfies quantum state indistinguishability obfuscation for quantum sampling programs for $\mathcal{F}$. Consider the following quantum sampling program $P_f = (|\psi_f\rangle, C_f)$ that implements classical randomized functions $f : \mathcal{X} \times \mathcal{R} \to \mathcal{Y}$ for $f \in \mathcal{F}$: the program state is a description of f, along with a uniform superposition over $\mathcal{R}$,

$$|\psi_f\rangle = |f\rangle \otimes \sum_{r\in\mathcal{R}} |r\rangle.$$

The circuit C_f maps

$$|x, u\rangle_{\mathcal{Q}} \otimes |r, f\rangle_{\mathcal{P}} \mapsto |x \oplus f(x; r)\rangle_{\mathcal{Q}} \otimes |r, f\rangle_{\mathcal{P}}.$$

Without loss of generality, we assume that all functionalities in $\mathcal{F}$ are padded to the same length. Define $\mathsf{OTP}^*(1^\lambda, f) := \mathsf{QObf}(1^\lambda, P_f)$ for $f \in \mathcal{F}$. We claim that OTP^* satisfies the best-possible one-time security guarantee of Definition 12. For every q.p.t. adversary $\mathcal{A}$, define a corresponding $\mathsf{Sim}(1^\lambda, P) \to \mathcal{A}(1^\lambda, \mathsf{QObf}(P))$. Then, for every $f \in \mathcal{F}$, for every program P that implements f, by the security of quantum state indistinguishability obfuscation, it must hold that for all q.p.t. distinguishers D,

$$\left|\Pr[1 \leftarrow D(1^\lambda, f, \mathcal{A}(\mathsf{OTP}^*(1^\lambda, f)))] - \Pr[1 \leftarrow D(1^\lambda, f, \mathsf{Sim}(1^\lambda, P))]\right|$$

Otherwise, there exists some $f \in \mathcal{F}$ and some program P that implements f such that the following two distributions are distinguishable:

$$\mathcal{A}(\mathsf{OTP}^*(1^\lambda, f)) = \mathcal{A}(\mathsf{QObf}(1^\lambda, P_f)) \text{ and}$$
$$\mathsf{Sim}(1^\lambda, P) = \mathcal{A}(1^\lambda, \mathsf{QObf}(P)),$$

even though P_f and P implement the same classical randomized functionality f. □

Corollary 3. *There exists a class of sampling functionalities $\mathcal{D}$ such that there is no quantum state indistinguishability obfuscator (as defined in Definition 29) for $\mathcal{D}$.*

Alternate proof of Corollary 3. Consider the following families of classical circuits.

1. $C_{\mathsf{pk}_{\mathsf{lossy}},r^*}(x;r) := \mathsf{Enc}(\mathsf{pk}_{\mathsf{lossy}}, 0; r^*)$, where $\mathsf{pk}_{\mathsf{lossy}}$ is sampled as a lossy encryption key.
2. $D_{\mathsf{pk}_{\mathsf{lossy}}}(x;r) := \mathsf{Enc}(\mathsf{pk}_{\mathsf{lossy}}, x; r)$
3. $E_{\mathsf{pk}_{\mathsf{inj}}}(x;r) := \mathsf{Enc}(\mathsf{pk}_{\mathsf{inj}}, x; r)$

We assume that the obfuscator preserves functionality, so in the case of the C circuits (which are deterministic and constant), evaluation does not entangle the input and output registers. We can test entanglement between the input and output registers by evaluating on the uniform superposition of inputs $|+\rangle := \sum_x |x\rangle$, measuring the output registers, and then checking that the input register is still in the $|+\rangle$ state.

In the E programs, however, by the correctness of the encryption scheme, evaluation must create entanglement the input and output registers. Since the only difference in the D and E circuits is that the public key is sampled from the lossy and injective modes respectively, they are indistinguishable, and therefore evaluation of D must also entangle the input and output registers.

However, consider the following two programs:

- Program description contains the mixed state $\sum_{r^*} |\mathsf{Enc}(\mathsf{pk}_{\mathsf{lossy}}, 0; r^*)\rangle \langle \mathsf{Enc}(\mathsf{pk}_{\mathsf{lossy}}, 0; r^*)|$, and the evaluation algorithm simply copies the contents of the program register onto the output register.
- Program description is simple the classical circuit $D_{\mathsf{pk}_{\mathsf{lossy}}}$ and evaluation algorithm is to evaluate this classical circuit on fresh randomness by producing $|+\rangle_{\mathcal{R}}$.

For every input x, these programs have the same output distributions. By padding, we can ensure their descriptions have the same length. So if we had IO for sampling quantum circuits (as in Definition 29), the obfuscations should be indistinguishable. However, they are not: The obfuscation of the first program is going to be a mixture of obfuscations over $C_{\mathsf{pk}_{\mathsf{lossy}},r^*}$, where the mixture is taken over r^*. Since each element of the mixture does not entangle the input and output registers, so does the overall mixture. But as we have already seen, the obfuscation of D must be entangling the input and output registers. □

D A Generic Multi-Observable Attack against Testable Programs

This section formalizes the attack sketched in the discussion section: a testable program can always be used to estimate arbitrarily many output observables on many inputs.

Theorem 16 (Marriott–Watrous Empirical Estimator [MW05, proof of Theorem 4, Fig. 2])**.** *Fix a verifier unitary A and projectors Π_1, Δ_1 as in the proof of [MW05, Theorem 4], and define*

$$Q := (I \otimes \langle 0^k|)A^\dagger \Pi_1 A (I \otimes |0^k\rangle).$$

For any integer $N \geq 1$, the Marriott–Watrous procedure B (proof of [MW05, Theorem 4]) outputs bits $z_1, \ldots, z_N \in \{0,1\}$. If the input witness is an eigenvector of Q with eigenvalue $p \in [0,1]$, then

$$\Pr[(z_1, \ldots, z_N) = z] = p^{w(z)}(1-p)^{N-w(z)}$$

for every $z \in \{0,1\}^N$ (see the analysis around Fig. 2 in the proof of [MW05, Theorem 4]). Consequently, for

$$\widetilde{p} := \frac{1}{N}\sum_{\ell=1}^{N} z_\ell,$$

we have

$$\Pr[|\widetilde{p} - p| > \alpha] \leq 2e^{-2N\alpha^2}$$

for every $\alpha > 0$. In particular, for $N = \Theta(\alpha^{-2}\log(1/\beta))$, $\widetilde{p}$ is an (α, β) additive estimator of p.

Corollary 4 (Gentleness of Laplace Noise Measurement [AR19a, Corollary 6])**.** *Let L_σ denote the Laplace-noise measurement applied to n registers. Then L_σ is $O(\sqrt{n}/\sigma)$-gentle on product states.*

We use the following finite-outcome wrapper abstracted from the camera-ready theorem statement [AR19a, Theorem 7] and the full-version derivation [AR19b, Proposition 55, Theorem 56, and §7.3 (proof of Theorem 7)].

Theorem 17 (Safe-Use Wrapper for Finite-Outcome Estimation)**.** *Fix a finite outcome set $\mathcal{Y}$ and a classical post-processing rule $\mathcal{C} : \mathcal{Y} \to \{\text{pass}, \text{fail}\}$. Suppose $\mathcal{A}$ is an estimation subroutine whose output lies in $\mathcal{Y}$. Then there is a compiled subroutine $\widetilde{\mathcal{A}}$ with two guarantees:*

1. ***Accuracy/copy complexity.*** *If there exists an (α, β) additive estimator for the same quantity that uses n copies of the input state, then $\widetilde{\mathcal{A}}$ uses*

$$n + \mathsf{poly}(1/\alpha, 1/\beta)$$

copies and matches that estimator's output with probability at least $1 - \beta$.

2. ***Coherent safety with side information.*** *For every joint pure state*

$$|\Psi\rangle = \sum_i c_i \, |u_i\rangle_W \, |v_i\rangle_S \,,$$

where S is arbitrary side information, define

$$p_i := \Pr[\mathcal{C}(\mathcal{A}(|u_i\rangle)) = \text{pass}], \qquad p := \sum_i |c_i|^2 p_i.$$

If $\widetilde{\mathcal{A}}$ is applied to register W and the post-processing output is pass*, then the post-measurement joint state $|\widetilde{\Psi}\rangle$ obeys*

$$\left\| |\Psi\rangle - |\widetilde{\Psi}\rangle \right\|_2 \leq \alpha(1-p)^2.$$

Proof. [AR19a, Theorem 7] (camera-ready version) gives the qualitative statement that such estimation subroutines can be used coherently inside larger quantum computations. For the quantitative wrapper we use the full-version derivation: [AR19b, proof of Theorem 7 in §7.3] gives the coherent estimate-and-uncompute template (including the explicit copy scaling parameter), while [AR19b, Proposition 55 and Theorem 56] formalize the garbage-uncomputation/QSampling step. To obtain the finite-outcome form above, encode $\mathcal{A}$ coherently so that its output register stores $y \in \mathcal{Y}$, append the predicate $\mathcal{C}(y)$, and treat "pass" as the acceptance event. Under this encoding, the full-version safe-subroutine argument yields item (2), with p_i equal to the pass probability on component $|u_i\rangle$ and $p = \sum_i |c_i|^2 p_i$. Item (1) is the corresponding polynomial-overhead consequence of that full-version analysis, instantiated with an (α, β) reference estimator using n copies. □

Lemma 25 (From Joint ℓ_2-Closeness to Reduced Trace Distance)**.** *Let $|\Psi\rangle, |\widetilde{\Psi}\rangle$ be pure states on registers WS, and let*

$$\rho_W := \mathsf{Tr}_S \, |\Psi\rangle\langle\Psi| \,, \qquad \widetilde{\rho}_W := \mathsf{Tr}_S \, |\widetilde{\Psi}\rangle\langle\widetilde{\Psi}| \,.$$

Then

$$\frac{1}{2}\|\rho_W - \widetilde{\rho}_W\|_1 \leq \frac{1}{2}\left\| |\Psi\rangle\langle\Psi| - |\widetilde{\Psi}\rangle\langle\widetilde{\Psi}| \right\|_1 \leq \left\| |\Psi\rangle - |\widetilde{\Psi}\rangle \right\|_2 .$$

Proof. The first inequality is contractivity of trace distance under partial trace. For pure states, the middle term equals $\sqrt{1-|\langle\Psi \mid \widetilde{\Psi}\rangle|^2}$, which is at most $\| \, |\Psi\rangle - |\widetilde{\Psi}\rangle \, \|_2$. □

Lemma 26 (Multi-observable estimation attack)**.** *Let $P = (|\psi\rangle, \mathsf{Eval}, R)$ be a testable quantum program that one-time implements a channel Φ. Let $x_1, \ldots, x_n$ be efficiently preparable input states, and let $O_1, \ldots, O_t$ be efficiently measurable Hermitian observables on the output register with $\|O_i\|_\infty \leq 1$. Then for every $\varepsilon, \delta > 0$, there is a QPT attacker with oracle access to $(\mathsf{Eval}, \mathsf{Eval}^\dagger, R)$ that outputs values $\widehat{\mu}_{i,j}$ for all $i \in [t], j \in [n]$ such that*

$$\Pr\left[\forall i \in [t], j \in [n] : |\widehat{\mu}_{i,j} - \mathsf{Tr}\left(O_i \Phi(x_j)\right)| \leq \varepsilon\right] \geq 1 - \delta.$$

The attacker runs in time $\mathsf{poly}(|P|, t, n, 1/\varepsilon, 1/\delta)$.

Proof. Let $N := tn$, and index the pairs $(i, j) \in [t] \times [n]$ in any fixed order

$$(i_1, j_1), \ldots, (i_N, j_N).$$

For each i, define the two-outcome effect

$$E_i := (I + O_i)/2.$$

Since $\|O_i\|_\infty \leq 1$ and O_i is Hermitian, we have $0 \preceq E_i \preceq I$. For each input x_j, define

$$p^\star_{i,j} := \mathsf{Tr}(E_i \Phi(x_j)), \qquad \mu^\star_{i,j} := \mathsf{Tr}(O_i \Phi(x_j)) = 2p^\star_{i,j} - 1.$$

Single-call primitive from MW + Laplace. Fix (i, j) and parameters $\alpha, \beta, \tau > 0$. Because O_i is efficiently measurable and $0 \preceq E_i \preceq I$, the two-outcome POVM $\{E_i, I - E_i\}$ has an efficient coherent (Naimark) implementation.

We define a verifier $A_{i,j}$ that first checks whether the program register is in $|\psi\rangle$ (using R), and rejects otherwise; conditioned on passing that check, it applies Eval on input x_j, performs the coherent two-outcome test for E_i on the output register, and uncomputes with $\mathsf{Eval}^\dagger$. For this verifier, the corresponding MW operator has the form

$$Q_{i,j} = \Pi_\psi M_{i,j} \Pi_\psi,$$

where $\Pi_\psi = |\psi\rangle\langle\psi|$ and $M_{i,j}$ is the acceptance effect of the coherent E_i-test under Eval. Hence $|\psi\rangle$ is an eigenvector of $Q_{i,j}$ with eigenvalue $p^\star_{i,j}$.

Let m_0 be the number of MW rounds used by the underlying (pre-compiler) estimator. Now define $\mathsf{Est}^{\alpha,\beta,\tau}_{i,j}$:

1. Refresh: measure $\{\Pi_\psi, I - \Pi_\psi\}$ using R. If reject, return a failure flag.
2. Conditioned on refresh success (so the program state is exactly $|\psi\rangle$), run m_0 MW rounds for $A_{i,j}$ and write outcome bits to fresh registers $Z_1, \ldots, Z_{m_0}$.
3. Let $S = \sum_{\ell=1}^{m_0} Z_\ell$, and sample $\eta \sim \mathrm{Lap}(\sigma)$. If $|\eta| > \alpha m_0/4$, return failure. Otherwise compute

$$\widetilde{p}_{\mathrm{cont}} = \mathrm{clip}_{[0,1]}\left(\frac{S + \eta}{m_0}\right),$$

then output the quantized value

$$\widetilde{p} := Q_r(\widetilde{p}_{\mathrm{cont}}),$$

where Q_r rounds to the nearest point of $\mathcal{Y}_r = \{0, 2^{-r}, \ldots, 1\}$ and r is chosen so $2^{-r} \leq \alpha/4$.

Because refresh-success inputs are exactly $|\psi\rangle$ and $|\psi\rangle$ is an eigenvector of $Q_{i,j}$ with eigenvalue $p^\star_{i,j}$, Theorem 16 gives i.i.d. Bernoulli bits with mean $p^\star_{i,j}$. Therefore, for the underlying (pre-compiler) call,

$$\begin{aligned}\Pr\left[\text{failure flag or } \left|\widetilde{p}-p^\star_{i,j}\right|>\alpha\right] &\le \Pr\left[\frac{|\eta|}{m_0}>\frac{\alpha}{4}\right]+\Pr\left[\left|\widetilde{p}-p^\star_{i,j}\right|>\alpha \ \wedge\ \frac{|\eta|}{m_0}\le\frac{\alpha}{4}\right]\\ &\le \Pr\left[\left|\frac{S}{m_0}-p^\star_{i,j}\right|>\frac{\alpha}{4}\right]+\Pr\left[\frac{|\eta|}{m_0}>\frac{\alpha}{4}\right]\\ &\le 2e^{-m_0\alpha^2/8}+e^{-\alpha m_0/(4\sigma)},\end{aligned}$$

where the second inequality uses that, on the event

$$\left|\frac{S}{m_0}-p^\star_{i,j}\right|\le\frac{\alpha}{4}\quad\text{and}\quad\frac{|\eta|}{m_0}\le\frac{\alpha}{4},$$

we have

$$\left|\widetilde{p}_{\text{cont}}-p^\star_{i,j}\right|\le\frac{\alpha}{2},$$

and then quantization contributes at most $2^{-r}\le\alpha/4$, so

$$\left|\widetilde{p}-p^\star_{i,j}\right|\le\frac{3\alpha}{4}<\alpha.$$

The output alphabet is finite ($\mathcal{Y}_r\cup\{\perp\}$), as required by Theorem 17.

Let

$$q:=\Pr[|\eta|>\alpha m_0/4]=e^{-\alpha m_0/(4\sigma)}.$$

Because the failure flag is triggered exactly by the event $|\eta|>\alpha m_0/4$, and η is sampled independently of the program register, the pass probability is exactly $1-q$ for every component in Theorem 17(2).

Choosing

$$\sigma=\Theta(\sqrt{m_0}/\tau),\qquad m_0=\Theta\left(\frac{1}{\alpha^2}\log\frac{1}{\beta}+\frac{1}{\alpha^2\tau^2}\log^2\frac{1}{\beta\tau}\right),$$

ensures

$$2e^{-m_0\alpha^2/8}+q\le\beta/3,\qquad q\le\sqrt{\tau}.$$

Thus the reference estimator has call-failure probability at most $\beta/3$.

Now apply Theorem 17 directly to this reference estimator with continuation rule

$$\mathcal{C}(y)=\begin{cases}\text{pass} & y\neq\perp,\\ \text{fail} & y=\perp.\end{cases}$$

and compiler parameters $(1/2,\beta/3)$. Let $\widetilde{\mathcal{A}}_{i,j}$ denote the compiled subroutine. By Theorem 17(1), $\widetilde{\mathcal{A}}_{i,j}$ disagrees with the reference estimator with probability at most $\beta/3$, so by union bound

$$\Pr[\widetilde{\mathcal{A}}_{i,j}\text{ call fails}]\le\beta/3+\beta/3<\beta.$$

For disturbance, consider any joint pure input state as in Theorem 17(2). Because the continuation event is exactly $|\eta|\le\alpha m_0/4$, independent of the program component, every component has pass probability $1-q$. Hence $p_i=1-q$ for all i, so $p=1-q$. Conditioned on pass, Theorem 17(2) gives

$$\left\||\Psi\rangle-|\widetilde{\Psi}\rangle\right\|_2\le\frac{1}{2}(1-p)^2=\frac{q^2}{2}\le\tau.$$

Applying Lemma 25 yields the same τ bound on reduced program-register trace distance. Therefore the compiled subroutine controls disturbance for the *entire* call (MW rounds plus Laplace readout), even with arbitrary side information.

Adding the AR19 compiler overhead

$$m_{\mathrm{wrap}} = \mathsf{poly}(1/\alpha, 1/\beta, 1/\tau),$$

yields:

1. per-call failure probability (failure flag or additive error $> \alpha$) at most β;
2. post-call program-state disturbance at most τ from $|\psi\rangle\langle\psi|$ on refresh-success executions;
3. time/query complexity

$$\mathsf{poly}(|P|, m_0 + m_{\mathrm{wrap}}) = \mathsf{poly}(|P|, 1/\alpha, 1/\beta, 1/\tau).$$

Attack algorithm. Set

$$\alpha := \varepsilon/4, \qquad \beta := \delta/(2N), \qquad \tau := \delta/(2N).$$

For $k = 1, \ldots, N$, run $\mathsf{Est}_{i_k,j_k}^{\alpha,\beta,\tau}$. If it returns failure, halt and output failure. Otherwise, with returned value $\widetilde{p}_k$, output

$$\widehat{\mu}_{i_k,j_k} := 2\widetilde{p}_k - 1.$$

Failure probability. The first refresh succeeds with probability 1 because the initial program state is $|\psi\rangle$. After a successful call, property (2) above gives

$$\frac{1}{2}\left\|\rho' - |\psi\rangle\langle\psi|\right\|_1 \le \tau.$$

Hence the next refresh fails with probability at most

$$1 - \mathsf{Tr}(\Pi_\psi \rho') \le \frac{1}{2}\left\|\rho' - |\psi\rangle\langle\psi|\right\|_1 \le \tau.$$

Each successful-refresh call has call-failure probability at most β. By union bound over all N calls, total failure probability is at most

$$N\tau + N\beta = \delta.$$

Accuracy on success. Condition on no failure. Then every call satisfies

$$\left|\widetilde{p}_k - p^\star_{i_k,j_k}\right| \le \alpha.$$

Therefore, for each k,

$$\left|\widehat{\mu}_{i_k,j_k} - \mu^\star_{i_k,j_k}\right| = 2\left|\widetilde{p}_k - p^\star_{i_k,j_k}\right| \le 2\alpha = \varepsilon/2 < \varepsilon.$$

So all $N = tn$ estimates are simultaneously ε-accurate with probability at least $1 - \delta$.

Finally, total running time and query complexity are

$$N \cdot \mathsf{poly}(|P|, 1/\alpha, 1/\beta, 1/\tau) = \mathsf{poly}(|P|, t, n, 1/\varepsilon, 1/\delta).$$

□